LLM INTERVIEW

LLM 算法学习手册

36 道高频题 · 原理推导 / 核心公式 / 张量流程图 / 代码实现 / 面试要点

👁️ 注意力机制 · 📏 归一化 · 📍 位置编码 · 🔗 前馈网络 · 📉 损失函数 · ⚙️ 优化器 · 🎮 强化学习 · 🎯 参数高效微调 · 💾 高效训练 · ⚡ 推理加速 · 🎲 采样策略 · 📐 模型架构 · 📖 基础

13 个主题 · 36 篇题解 · 生成于 2026-10-11

目录

👁️ 注意力机制 • Attention

01 因果掩码　02 Flash Attention　03 分组查询注意力　04 KV Cache　05 多头潜在注意力　06 多头注意力
07 缩放点积注意力　08 稀疏注意力

📏 归一化 • Normalization

09 层归一化　10 RMS 归一化

📍 位置编码 • Position

11 旋转位置编码

🔗 前馈网络 • FFN

12 标准前馈网络　13 混合专家模型　14 SwiGLU

📉 损失函数 • Loss

15 交叉熵 / 语言模型损失　16 DAPO　17 DPO 损失　18 GRPO 损失　19 组序列策略优化　20 在线策略蒸馏
21 PPO 损失　22 在线自蒸馏

⚙️ 优化器 • Optimizer

23 AdamW 优化器　24 Muon 优化器

🎮 强化学习 • RL

25 广义优势估计

🎯 参数高效微调 • PEFT

26 LoRA

💾 高效训练 • Efficient

27 FP8 分块量化训练　28 梯度检查点　29 混合精度训练

⚡ 推理加速 • Inference

30 分页注意力　31 投机解码

🎲 采样策略 • Sampling

32 采样策略

🏗️ 模型架构 • Architecture

33 Decoder-Only Transformer　34 多 Token 预测

📖 基础 • Basics

35 激活函数　36 梯度与反向传播

👁️ 注意力机制 Attention　8 题

因果掩码

Causal Mask

下三角矩阵，防止看到未来信息

📌 原理与思想

核心概念

在 decoder 中使用下三角矩阵作为 mask，使得位置 i 只能关注位置 ≤i 的 token。这是自回归生成的基础，相比无 mask 的注意力，确保模型在训练时不会"看到未来"。

核心思想

通过下三角矩阵屏蔽未来位置的信息，使每个位置只能 attend 到当前及之前的 token。被屏蔽的位置填充 -inf，softmax 后变为 0，从而实现因果约束。

算法步骤

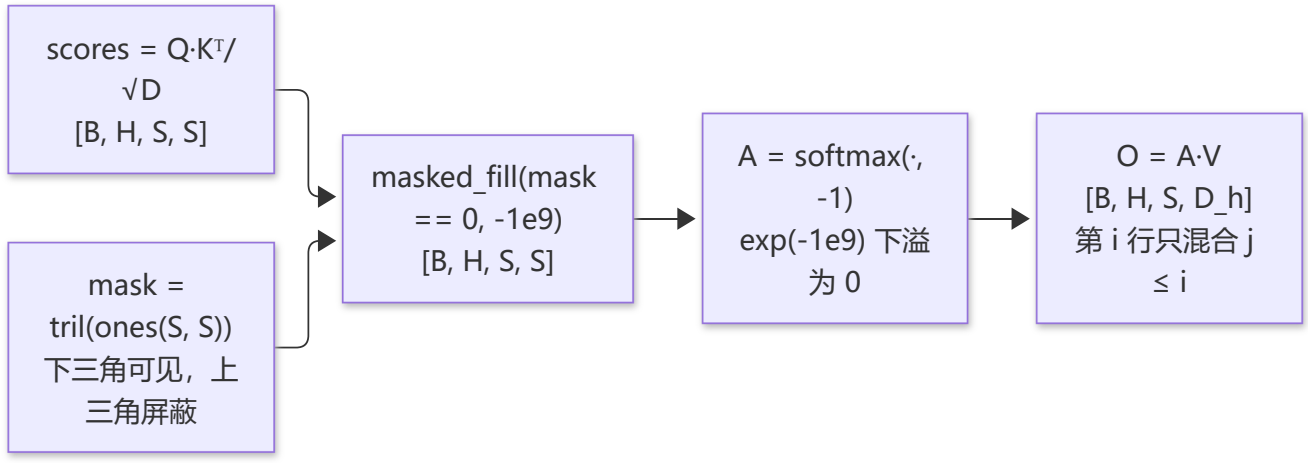
1. 创建下三角矩阵：mask = torch.tril(torch.ones(S, S))
2. 调整维度：mask = mask.unsqueeze(0).unsqueeze(0) → [1, 1, S, S]
3. 在注意力计算中应用：scores.masked_fill(mask == 0, -inf)
4. Softmax 归一化：attn = softmax(scores)

📐 核心公式

$$M_{ij} = \begin{cases} 1, & j \le i \\ 0, & j > i \end{cases} \qquad S'_{ij} = S_{ij} - \infty\,(1 - M_{ij})$$

$$A = \mathrm{softmax}(S'), \qquad A_{ij} = 0 \ \text{ for } \ j > i$$

📊 张量流程图



💻 代码实现

```python
import torch

def create_causal_mask(seq_len, device='cpu'):
    """创建因果掩码 (下三角)"""
    mask = torch.tril(torch.ones(seq_len, seq_len, device=device))
    return mask.unsqueeze(0).unsqueeze(0)  # [1, 1, S, S]

# 在 attention 中使用
def attention_with_causal_mask(q, k, v):
    B, H, S, D = q.shape
    mask = create_causal_mask(S, device=q.device)

    scores = torch.matmul(q, k.transpose(-2, -1)) / (D ** 0.5)
    scores = scores.masked_fill(mask == 0, float('-inf'))
    attn = torch.softmax(scores, dim=-1)
    return torch.matmul(attn, v)

# 测试
if __name__ == "__main__":
    mask = create_causal_mask(4)
    print(mask)
    # tensor([[[[1., 0., 0., 0.],
    #           [1., 1., 0., 0.],
    #           [1., 1., 1., 0.],
    #           [1., 1., 1., 1.]]]])
```

🎯 面试要点

- **torch.tril**: 生成下三角矩阵
- **被屏蔽位置填 -inf**: softmax 后为 0
- **decoder-only (GPT)**: 全程使用 causal mask

- **训练时可用 causal mask 并行计算所有位置的 loss**
- **面试常问**: 为什么训练时可以并行？因为 causal mask 保证了每个位置只能看到之前的 token
- **推理时**: 每步只有 1 个 token，掩码自动失效 —— 训练与推理走的是同一套代码

Flash Attention

分块计算 + Online Softmax → O(N) 显存

📌 原理与思想

核心概念

将 Q/K/V 分成块，在 SRAM 中完成注意力计算，避免将 O(N²) 的注意力矩阵写入 HBM。相比标准 Attention 的 O(N²) IO 复杂度，Flash Attention 将其降至 O(N²d/M)，大幅提升长序列训练/推理效率。

核心思想

利用 Online Softmax 算法，分块计算注意力而不需要存储完整的 N×N 注意力矩阵。GPU SRAM 快但小(20MB)，HBM 慢但大(40GB)，算法设计围绕减少 HBM 访问。

算法步骤

1. 将 Q 分成块，逐块处理
2. 对每个 Q 块，遍历所有 K/V 块
3. 计算当前块的 attention scores
4. Online Softmax 更新：维护最大值 m 和分母 l
5. 累加输出：O = O * exp(m_old - m_new) + P @ V_block
6. 最终归一化：O = O / l

📐 核心公式

$$\text{standard}: \quad \text{IO} = O(S^2)$$

$$\text{FlashAttention}: \quad \text{IO} = O\left(\frac{S^2 d}{M}\right), \qquad M = \text{SRAM size}$$

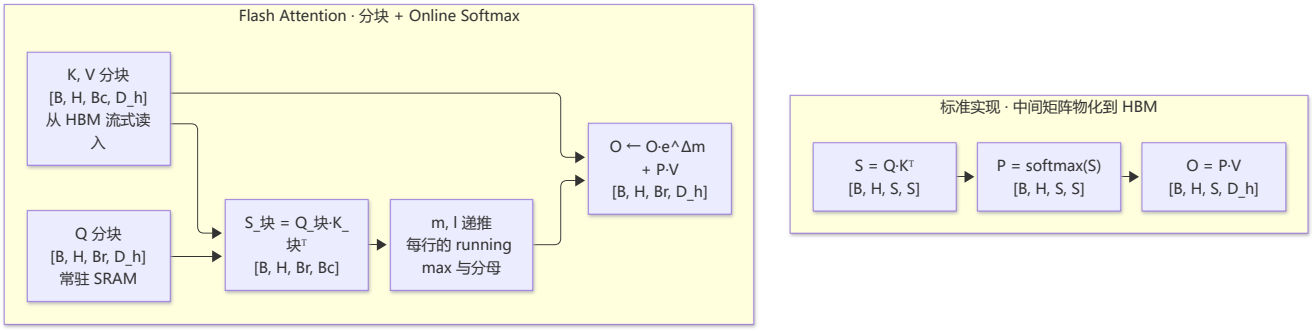
$$m^{(\text{new})} = \max\left(m^{(\text{old})},\ \text{rowmax}(S_{\text{blk}})\right)$$

$$\ell^{(\text{new})} = \ell^{(\text{old})} e^{\,m^{(\text{old})} - m^{(\text{new})}} + \text{rowsum}\left(e^{\,S_{\text{blk}} - m^{(\text{new})}}\right)$$

$$O^{(\text{new})} = O^{(\text{old})} e^{\,m^{(\text{old})} - m^{(\text{new})}} + e^{\,S_{\text{blk}} - m^{(\text{new})}} V_{\text{blk}}$$

$$O = \frac{O^{(\text{final})}}{\ell^{(\text{final})}}, \qquad \text{memory}: O(S^2) \to O(S)$$

📊 张量流程图



💻 代码实现

```
import torch
import math

def flash_attention(Q, K, V, block_size=64):
    """
    Flash Attention 简化实现 (核心思想)
    Q, K, V: [B, H, N, D]
    """
    B, H, N, D = Q.shape
    O = torch.zeros_like(Q)
    l = torch.zeros(B, H, N, 1, device=Q.device)
    m = torch.full((B, H, N, 1), float('-inf'), device=Q.device)

    for i in range(0, N, block_size):
        Qi = Q[:, :, i:i+block_size]
        oi = torch.zeros_like(Qi)
        li = torch.zeros(B, H, Qi.shape[2], 1, device=Q.device)
        mi = torch.full((B, H, Qi.shape[2], 1), float('-inf'), device=Q.device)

        for j in range(0, N, block_size):
            Kj = K[:, :, j:j+block_size]
            Vj = V[:, :, j:j+block_size]
            Sij = torch.matmul(Qi, Kj.transpose(-2, -1)) / math.sqrt(D)

            # Online Softmax 更新
            m_new = torch.maximum(mi, Sij.max(dim=-1, keepdim=True).values)
            exp_corr = torch.exp(mi - m_new)
            oi = oi * exp_corr
            li = li * exp_corr
            Pij = torch.exp(Sij - m_new)
            oi = oi + torch.matmul(Pij, Vj)
            li = li + Pij.sum(dim=-1, keepdim=True)
            mi = m_new

        O[:, :, i:i+block_size] = oi / li
    return O

# 测试
if __name__ == "__main__":
```

```
    B, H, N, D = 2, 8, 256, 64
    Q = torch.randn(B, H, N, D)
    K = torch.randn(B, H, N, D)
    V = torch.randn(B, H, N, D)

    output = flash_attention(Q, K, V, block_size=32)
    print(f"Output shape: {output.shape}")  # [2, 8, 256, 64]
```

🎯 面试要点

- **IO-aware**: GPU SRAM 快但小(20MB), HBM 慢但大(40GB)
- **Online Softmax**: 避免存储 N×N 注意力矩阵
- **IO 复杂度从 $O(N^2)$ 降到 $O(N^2d/M)$**
- **实际用 CUDA/Triton kernel 实现**，这里展示算法思想
- **Flash Attention 2/3** 进一步优化了并行度和 warp 级别操作
- **面试重点**: 理解分块计算和 Online Softmax 的思想

分组查询注意力

Grouped Query Attention, GQA

多 Q 头共享 KV 头，LLaMA 2 标配

📌 原理与思想

核心概念

MHA 和 MQA 的折中方案：Q 有 H 个头，KV 只有 G 个头 (G<H)。多个 Q 头共享同一组 KV 头，大幅减少 KV Cache。相比 MHA 节省推理显存，相比 MQA 保持更好的模型质量。

核心思想

通过让多个 Q 头共享同一组 KV 头，在保持模型表达能力的同时大幅减少 KV Cache 大小。核心操作是 repeat_kv：将 G 个 KV 头复制扩展为 H 个，以匹配 Q 的头数。

算法步骤

1. 线性投影：Q → [B, S, H, Dh]，K,V → [B, S, G, Dh]
2. 复制 KV 头：repeat_kv(K, H/G) → [B, S, H, Dh]
3. 转置维度：transpose(1, 2) → [B, H, S, Dh]
4. 计算缩放点积注意力
5. 合并多头并输出投影

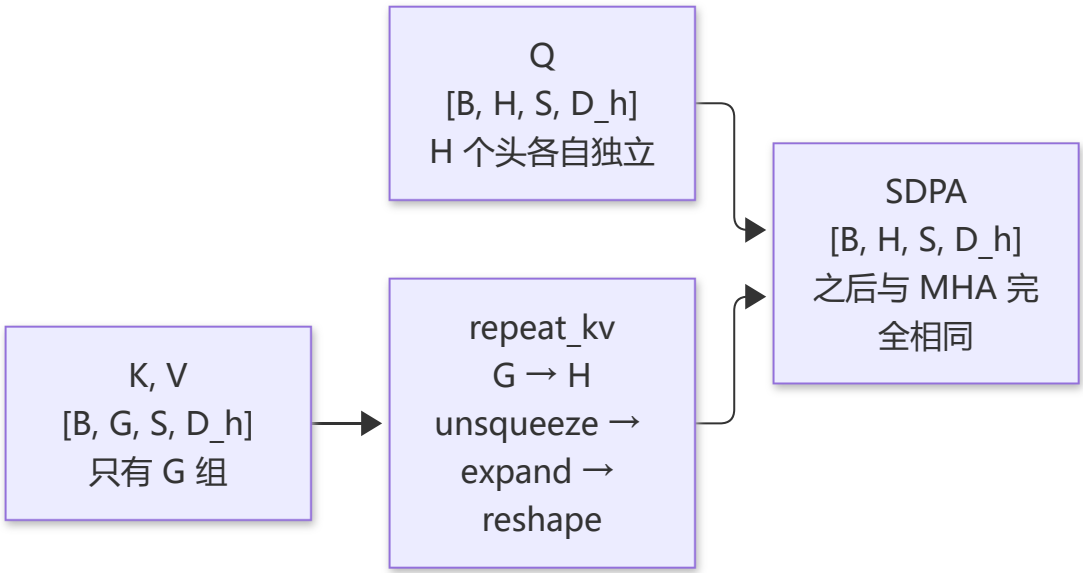
📐 核心公式

$$Q \in \mathbb{R}^{B\times H\times S\times D_h}, \qquad K, V \in \mathbb{R}^{B\times G\times S\times D_h}, \qquad 1 \le G \le H$$

$$\text{KV cache} = 2\, n_{\text{layers}}\, G\, S\, D_h\, b$$

$$\begin{aligned} G = H &\Rightarrow \text{MHA}, && \text{one KV per query head} \\ 1 < G < H &\Rightarrow \text{GQA}, && H/G \text{ query heads share one KV} \\ G = 1 &\Rightarrow \text{MQA}, && \text{all query heads share one KV} \end{aligned}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class GQA(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads):
        super().__init__()
        assert n_heads % n_kv_heads == 0
        self.h, self.g = n_heads, n_kv_heads
        self.dh = d_model // n_heads
        self.n_rep = n_heads // n_kv_heads

        self.wq = nn.Linear(d_model, n_heads * self.dh, bias=False)
        self.wk = nn.Linear(d_model, n_kv_heads * self.dh, bias=False)
        self.wv = nn.Linear(d_model, n_kv_heads * self.dh, bias=False)
        self.wo = nn.Linear(d_model, d_model)

    def repeat_kv(self, x, n_rep):
        """[B, G, S, Dh] → [B, H, S, Dh]"""
```

```
        if n_rep == 1:
            return x
        B, G, S, D = x.shape
        return (x[:, :, None, :, :]
                .expand(B, G, n_rep, S, D)
                .reshape(B, G * n_rep, S, D))

    def forward(self, x, mask=None):
        B, S, _ = x.shape
        q = self.wq(x).view(B, S, self.h, self.dh).transpose(1, 2)
        k = self.wk(x).view(B, S, self.g, self.dh).transpose(1, 2)
        v = self.wv(x).view(B, S, self.g, self.dh).transpose(1, 2)

        # 核心：复制 KV 以匹配 Q 的头数
        k = self.repeat_kv(k, self.n_rep)
        v = self.repeat_kv(v, self.n_rep)

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.dh)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        out = torch.matmul(F.softmax(scores, dim=-1), v)
        out = out.transpose(1, 2).contiguous().view(B, S, -1)
        return self.wo(out)

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 512
    x = torch.randn(B, S, D)
    gqa = GQA(D, n_heads=32, n_kv_heads=8)  # 4 个 Q 头共享 1 个 KV 头
    out = gqa(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 512]
    print(f"Output shape: {out.shape}") # [2, 10, 512]
```

> 🎯 **面试要点**

- **repeat_kv**: `unsqueeze → expand → reshape`，面试必写
- **KV Cache 减少到 G/H × 100%**（如 8/32 = 25%）
- **MHA: G=H; MQA: G=1; GQA: 1<G<H**
- **LLaMA 2 70B 用 GQA(8 groups)**，推理内存降 75%
- **训练速度几乎不受影响，推理速度显著提升**

KV Cache

缓存历史 KV，避免自回归重复计算

📌 原理与思想

核心概念

自回归生成时，每步只处理新 token，但需要与所有历史 token 做注意力。KV Cache 缓存历史的 K/V，避免重复计算。相比每步重新计算所有 token 的 O(N²) 复杂度，KV Cache 将其降至 O(N)，是推理加速的核心技术。

核心思想

空间换时间：缓存历史 token 的 K/V，每步只计算新 token 的 K/V，然后与缓存拼接。代价是额外的显存占用 O(L × H × Dh)。

算法步骤

1. **Prefill 阶段**：处理整个 prompt，计算并缓存所有 KV
2. **Decode 阶段**：每步只处理 1 个新 token
3. 计算新 token 的 Q, K, V
4. 拼接历史 KV：K = cat(K_cache, K_new)
5. 计算注意力：attn(Q, K, V)
6. 更新 cache，输出下一个 token

📐 核心公式

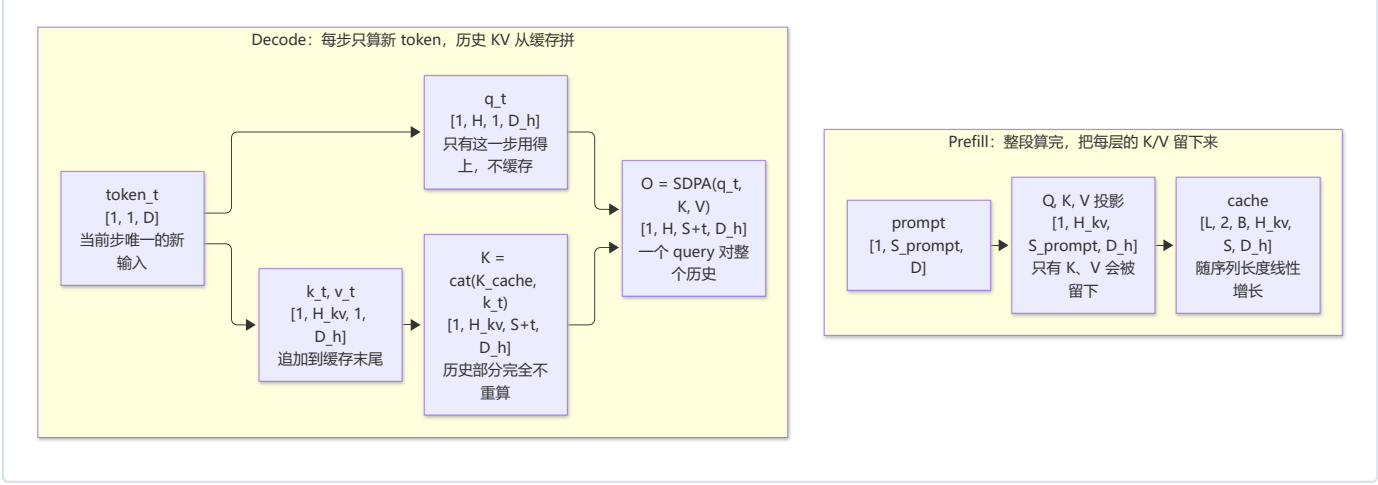
$$\text{KV cache} = 2\, n_{\text{layers}}\, n_{\text{kv}}\, S\, D_h\, b$$

$$\text{LLaMA 2 70B:}\quad 2 \times 80 \times 8 \times 4096 \times 128 \times 2\ \text{B} = 80\ \text{GB}$$

$$\text{decode}: \quad K \leftarrow \text{concat}(K,\, k_{\text{new}}), \qquad V \leftarrow \text{concat}(V,\, v_{\text{new}})$$

$$\text{no cache}: \sum_{t=1}^{S} t = O(S^2), \qquad \text{cache}: O(S)\ \text{per step}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class KVCacheAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.h, self.dh = n_heads, d_model // n_heads
        self.wq = nn.Linear(d_model, d_model, bias=False)
        self.wk = nn.Linear(d_model, d_model, bias=False)
        self.wv = nn.Linear(d_model, d_model, bias=False)
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, kv_cache=None):
        """
        x: [batch, seq_len, d_model]
        kv_cache: (cached_k, cached_v) 各为 [B, H, past_len, Dh]
        """
        B, S, _ = x.shape
        q = self.wq(x).view(B, S, self.h, self.dh).transpose(1, 2)
        k = self.wk(x).view(B, S, self.h, self.dh).transpose(1, 2)
        v = self.wv(x).view(B, S, self.h, self.dh).transpose(1, 2)

        # 核心：拼接历史 KV
        if kv_cache is not None:
            k = torch.cat([kv_cache[0], k], dim=2)
            v = torch.cat([kv_cache[1], v], dim=2)
        new_cache = (k, v)

        # 注意力计算
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.dh)
        out = torch.matmul(F.softmax(scores, dim=-1), v)
        out = out.transpose(1, 2).contiguous().view(B, S, -1)
        return self.wo(out), new_cache

# 测试
if __name__ == "__main__":
    B, S, D, H = 1, 10, 512, 8
    attn = KVCacheAttention(D, H)

    # Prefill: 处理整个 prompt
    x = torch.randn(B, S, D)
    out, cache = attn(x)
    print(f"Prefill output: {out.shape}")  # [1, 10, 512]
    print(f"Cache K shape: {cache[0].shape}")  # [1, 8, 10, 64]

    # Decode: 逐个生成 token
    new_token = torch.randn(B, 1, D)
    out, cache = attn(new_token, kv_cache=cache)
    print(f"Decode output: {out.shape}")  # [1, 1, 512]
    print(f"Cache K shape: {cache[0].shape}")  # [1, 8, 11, 64]
```

🎯 面试要点

- **空间换时间**: 缓存 O(L × H × Dh) 显存
- **Prefill 处理整个 prompt，Decode 逐个生成**
- **GQA 减少 KV Cache** (多个 Q 共享 KV)
- **长序列推理的主要显存瓶颈**
- **PagedAttention (vLLM)** 用分页管理 KV Cache 减少碎片
- **面试常问**: KV Cache 有多大？如何优化?
- **不缓存的代价**: 每步要对全部历史重算，累计 $\Sigma t = O(S^2)$；LLaMA 2 70B 在 4096 长度、fp16 下缓存约 80GB，比模型本身还大

多头潜在注意力

Multi-Latent Attention, MLA

KV 低秩压缩到潜空间，DeepSeek-V2 核心

📌 原理与思想

核心概念

将 KV 先下投影压缩到低维潜空间（存入 Cache），再上投影恢复。压缩比可达 90%+，远超 GQA 的 75%。相比 GQA，MLA 通过低秩压缩实现更极致的 KV Cache 压缩，是 DeepSeek-V2 的核心创新。

核心思想

利用低秩矩阵分解压缩 KV：先下投影到 latent_dim（存入 Cache），再上投影恢复完整的 K/V。Q 也使用低秩投影，但不缓存（只用于当前 token）。RoPE 只应用于 k_rope 和 q_rope 部分。

算法步骤

1. KV 下投影压缩：c_kv = W_down(x) → [B, S, latent_dim]
2. KV 上投影恢复：K,V = W_up(c_kv) → split → k_content, k_rope, v
3. Q 下投影压缩：c_q = W_down_q(x)
4. Q 上投影恢复：q = W_up_q(c_q) → split → q_content, q_rope
5. 应用 RoPE：q_rope, k_rope = rope(q_rope, k_rope)
6. 合并内容：q = cat(q_content, q_rope), k = cat(k_content, k_rope)
7. 计算注意力并输出投影

📐 核心公式

$$c^{KV} = x\,W^{DKV} \in \mathbb{R}^{B\times S\times C}, \qquad [K;V] = \mathrm{split}\big(c^{KV}W^{UK}\big)$$

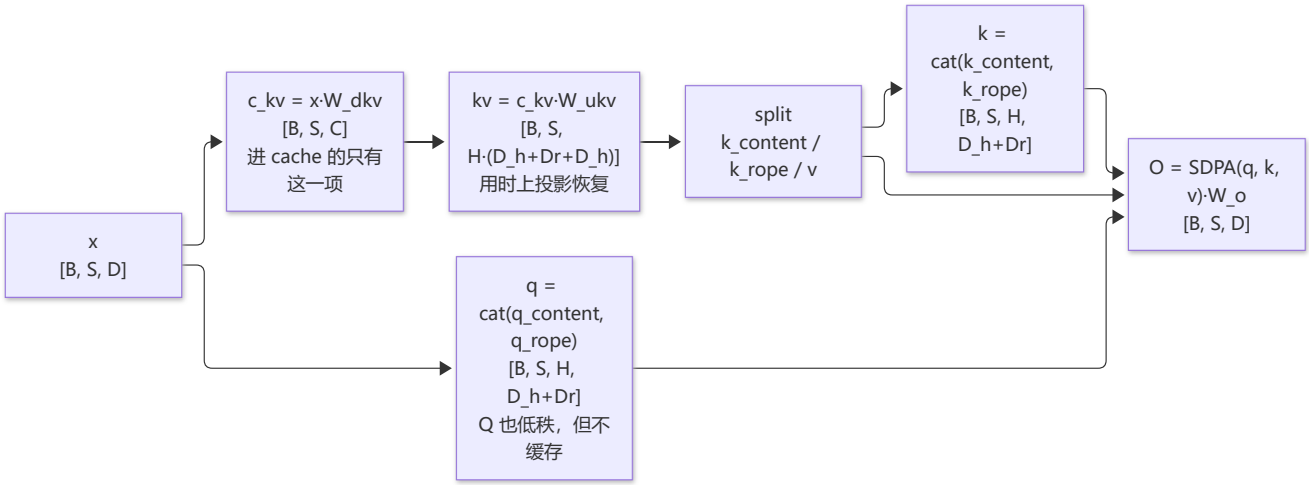
$$c^{Q} = x\,W^{DQ}, \qquad Q = \mathrm{split}\big(c^{Q}W^{UQ}\big)$$

$$k = [k_{\text{content}}; k_{\text{rope}}], \qquad q = [q_{\text{content}}; q_{\text{rope}}]$$

$$\text{GQA}: \quad \text{KV cache} = 2\,G\,D_h$$

$$\text{MLA}: \quad \text{KV cache} = C \qquad (C = 512,\ D_h = 128)$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MLA(nn.Module):
    def __init__(self, d_model, n_heads, dh, latent_dim, rope_dim):
        super().__init__()
        self.h, self.dh, self.dr = n_heads, dh, rope_dim

        # KV 压缩投影
        self.kv_down = nn.Linear(d_model, latent_dim, bias=False)
        self.kv_up = nn.Linear(latent_dim, n_heads*(dh+rope_dim+dh), bias=False)

        # Q 压缩投影
        self.q_down = nn.Linear(d_model, latent_dim, bias=False)
        self.q_up = nn.Linear(latent_dim, n_heads*(dh+rope_dim), bias=False)

        self.wo = nn.Linear(n_heads*dh, d_model, bias=False)

    def forward(self, x, mask=None):
        B, S, _ = x.shape
```

```
        # KV 路径
        kv = self.kv_up(self.kv_down(x)).view(B, S, self.h, -1)
        k_c, k_r, v = kv.split([self.dh, self.dr, self.dh], dim=-1)

        # Q 路径
        q = self.q_up(self.q_down(x)).view(B, S, self.h, -1)
        q_c, q_r = q.split([self.dh, self.dr], dim=-1)

        # RoPE (需要配合 RotaryEmbedding)
        # q_r, k_r = rope(q_r, k_r)

        q = torch.cat([q_c, q_r], dim=-1).transpose(1, 2)
        k = torch.cat([k_c, k_r], dim=-1).transpose(1, 2)
        v = v.transpose(1, 2)

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.dh+self.dr)
        if mask is not None:
            scores = scores.masked_fill(mask==0, float('-inf'))
        out = torch.matmul(F.softmax(scores, dim=-1), v)
        out = out.transpose(1, 2).contiguous().view(B, S, -1)
        return self.wo(out)

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 512
    x = torch.randn(B, S, D)
    mla = MLA(D, n_heads=8, dh=64, latent_dim=128, rope_dim=32)
    out = mla(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 512]
    print(f"Output shape: {out.shape}") # [2, 10, 512]
```

🎯 面试要点

- **KV 先压缩到 latent_dim 再恢复**，Cache 只存压缩后向量
- **压缩比可达 90%+** (vs GQA 75%)
- **Q 也使用低秩投影**，但不缓存（只用于当前 token）
- **DeepSeek-V2 用 MLA + MoE** 实现极高效率
- **面试重点**: 理解低秩压缩的思想，不要求完整实现
- **压缩发生的位置**: c_kv 是唯一进 cache 的东西 —— 压缩的是存储维度，不是头数（这是与 GQA 的本质区别）
- **RoPE 必须单独走不压缩的分量**: 它是位置相关的，挤进低秩空间会被压坏

多头注意力

Multi-Head Attention

并行多头 → 拼接 → 输出投影

📌 原理与思想

核心概念

将输入投影到多个子空间，每个头独立计算注意力，最后拼接并通过线性层融合。相比单头注意力，多头机制允许模型同时关注不同位置的不同表示子空间，捕捉更丰富的语义关系。

核心思想

通过多个独立的注意力头，每个头学习不同的注意力模式（如语法关系、语义关系等）。最后通过输出投影融合所有头的信息，增强模型的表达能力。

中间那个注意力矩阵是 [B, H, S, S]，显存与头数成正比 —— 这正是 GQA、MQA 要砍掉的那一项；而输出投影 W_o 是唯一发生跨头交互的地方。

算法步骤

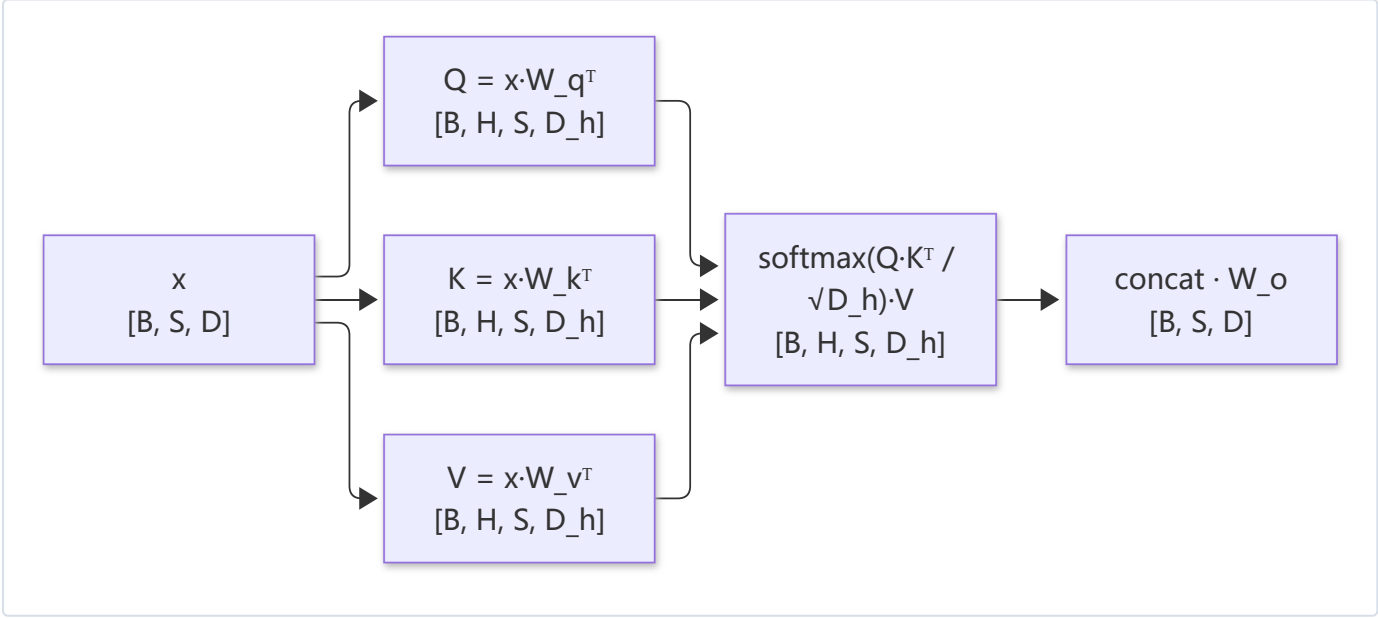
1. 线性投影：Q = x @ Wq, K = x @ Wk, V = x @ Wv
2. 分头：view(B, S, H, Dh).transpose(1, 2) → [B, H, S, Dh]
3. 对每个头计算缩放点积注意力
4. 合并多头：transpose(1, 2).contiguous().view(B, S, D)
5. 输出投影：output = concat_output @ Wo

📐 核心公式

$$\mathrm{MultiHead}(Q,K,V) = \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_H)\,W_O$$

$$\mathrm{head}_i = \mathrm{Attention}\left(QW_i^Q,\ KW_i^K,\ VW_i^V\right), \qquad D_h = \frac{D}{H}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dh = n_heads, d_model // n_heads

        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)

    def forward(self, xq, xk=None, mask=None):
        """
        xq: [batch, seq_len_q, d_model]
        xk: [batch, seq_len_k, d_model] (None for self-attention)
        mask: [batch, 1, seq_len_q, seq_len_k]
        """
        B = xq.size(0)
        xk = xk if xk is not None else xq  # self-attention

        # 线性投影
        q = self.wq(xq).view(B, -1, self.h, self.dh).transpose(1, 2)
        k = self.wk(xk).view(B, -1, self.h, self.dh).transpose(1, 2)
        v = self.wv(xk).view(B, -1, self.h, self.dh).transpose(1, 2)

        # 缩放点积注意力
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.dh)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = F.softmax(scores, dim=-1)

        # 加权求和
        out = torch.matmul(attn, v)  # [B, H, S, Dh]

        # 合并多头
        out = out.transpose(1, 2).contiguous().view(B, -1, self.h * self.dh)
        return self.wo(out)

# 测试
if __name__ == "__main__":
    B, S, D, H = 2, 10, 64, 8
    x = torch.randn(B, S, D)
    mha = MultiHeadAttention(D, H)
    out = mha(x, x)  # self-attention
    print(f"Input shape: {x.shape}")   # [2, 10, 64]
    print(f"Output shape: {out.shape}") # [2, 10, 64]
```

🎯 面试要点

- **分头操作**: `view(B,S,H,Dh).transpose(1,2)` — 面试必写
- **合并操作**: `transpose(1,2).contiguous().view(B,S,D)`
- **自注意力**: Q=K=V=x
- **交叉注意力**: Q=x_dec, K=V=x_enc
- **head_dim = d_model / n_heads**，通常 64 或 128
- **参数量**: 与单头完全相同，4 × d_model² (Wq, Wk, Wv, Wo)，只多出一组中间张量（H·Dh = D）

缩放点积注意力

Scaled Dot-Product Attention

`softmax(QKᵀ/√d)V` — 所有注意力的基础

📌 原理与思想

核心概念

计算 Q 和 K 的点积，除以缩放因子 √d_k 后通过 softmax 得到注意力权重，最后加权求和 V。缩放防止点积过大导致梯度消失，是所有注意力变体（MHA/GQA/Flash Attention）的基础。

核心思想

通过点积衡量 Q 和 K 的相似度，softmax 归一化后作为权重对 V 加权求和。缩放因子 1/√d_k 确保方差稳定，使 softmax 不会进入饱和区。

算法步骤

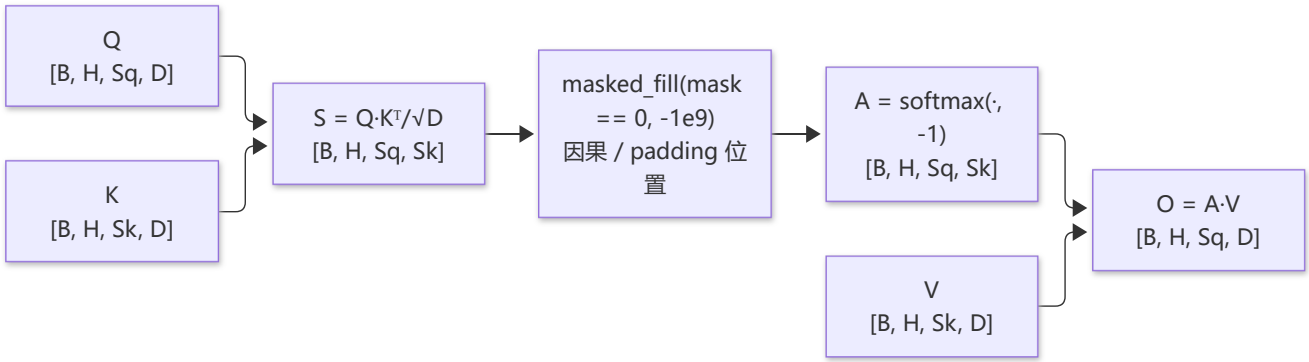
1. 计算 Q 和 K 的点积：scores = Q @ K^T
2. 缩放：scores = scores / √d_k
3. 应用 mask（可选）：masked_fill(mask == 0, -inf)
4. Softmax 归一化：attn_weights = softmax(scores)
5. 加权求和：output = attn_weights @ V

📐 核心公式

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad \text{Var}(q \cdot k) = d_k, \qquad Q, K, V \in \mathbb{R}^{n \times d_k}, \qquad d_k = \frac{D}{H}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn.functional as F
import math

def scaled_dot_product_attention(q, k, v, mask=None):
    """
    缩放点积注意力

    Args:
        q, k, v: [batch, num_heads, seq_len, head_dim]
        mask: [batch, 1, seq_len_q, seq_len_k] (0=屏蔽)

    Returns:
        output: [batch, num_heads, seq_len_q, head_dim]
        attn_weights: [batch, num_heads, seq_len_q, seq_len_k]
    """
    d_k = q.size(-1)

    # 步骤1: 计算注意力得分（缩放点积）
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

    # 步骤2: 应用注意力掩码
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # 步骤3: Softmax 归一化
    attn_weights = F.softmax(scores, dim=-1)

    # 步骤4: 加权求和
    output = torch.matmul(attn_weights, v)

    return output, attn_weights

# 测试
if __name__ == "__main__":
    B, H, S, D = 2, 4, 8, 64
    q = torch.randn(B, H, S, D)
    k = torch.randn(B, H, S, D)
```

```
    v = torch.randn(B, H, S, D)

    # 因果掩码（下三角）
    causal_mask = torch.tril(torch.ones(S, S)).view(1, 1, S, S)

    output, weights = scaled_dot_product_attention(q, k, v, mask=causal_mask)
    print(f"Output shape: {output.shape}")      # [2, 4, 8, 64]
    print(f"Weights shape: {weights.shape}")    # [2, 4, 8, 8]
```

🎯 面试要点

- **缩放因子 1/√d_k**: 当 d_k 大时点积方差大，softmax 进入饱和区梯度消失
- **mask=0 位置填 -1e9**: softmax 后≈0，实现因果/填充屏蔽
- **这是 MHA / GQA / MQA / Flash Attention 的公共基础**
- **时间复杂度**: O(n²d)，空间复杂度 O(n²)
- **面试常问**: 为什么要缩放？不缩放会怎样？

稀疏注意力

Native Sparse Attention / DeepSeek Sparse Attention

长上下文下 O(S²) 不可行：压缩 + 选择 + 滑窗，或索引器 + top-k

📌 原理与思想

核心概念

长上下文注意力是 `O(S²)`，必须稀疏化。NSA 走三分支路线：压缩块注意力（粗粒度全局）、top-n 块选择（中粒度重要区域）、滑动窗口（局部精确），再用学到的门控加权求和；DSA 走另一条路，用一个极轻的 lightning indexer 给每个历史 token 打分，只对 top-k 个 token 做真正的注意力。两者都把复杂度降到 `O(S·n)` 或 `O(S·k)`，也都要求块对齐以适配硬件 —— 代价是块大小 `l` 成了超参，太小则选择本身变贵，太大则选得不够精细。

核心思想

注意力权重实际上是稀疏的 —— 绝大多数位置对当前 token 无关，稠密计算在浪费算力。难点在「怎么稀疏」：套固定模式（只看滑窗）会丢掉长程依赖，随机或启发式选择非连续，gather 会让 GPU 利用率崩掉。所以能用的方案必须同时满足三条：保住长程信息、选择是学出来的、粒度块对齐。

算法步骤

1. NSA：先把 `K/V` 按块均值池化成 `K_cmp`（块大小 `l`），用压缩后的表示做一次粗粒度注意力。
2. 同一份 `K_cmp` 的分数用来挑 top-n 个块，再取回这些块的**原始** `KV` 做精注意力；同时保留最近 `w` 个 token 的滑窗。

3. 三路输出按 `g = σ(wᵀ·[q_t ; ...])` 加权求和，门控是学出来的，模型自己决定何时依赖全局、何时只看局部。
4. DSA： `k_s = W^{K,l}·h_s` 是一个很轻的 key 投影（`d^I` 远小于 `d`），`I_{t,s} = Σ_j w_{t,j}·ReLU(q_{t,j}·k_s)` 给每个历史 token 打分，取 top-k 后再在这 `k` 个 token 上做 MLA 注意力。
5. Indexer 维度极低且能用 FP8 跑，所以「给所有历史 token 打分」这一步的代价可以忽略。

📐 核心公式

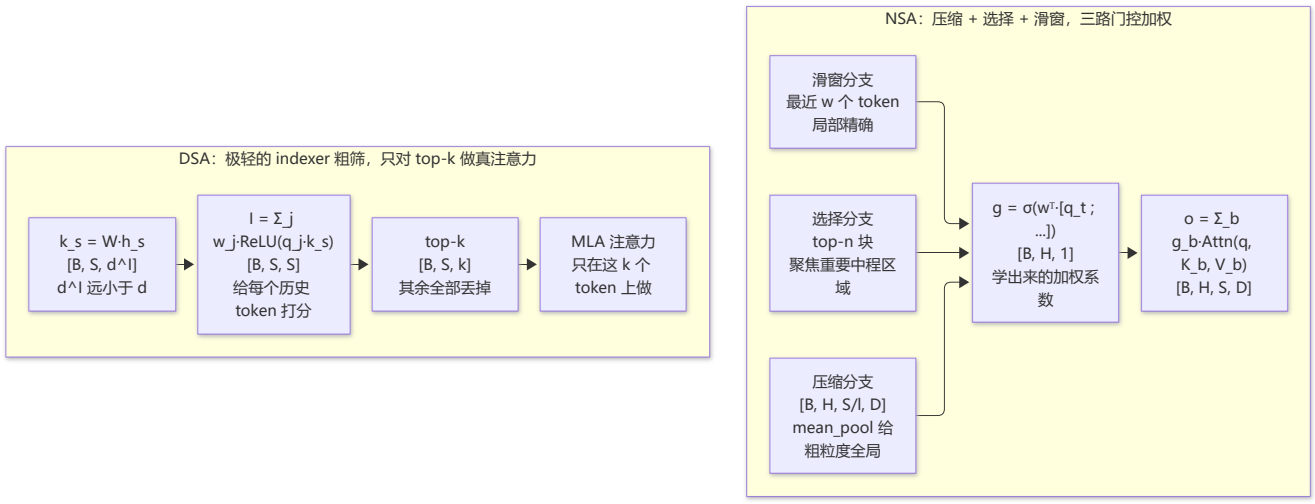
$$
\begin{aligned}
K_{\text{cmp},j} &= \text{mean}\big(K_{jl:(j+1)l}\big) \\
p_j &= \text{score}(q_t, K_{\text{cmp},j}), \qquad \mathcal{B}_{\text{slc}} = \text{top}_n(p) \\
\mathcal{B}_{\text{win}} &= \{t-w, \dots, t\}
\end{aligned}
$$

$$
o_t = \sum_{b \in \{\text{cmp},\text{slc},\text{win}\}} g_b \,\text{Attn}(q_t, K_b, V_b), \qquad g = \sigma\big(w^\top [q_t; \dots]\big)
$$

$$
k_s = W^{K,l} h_s, \qquad I_{t,s} = \sum_j w_{t,j} \text{ReLU}(q_{t,j} \cdot k_s), \qquad \mathcal{S}_t = \text{top}_k(I_{t,\cdot})
$$

$$
\mathcal{S}_t \subseteq \{s \le t\}, \qquad k = 2048, \qquad O(S^2) \to O(S\,k)
$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


def compress_blocks(x, block_size):
    """块内均值池化: [B,H,S,D] → [B,H,S//l,D]"""
    B, H, S, D = x.shape
```

```
    n_blocks = S // block_size
    x = x[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    return x.mean(dim=3)                                    # [B, H, n_blocks, D]


def sdpa(q, k, v, scale):
    """标准缩放点积注意力（q 可以是多一个维度的 query 维）"""
    return F.softmax(q @ k.transpose(-1, -2) * scale, dim=-1) @ v


def nsa_attention(q, k, v, block_size=8, top_n=2, window=16):
    """
    NSA 三分支稀疏注意力
    q, k, v: [B, H, S, D]；返回 o: [B, H, S, D]
    注意：为简洁起见这里省略了因果掩码，真实实现必须加
    """
    B, H, S, D = q.shape
    scale = 1.0 / math.sqrt(D)
    n_blocks = S // block_size

    # ---------- 分支 1: 压缩块注意力（粗粒度全局）----------
    o_cmp = sdpa(q, compress_blocks(k, block_size), compress_blocks(v, block_size), scale)

    # ---------- 分支 2: top-n 块选择（中粒度重要区域）----------
    k_cmp = compress_blocks(k, block_size)                  # 用压缩表示来粗选
    scores = q @ k_cmp.transpose(-1, -2) * scale            # [B, H, S, n_blocks]
    top_idx = scores.topk(min(top_n, n_blocks), dim=-1).indices        # [B,H,S,top_n]

    # 把 K/V 重排成块，再按选中的块索引取回（块对齐，硬件友好）
    k_blk = k[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    v_blk = v[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    bidx = torch.arange(B, device=q.device).view(B, 1, 1, 1)
    hidx = torch.arange(H, device=q.device).view(1, H, 1, 1)
    k_sel = k_blk[bidx, hidx, top_idx]                      # [B,H,S,top_n,l,D]
    v_sel = v_blk[bidx, hidx, top_idx]
    k_sel = k_sel.reshape(B, H, S, -1, D)                   # [B,H,S,top_n*l,D]
    v_sel = v_sel.reshape(B, H, S, -1, D)
    o_slc = sdpa(q.unsqueeze(3), k_sel, v_sel, scale).squeeze(3)

    # ---------- 分支 3: 滑动窗口（局部精确）----------
    win = min(window, S)
    o_win = sdpa(q, k[:, :, -win:, :], v[:, :, -win:, :], scale)

    # ---------- 门控加权求和 ----------
    g = torch.sigmoid(q.mean(dim=(1, 3), keepdim=True)).expand(-1, 1, -1, 3)
    w_cmp, w_slc, w_win = g[..., 0:1], g[..., 1:2], g[..., 2:3]
    return w_cmp * o_cmp + w_slc * o_slc + w_win * o_win


class LightningIndexer(nn.Module):
    """DSA 的 lightning indexer: 轻量投影 + ReLU 打分 + top-k"""

    def __init__(self, n_heads, d_model, d_index=16):
        super().__init__()
        self.wq = nn.Linear(d_model, n_heads * d_index, bias=False)
        self.wk = nn.Linear(d_model, d_index, bias=False)
        self.w_scale = nn.Linear(d_model, n_heads, bias=False)    # 每个 head 的标量权重
        self.n_heads, self.d_index = n_heads, d_index
```

```
    def forward(self, h, top_k=8):
        """h: [B, S, D] 隐状态；返回每个 query 选中的 token 索引 [B, S, top_k]"""
        B, S, _ = h.shape
        q = self.wq(h).view(B, S, self.n_heads, self.d_index).transpose(1, 2)  # [B,H,S,d]
        k = self.wk(h)                                                          # [B,S,d]
        w = self.w_scale(h).transpose(1, 2)                                     # [B,H,S]

        # 每个 indexer head 打分后 ReLU（可跑 FP8），再用标量权重加权求和
        dot = q @ k.transpose(-1, -2)                                           # [B,H,S,S]
        score = (w * torch.relu(dot)).sum(dim=1)                                # [B,S,S]

        # 因果：只看过去
        causal = torch.tril(torch.ones(S, S, dtype=torch.bool, device=h.device))
        score = score.masked_fill(~causal, float("-inf"))
        return score.topk(min(top_k, S), dim=-1).indices                        # [B,S,top_k]


if __name__ == "__main__":
    torch.manual_seed(0)
    B, H, S, D = 1, 4, 64, 16
    q, k, v = (torch.randn(B, H, S, D) for _ in range(3))

    o = nsa_attention(q, k, v, block_size=8, top_n=2, window=16)
    print("NSA 输出:", tuple(o.shape), "(与输入同形)")

    indexer = LightningIndexer(H, D, d_index=16)
    idx = indexer(torch.randn(B, S, D), top_k=8)
    print("indexer 选中索引:", tuple(idx.shape))
    print("每个 query 只算 8 个 token → O(S·k) 而非 O(S²)")
```

🎯 面试要点

- **一句话**: 长上下文的注意力必须稀疏 —— NSA 靠三分支 + 门控，DSA 靠极轻的 indexer 选 top-k
- **三分支各解决什么**: 压缩块给粗粒度全局视野，top-n 选择聚焦重要中程区域，滑窗保证局部精确 —— 缺一个就有明显退化
- **为什么需要门控**: 三条分支的相对重要性随层数、位置、任务变化，硬相加不行，必须有可学习的权重
- **块级 vs token 级选择**: 块级选择（NSA）对硬件友好；DSA 的 token 级 top-k 靠极轻的 indexer 把选择成本压到可忽略
- **indexer 为什么能用 ReLU**: softmax 需要全序列归一化，ReLU 不需要，因此可以分块算、能跑 FP8 —— 这是「lightning」的工程前提
- **indexer 的维度要小**: `d^I` 刻意做得很小（几十维量级），选择成本才可以忽略
- **复杂度**: `O(S²)` → `O(S·k)`。`k=2048`、`S=128K` 时计算量约降两个数量级
- **训练不是一步到位**: DSA 先冻结主模型、只训 indexer 对齐分布（warm-up），再放开全部参数做稀疏训练
- **因果性**: 只能从当前 token 往前的历史里选，未来的块必须屏蔽
- **面试常问**: 稀疏注意力怎么保证不丢长程信息？为什么用块级别选择而不是 token 级别？

层归一化

Layer Normalization

沿特征维度归一化，Transformer 标配

📌 原理与思想

核心概念

在每个样本的特征维度上计算均值和方差进行归一化，再通过可学习的 gamma/beta 进行仿射变换。与 BatchNorm 不同，不依赖 batch size，适合序列模型，能稳定训练并加速收敛。

核心思想

沿特征维度归一化，使每个样本的特征分布稳定在均值为 0、方差为 1 附近。通过 gamma（缩放）和 beta（偏移）两个可学习参数，让模型自适应调整归一化后的分布。

算法步骤

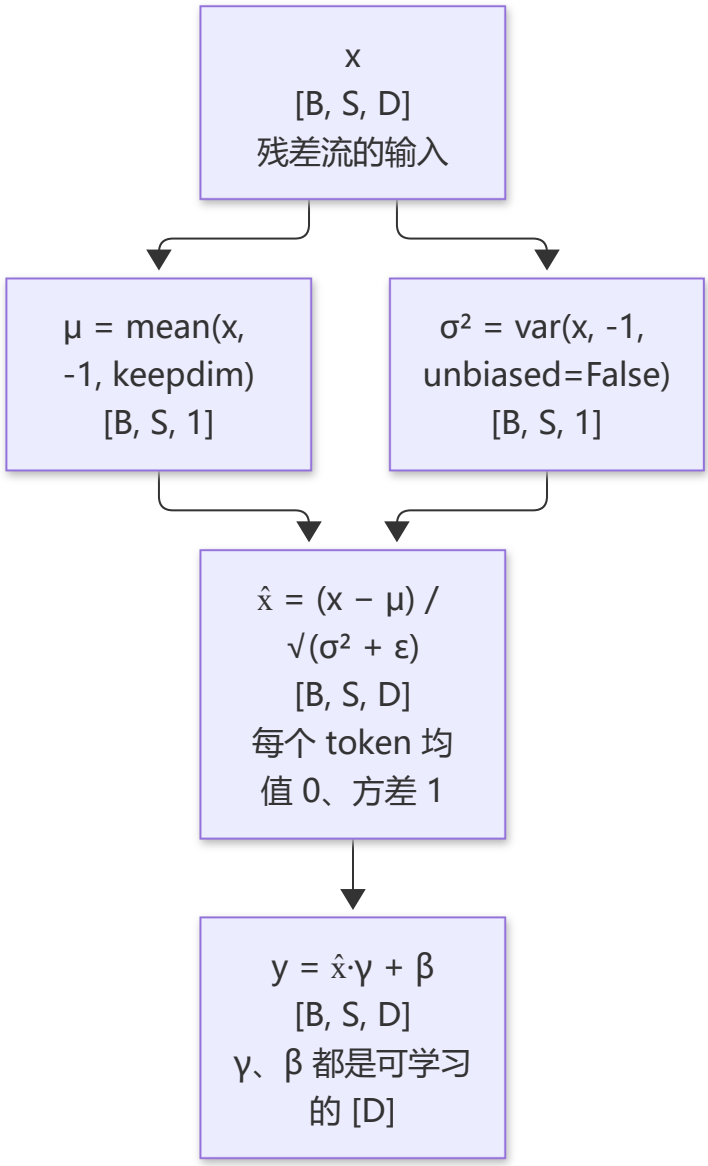
1. 计算均值：μ = mean(x, dim=-1, keepdim=True)
2. 计算方差：σ² = var(x, dim=-1, keepdim=True, unbiased=False)
3. 归一化：x_norm = (x - μ) / √(σ² + ε)
4. 仿射变换：output = x_norm × γ + β

📐 核心公式

$$\mu = \text{mean}(x,\ \text{dim} = -1), \qquad \sigma^2 = \text{var}(x,\ \text{dim} = -1,\ \text{unbiased} = \text{False})$$

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \qquad \gamma, \beta \in \mathbb{R}^D, \qquad \epsilon = 10^{-5}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn

class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.gamma = nn.Parameter(torch.ones(d_model))     # 缩放
        self.beta = nn.Parameter(torch.zeros(d_model))     # 偏移

    def forward(self, x):
        """x: [B, S, D]"""
        # 计算均值和方差（沿最后一维）
        mean = x.mean(-1, keepdim=True)
```

```
        # 面试陷阱: unbiased=False (除以 N, 不是 N-1)
        var = x.var(-1, keepdim=True, unbiased=False)

        # 归一化 + 仿射变换
        x_norm = (x - mean) / torch.sqrt(var + self.eps)
        return x_norm * self.gamma + self.beta

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 64
    x = torch.randn(B, S, D)
    ln = LayerNorm(D)
    out = ln(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 64]
    print(f"Output shape: {out.shape}") # [2, 10, 64]

    # 验证归一化效果
    print(f"Mean: {out[0, 0].mean().item():.6f}")   # ≈ 0
    print(f"Var: {out[0, 0].var().item():.6f}")     # ≈ 1
```

🎯 面试要点

- **面试陷阱**: `torch.var` 默认 `unbiased=True` (N-1)，LN 要用 `False` (N)
- **gamma(缩放) 和 beta(偏移)** 两个可学习参数
- **Pre-Norm**: `x = x + SubLayer(LN(x))` — 现代 LLM 标配
- **Post-Norm**: `x = LN(x + SubLayer(x))` — 原始 Transformer
- **对比 BatchNorm**: BN 沿 batch 维度归一化，LN 沿特征维度
- **与 batch 无关**: 统计量取自每个 token 自己的 D 维；训练与推理走同一条路径，不像 BatchNorm 要维护滑动平均

RMS 归一化

RMS Normalization

去掉均值中心化，LLaMA/Mistral 标配

📌 原理与思想

核心概念

LayerNorm 的简化版：不做均值中心化，只用 RMS (均方根) 归一化。相比 LayerNorm 计算更快、参数更少（只有 gamma 无 beta），实践中效果相当。LLaMA/Mistral/PaLM/Gemma 等主流模型都采用。

核心思想

去掉均值中心化步骤，只用均方根归一化。使用 rsqrt 替代 1/sqrt，计算更高效。在 float32 下计算保证数值稳定性。

算法步骤

1. 计算均方值：ms = mean(x², dim=-1, keepdim=True)
2. 计算均方根倒数：rsqrt = 1/√(ms + ε)
3. 归一化：x_norm = x × rsqrt
4. 缩放：output = x_norm × γ

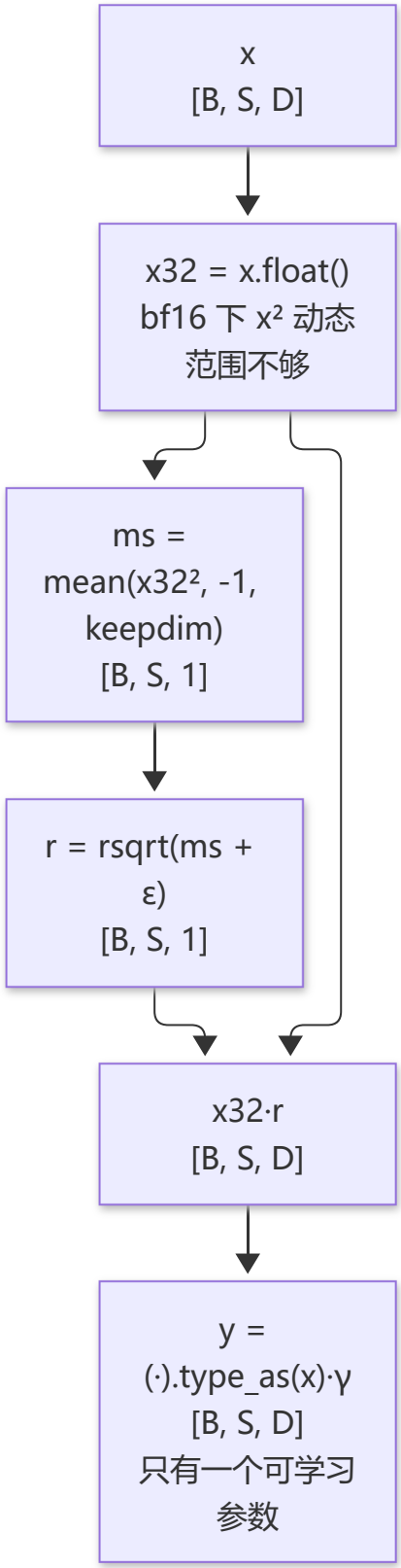
📐 核心公式

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \odot \gamma, \qquad \epsilon = 10^{-5}$$

$$\text{LayerNorm}: \quad \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

$$\text{RMSNorm}: \quad \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \odot \gamma$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn
```

```python
class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-8):
        super().__init__()
        self.eps = eps
        self.gamma = nn.Parameter(torch.ones(d_model))  # 只有缩放

    def _norm(self, x):
        """RMS 归一化核心计算"""
        # 计算均方根倒数 (rsqrt 比 1/sqrt 快)
        ms = x.float().pow(2).mean(-1, keepdim=True)
        return x.float() * torch.rsqrt(ms + self.eps)

    def forward(self, x):
        """x: [B, S, D]"""
        # 在 float32 下计算保证数值稳定性
        return self._norm(x).type_as(x) * self.gamma

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 64
    x = torch.randn(B, S, D)
    rms = RMSNorm(D)
    out = rms(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 64]
    print(f"Output shape: {out.shape}") # [2, 10, 64]
```

🎯 面试要点

- **只有 gamma 没有 beta**（无偏移参数）
- **rsqrt 比 1/sqrt 更高效**（硬件优化）
- **在 float32 下计算保证数值稳定性**
- **LLaMA, Mistral, PaLM, Gemma 等都用 RMSNorm**
- **对比 LayerNorm**: 计算更快，参数更少，效果相当
- **相比 LayerNorm 少一次求均值与减均值**: 归约次数减半，也没有 β；LLaMA、PaLM、Qwen 都把它放在每个子层之前

📍 位置编码 Position

1 题

旋转位置编码

Rotary Position Embedding, RoPE

旋转 Q/K 向量注入位置信息，LLaMA 标配

📌 原理与思想

核心概念

将位置信息编码为旋转角度，对 Q 和 K 的每对相邻维度施加旋转。旋转后 Q·K 的点积自然包含相对位置信息。相比绝对位置编码（无法处理变长序列）和 ALiBi（需要额外参数），RoPE 无需额外参数即可自然编码相对位置。

核心思想

通过旋转矩阵将位置信息注入 Q 和 K。对每对相邻维度 [x_1, x_2] 施加旋转 [$-x_2$, x_1]，等价于旋转 90°。旋转后 q_m · k_n 只依赖 m-n，天然具有相对位置感知能力。只对 Q 和 K 施加旋转，V 不变。

算法步骤

1. 预计算频率：inv_freq = 1/(10000^(2i/d))
2. 计算角度：angles = outer(positions, inv_freq)
3. 预计算 cos/sin：cos = cos(angles), sin = sin(angles)
4. 定义 rotate_half：chunk(x, 2, dim=-1) → cat(-x2, x1)
5. 应用旋转：q_rot = q × cos + rotate_half(q) × sin
6. 对 k 同样应用旋转

📐 核心公式

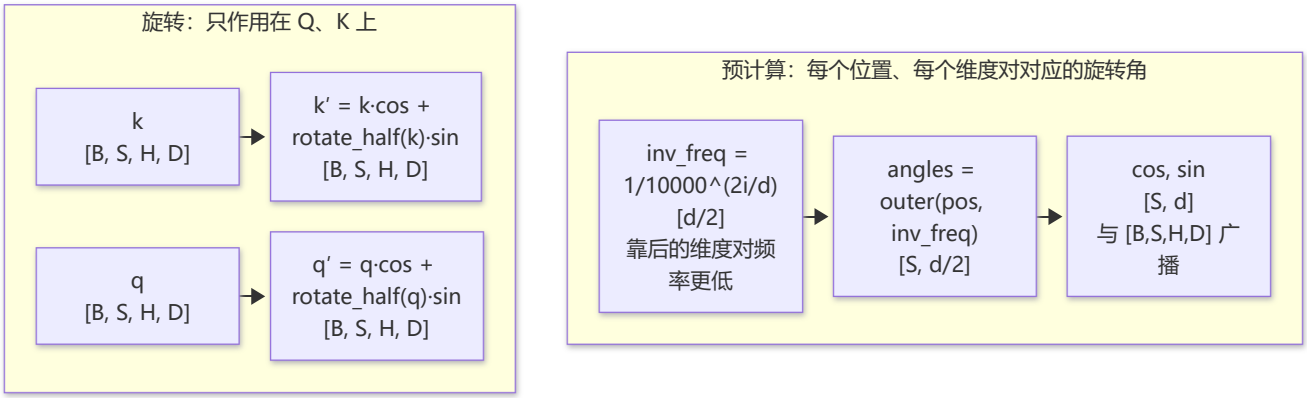
$$f(q, m) = q \odot \cos(m\theta) + \text{rotate_half}(q) \odot \sin(m\theta)$$

$$\text{rotate_half}([x_1, x_2]) = [-x_2,\ x_1], \qquad R(m\theta) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}$$

$$(R_m q) \cdot (R_n k) = q^\top R_{n-m}\, k$$

$$\theta_i = 10000^{-2i/d}, \qquad i = 0, \ldots, \frac{d}{2} - 1$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn

class RotaryEmbedding(nn.Module):
    def __init__(self, head_dim, max_seq_len=2048, theta=10000.0):
        super().__init__()
        # 预计算 cos/sin
        inv_freq = 1.0 / (theta ** (torch.arange(0, head_dim, 2).float() / head_dim))
        t = torch.arange(max_seq_len).float()
        angles = torch.outer(t, inv_freq)           # [S, D/2]
        angles = torch.cat([angles, angles], dim=-1) # [S, D]
        self.register_buffer('cos', angles.cos())
        self.register_buffer('sin', angles.sin())

    def forward(self, q, k):
        """q,k: [B, S, H, D]"""
        S = q.size(1)
        cos = self.cos[:S].view(1, S, 1, -1)
        sin = self.sin[:S].view(1, S, 1, -1)

        def rotate_half(x):
            x1, x2 = x.chunk(2, dim=-1)
            return torch.cat([-x2, x1], dim=-1)

        q_rot = q * cos + rotate_half(q) * sin
        k_rot = k * cos + rotate_half(k) * sin
        return q_rot, k_rot

# 测试
if __name__ == "__main__":
    B, S, H, D = 2, 10, 8, 64
    q = torch.randn(B, S, H, D)
    k = torch.randn(B, S, H, D)

    rope = RotaryEmbedding(D)
    q_rot, k_rot = rope(q, k)

    print(f"Input q shape: {q.shape}")     # [2, 10, 8, 64]
    print(f"Output q shape: {q_rot.shape}") # [2, 10, 8, 64]
```

🎯 面试要点

- **rotate_half**: `chunk` 两半 → `cat(-x2, x1)` = 旋转 90°
- **只对 Q 和 K 施加旋转**，V 不变
- **预计算 cos/sin** 避免重复计算
- **天然具有相对位置感知**: q_m · k_n 只依赖 m-n
- **theta=10000 是标准值**，调整可实现长度外推 (NTK/YaRN)
- **LLaMA, Mistral, Qwen 等主流模型都使用 RoPE**

标准前馈网络

Feed-Forward Network

两层 MLP，占 Transformer 2/3 参数量

📌 原理与思想

核心概念

注意力层之后的两层全连接网络。先上投影扩展维度（通常 4 倍），应用激活函数，再下投影恢复维度。Attention 捕捉 token 间的关系，FFN 对每个 token 独立做非线性变换，占 Transformer 参数量的 2/3。

核心思想

通过上投影将维度从 D 扩展到 4D，在高维空间做非线性变换，再下投影回 D 维。这种"扩展-变换-压缩"的结构增强了模型的表达能力。

算法步骤

1. 上投影：h = $W_1 \cdot x + b_1$，维度 D → 4D
2. 激活函数：a = ReLU(h)
3. 下投影：output = $W_2 \cdot a + b_2$，维度 4D → D

📐 核心公式

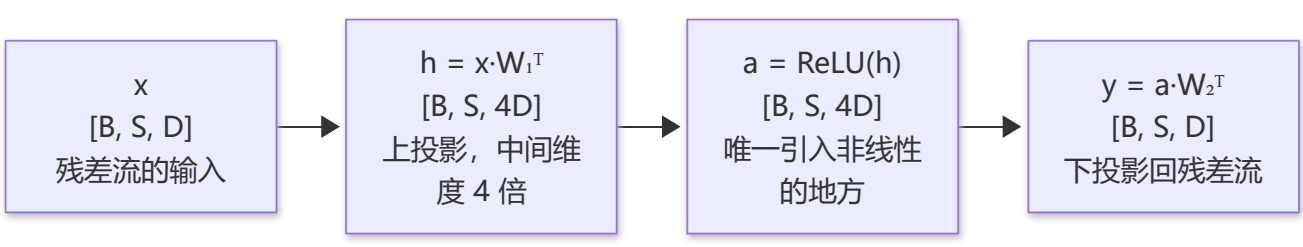
$$\mathrm{FFN}(x) = W_2 \mathrm{ReLU}(W_1 x + b_1) + b_2$$

$$W_1 \in \mathbb{R}^{D \times 4D}, \qquad W_2 \in \mathbb{R}^{4D \times D}$$

$$\mathrm{FFN}: \quad D \cdot 4D + 4D \cdot D = 8D^2$$

$$\mathrm{Attention}: \quad 4D^2$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class FFN(nn.Module):
    def __init__(self, d_model, d_ff=None):
        super().__init__()
        d_ff = d_ff or 4 * d_model
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        """x: [B, S, D]"""
        return self.w2(F.relu(self.w1(x)))

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 512
    x = torch.randn(B, S, D)
    ffn = FFN(D)
    out = ffn(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 512]
    print(f"Output shape: {out.shape}") # [2, 10, 512]

    # 统计参数量
    total_params = sum(p.numel() for p in ffn.parameters())
    print(f"FFN params: {total_params:,}")  # 约 4M
```

🎯 面试要点

- **中间维度通常 4× d_model**
- **占 Transformer 参数量的 2/3**
- **现代 LLM 多用 SwiGLU 替代标准 FFN+ReLU**
- **MoE 本质上是对 FFN 层的稀疏化**
- **面试常问**: FFN 的作用是什么？为什么参数量这么大?
- **逐 token 独立**: 同一个 W_1 作用在所有位置上，没有任何跨 token 交互 —— 跨 token 的信息交换全部由注意力负责

混合专家模型

Mixture of Experts, MoE

稀疏激活，大参数量小计算量

📌 原理与思想

核心概念

将 FFN 替换为多个"专家"网络，Router 为每个 token 选择 Top-K 个专家处理。总参数量大但每次只激活部分，计算量可控。相比 Dense 模型，MoE 可以增加模型容量但不增加计算量，如 Mixtral 8x7B 有 46.7B 参数但实际计算量仅约 12.9B。

核心思想

通过 Router 网络为每个 token 动态选择最相关的 K 个专家处理，实现稀疏激活。每个专家是独立的 FFN，只处理分配到的 token，最后加权融合。

总共有 N 份专家参数，而每个 token 只算 K 份，所以 FLOPs 只占 K/N —— 参数量与计算量解耦。

算法步骤

1. Router 计算：logits = Router(x)，得到每个专家的得分
2. Top-K 选择：选出得分最高的 K 个专家
3. 权重归一化：weights = softmax(top_k_logits)
4. 专家处理：每个 expert 处理分配到的 token
5. 加权融合：output = Σ(weights[i] × expert[i](x))

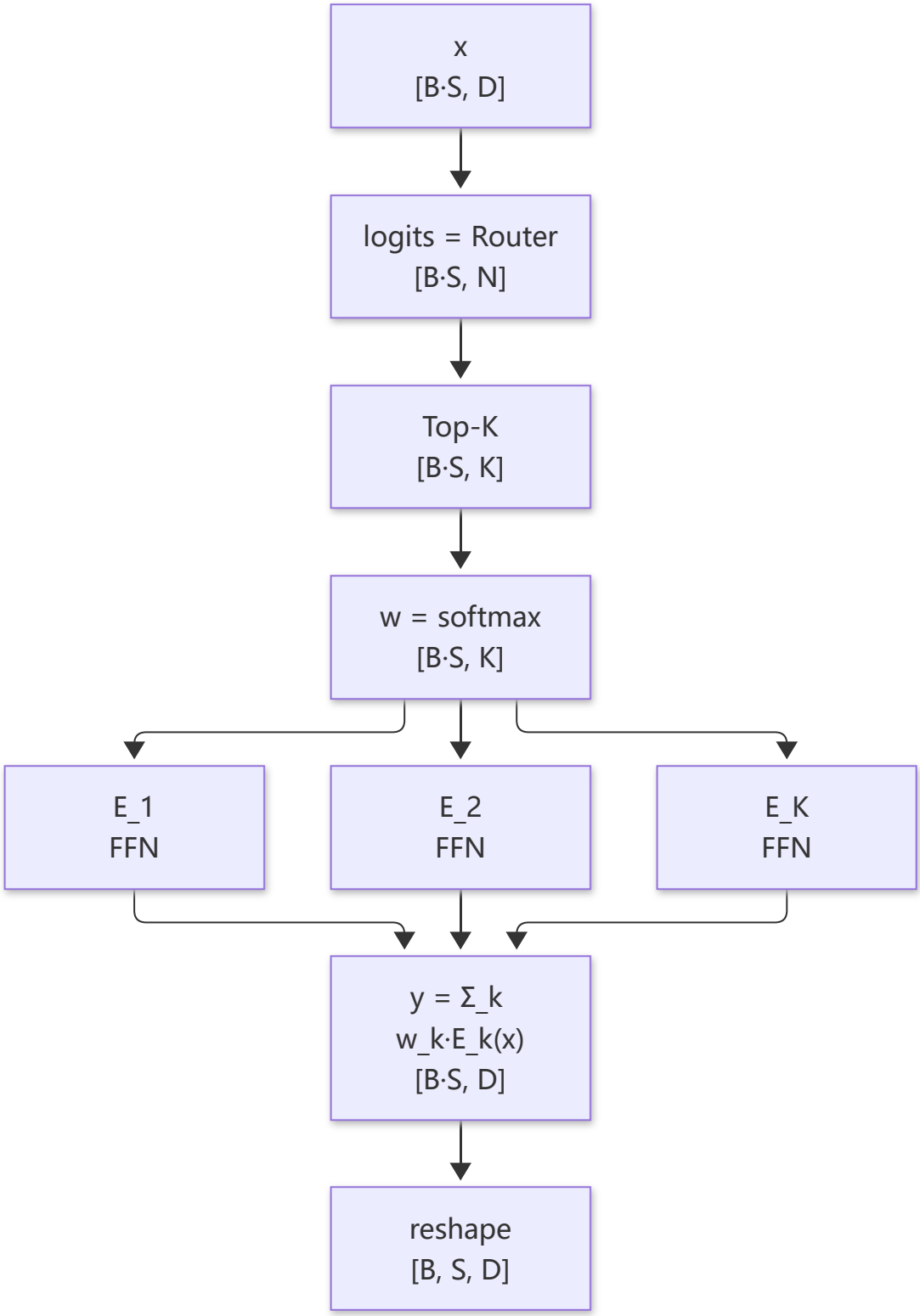
📐 核心公式

$$\text{MoE}(x) = \sum_{i \in \text{TopK}} \text{softmax}(\text{Router}(x))_i \, E_i(x)$$

$$\text{Router}(x) = W_r x, \qquad W_r \in \mathbb{R}^{D \times N}, \qquad K = 2$$

$$\text{Mixtral 8x7B:} \quad N = 8,\ K = 2, \qquad \text{FLOPs} \approx 12.9\text{B vs } 46.7\text{B total}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```python
class MoE(nn.Module):
    def __init__(self, d_model, n_experts, top_k):
        super().__init__()
        self.n_experts, self.top_k = n_experts, top_k
        self.router = nn.Linear(d_model, n_experts, bias=False)
        self.experts = nn.ModuleList([
            nn.Sequential(nn.Linear(d_model, 4*d_model), nn.SiLU(),
                          nn.Linear(4*d_model, d_model))
            for _ in range(n_experts)
        ])

    def forward(self, x):
        B, S, D = x.shape
        x_flat = x.view(-1, D)                          # [B*S, D]
        logits = self.router(x_flat)                     # [B*S, N]
        weights, indices = torch.topk(logits, self.top_k, dim=-1)
        weights = F.softmax(weights, dim=-1)             # 归一化

        output = torch.zeros_like(x_flat)
        for i, expert in enumerate(self.experts):
            mask = (indices == i)                        # 哪些 token 选了 expert i
            token_idx, topk_pos = torch.where(mask)
            if token_idx.numel() > 0:
                out = expert(x_flat[token_idx])
                output.index_add_(0, token_idx, out * weights[token_idx, topk_pos].unsqueeze(-1))
        return output.view(B, S, D)

# 测试
if __name__ == "__main__":
    B, S, D = 2, 16, 512
    x = torch.randn(B, S, D)
    moe = MoE(D, n_experts=8, top_k=2)
    out = moe(x)
    print(f"Input shape: {x.shape}")   # [2, 16, 512]
    print(f"Output shape: {out.shape}") # [2, 16, 512]
```

🎯 面试要点

- **Router 决定每个 token 由哪些专家处理**
- **Top-K 路由**: 每个 token 只激活 K 个专家
- **需要 load balancing loss** 防止专家负载不均
- **Mixtral 8x7B**: 8 专家选 2，计算量约 12.9B
- **DeepSeek-V2/V3**: MoE + MLA 实现极致效率
- **面试常问**: MoE 如何减少计算量？如何保证负载均衡？

SwiGLU

门控 + SiLU 激活，LLaMA/PaLM 标配

📌 原理与思想

核心概念

引入门控机制：一个分支用 SiLU 激活作为门，另一个分支无激活，两者逐元素相乘后下投影。相比标准 FFN+ReLU，门控机制让模型学习哪些信息通过，SiLU 激活更平滑，实验证明效果更好。LLaMA/PaLM/Mistral/Qwen 等主流模型都使用。

核心思想

通过门控机制控制信息流：Gate 分支用 SiLU 激活学习"哪些信息应该通过"，Up 分支无激活提供"信息内容"，两者逐元素相乘实现选择性传递。

算法步骤

1. 门控分支：gate = SiLU(W_gate · x)
2. 值分支：up = W_up · x
3. 门控相乘：activated = gate ⊙ up（逐元素乘法）
4. 下投影：output = W_down · activated

📐 核心公式

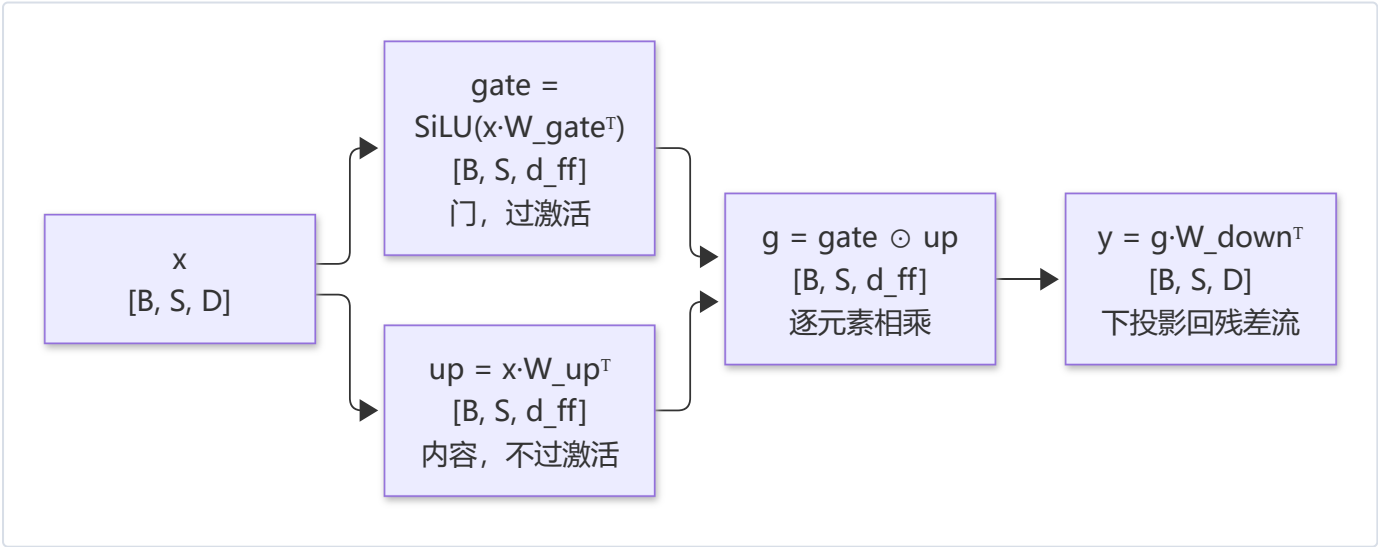
$$\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x)$$

$$\text{SiLU}(x) = x\,\sigma(x), \qquad d_{ff} = \frac{8}{3}D$$

$$\text{SwiGLU}: \quad 3D\,d_{ff} \approx 8D^2$$

$$\text{FFN}: \quad 8D^2$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SwiGLU(nn.Module):
    def __init__(self, d_model, d_ff=None):
        super().__init__()
        d_ff = d_ff or int(8/3 * d_model)  # 保持参数量与标准FFN相当
        self.w_gate = nn.Linear(d_model, d_ff, bias=False)
        self.w_up = nn.Linear(d_model, d_ff, bias=False)
        self.w_down = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        """x: [B, S, D]"""
        gate = F.silu(self.w_gate(x))   # 门控分支
        up = self.w_up(x)               # 值分支
        return self.w_down(gate * up)    # 门控相乘 → 下投影

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 512
    x = torch.randn(B, S, D)
    swiglu = SwiGLU(D)
    out = swiglu(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 512]
    print(f"Output shape: {out.shape}") # [2, 10, 512]
```

🎯 面试要点

- **三个权重矩阵** (Gate/Up/Down)，标准 FFN 只有两个
- **SiLU(x) = x × sigmoid(x)**，也称 Swish
- **d_ff 通常取 8/3 × d_model**（保持总参数量相当）
- **LLaMA, PaLM, Mistral, Qwen 等主流模型都使用**
- **面试常问**: SwiGLU 和标准 FFN 的区别？为什么效果更好？

📉 损失函数 Loss

8 题

交叉熵 / 语言模型损失

Cross Entropy / LM Loss

下一个 token 预测，LLM 训练基础

📌 原理与思想

核心概念

语言模型的核心训练目标：给定前文预测下一个 token。通过 shift 操作将 logits 和 labels 对齐，计算交叉熵。交叉熵衡量预测分布与真实分布的差异，梯度计算简洁（softmax + CE 的梯度 = y - one_hot），是 LLM 训练的基础。

核心思想

用前面的 token 预测下一个 token。通过 shift 操作：logits 去尾（去掉最后一个位置的预测），labels 去头（去掉第一个位置的目标），使 logits[:, :-1] 预测 labels[:, 1:]。

算法步骤

1. Shift logits：shift_logits = logits[:, :-1, :]
2. Shift labels：shift_labels = labels[:, 1:]
3. 展平：flattened_logits = shift_logits.view(-1, V)
4. 展平：flattened_labels = shift_labels.view(-1)
5. 计算交叉熵：loss = CE(flattened_logits, flattened_labels)

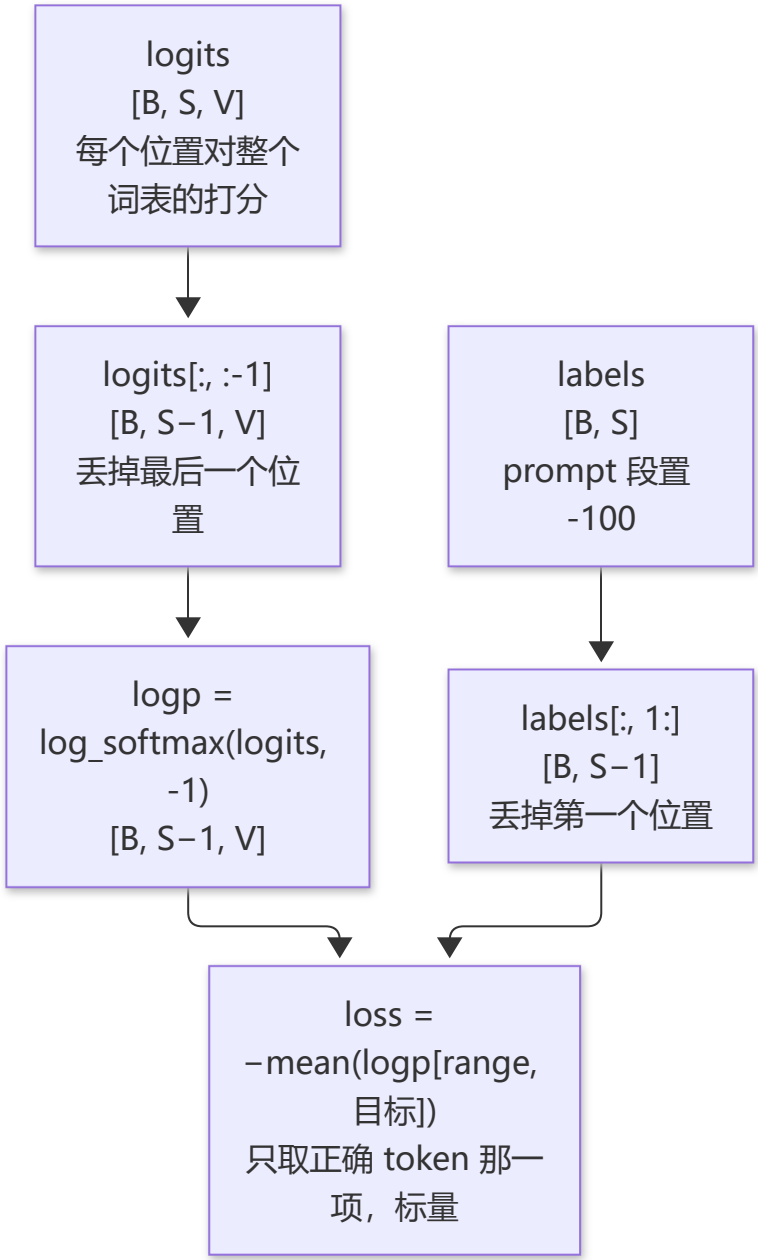
📐 核心公式

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|x|}\sum_{t=1}^{|x|}\log P_\theta(x_t \mid x_{<t})$$

$$\text{CE}(\text{shift}(\text{logits}),\ \text{shift}(\text{labels}))\,, \qquad \text{logits}[:, :-1] \text{ predicts } \text{labels}[:, 1:]$$

$$\begin{aligned} \text{Pretrain}: &\quad \text{all tokens counted} \\ \text{SFT}: &\quad \text{label}_{\text{prompt}} = -100, \qquad \text{ignore_index} = -100 \end{aligned}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn.functional as F

def lm_loss(logits, labels, ignore_index=-100):
    """
    语言模型损失（下一个 token 预测）

    logits: [B, S, vocab_size]
    labels: [B, S]
    """
    # Shift: 用前面的 token 预测下一个
    shift_logits = logits[:, :-1, :].contiguous()   # [B, S-1, V]
```

```
    shift_labels = labels[:, 1:].contiguous()        # [B, S-1]

    return F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        ignore_index=ignore_index
    )

# 测试
if __name__ == "__main__":
    B, S, V = 2, 10, 32000
    logits = torch.randn(B, S, V)
    labels = torch.randint(0, V, (B, S))

    loss = lm_loss(logits, labels)
    print(f"Loss: {loss.item():.4f}")

# SFT Loss: 只计算 response 部分
def sft_loss(logits, labels, prompt_lengths):
    """
    SFT 损失：prompt 部分设为 -100
    """
    masked_labels = labels.clone()
    for i, plen in enumerate(prompt_lengths):
        masked_labels[i, :plen] = -100
    return lm_loss(logits, masked_labels)
```

🎯 面试要点

- **Shift 操作是核心**: logits 去尾，labels 去头
- **ignore_index=-100** 的位置不参与梯度
- **Pretrain**: 所有 token 参与 loss
- **SFT**: prompt 部分 label 设为 -100，只算 response
- **面试常问**: Pretrain Loss 和 SFT Loss 的区别?
- **梯度形式**: ∂L/∂logits = softmax(logits) − one_hot(目标)，预测越离谱梯度越大

DAPO

Decoupled Clip and Dynamic sAmpling Policy Optimization

四处改动修 GRPO：解耦裁剪、动态采样、token 级损失、超长惩罚

📌 原理与思想

核心概念

DAPO 不改 GRPO 的骨架，只针对四个已知缺陷动手：(1) clip-higher 解耦裁剪上下界，放开低概率 token 的上升空间以维持熵；(2) 动态采样过滤掉全对/全错的组，只留有梯度信号的组；(3) 用 token 级损失替代序列级平均，让长回答的每个 token 权重一致；(4) 超长奖励塑形，惩罚被截断的超长回答。四个改动互不耦合，最终在长链推理任务上明显更强；代价是超参变多（多一个 `ε_high`、一个 `α`、一个 `L_max`），动态采样在有效组不足时还要额外的重采样逻辑。

核心思想

不改 GRPO 的骨架，只针对上面四个已知缺陷定点修补。`clip-higher` 解耦上下界，把上界放得比下界宽，给低概率 token 留出上升通道以维持熵；动态采样过滤掉没有梯度信号的组；损失改为 token 级求和；再对超长回答做奖励塑形。

算法步骤

1. `clip-higher`：用 `clip(ρ, 1-ε_low, 1+ε_high)`，典型 `ε_low = 0.2`、`ε_high = 0.28`，抬高的是熵的下界。
2. 动态采样：丢掉 `Â` 全为 0 的组，只留组内奖励有正有负的组，等于把算力全花在有效样本上。
3. token 级损失：`L = -1/Σ_i|y_i| · Σ_i Σ_t min(...)`，分母是总 token 数而不是每组平均，长回答的每个 token 权重一致。
4. 超长奖励塑形：`R̃(y) = R(y) - α·max(0, |y| - L_max)`，超过上限就线性扣分。

📐 核心公式

$$\rho_{i,t} = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t} \mid x, y_{i,<t})}, \qquad \hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \epsilon}$$

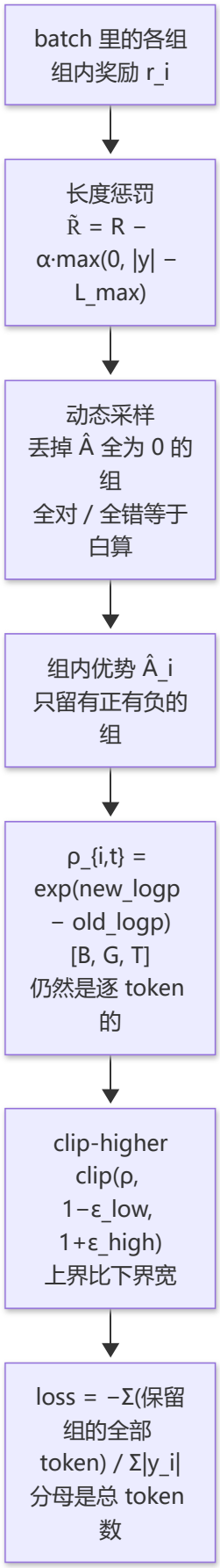
$$\text{clip-higher}: \quad \min\Big(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t},\, 1-\epsilon_{\text{low}},\, 1+\epsilon_{\text{high}})\,\hat{A}_i\Big), \qquad \epsilon_{\text{low}} = 0.2, \quad \epsilon_{\text{high}} = 0.28$$

$$\text{dynamic sampling}: \quad \text{discard groups with } \hat{A}_i \equiv 0$$

$$\text{token-level loss}: \quad \mathcal{L} = -\frac{1}{\sum_i |y_i|} \sum_i \sum_t \min\Big(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t},\, 1-\epsilon_{\text{low}},\, 1+\epsilon_{\text{high}})\,\hat{A}_i\Big)$$

$$\text{overlong shaping}: \quad \tilde{R}(y) = R(y) - \alpha \max(0,\ |y| - L_{\max})$$

📊 张量流程图


batch 里的各组
组内奖励 r_i
长度惩罚
R̃ = R −
α·max(0, |y| −
L_max)
动态采样
丢掉 Â 全为 0 的
组
全对 / 全错等于
白算
组内优势 Â_i
只留有正有负的
组
ρ_{i,t} =
exp(new_logp
− old_logp)
[B, G, T]
仍然是逐 token
的
clip-higher
clip(ρ,
1−ε_low,
1+ε_high)
上界比下界宽
loss = −Σ(保留
组的全部
token) / Σ|y_i|
分母是总 token
数

💻 代码实现

```python
import torch

def overlong_reward_shaping(rewards, lengths, max_len, alpha=1.0):
    """超长奖励塑形：超出 max_len 的部分线性扣分"""
    over = (lengths - max_len).clamp_min(0).float()
    return rewards - alpha * over


def dynamic_sampling_filter(rewards, group_size=8):
    """
    动态采样：丢掉组内奖励全相同的组（Â ≡ 0，没有梯度信号）
    rewards: [num_groups * group_size]  扁平化的组内奖励
    返回：保留下来的组的索引
    """
    groups = rewards.view(-1, group_size)
    keep = groups.max(dim=-1).values != groups.min(dim=-1).values
    return keep.nonzero(as_tuple=True)[0]


def dapo_loss(per_token_logp_new, per_token_logp_old, completion_mask,
              advantages, eps_low=0.2, eps_high=0.28):
    """
    DAPO 损失：clip-higher + token 级归一化
    per_token_logp_*: [B, G, T]    advantages: [B, G]
    """
    # 1. token 级重要性比率
    ratio = (per_token_logp_new - per_token_logp_old).exp()      # [B, G, T]
    adv = advantages.unsqueeze(-1)                               # [B, G, 1]

    # 2. 解耦的上下界：上界放松 → 低概率 token 有更大的上升空间
    clipped = torch.clamp(ratio, 1 - eps_low, 1 + eps_high)
    per_token = torch.min(ratio * adv, clipped * adv)            # [B, G, T]

    # 3. token 级归一化：除以整个 batch 的总 token 数
    mask = completion_mask.float()
    return -(per_token * mask).sum() / mask.sum().clamp_min(1.0)


if __name__ == "__main__":
    # 动态采样：前两组奖励全相同 → 丢弃
    rewards = torch.tensor([1., 1., 1., 1.,   0., 0., 0., 0.,   1., 0., 1., 0.,   1., 0., 0., 1.])
    print("保留的组:", dynamic_sampling_filter(rewards, group_size=4).tolist())

    # 超长塑形
    shaped = overlong_reward_shaping(
        torch.tensor([1.0, 1.0]), torch.tensor([100., 600.]), max_len=512, alpha=0.5)
    print("超长塑形后奖励:", shaped.tolist())

    # 主损失
    torch.manual_seed(0)
    B, G, T = 2, 4, 5
    old_logp = torch.randn(B, G, T) - 2.0
    new_logp = old_logp + 0.02 * torch.randn(B, G, T)
    mask = torch.ones(B, G, T)
    mask[:, :, -1] = 0
```

```
    adv = torch.randn(B, G)
    print("dapo loss:", dapo_loss(new_logp, old_logp, mask, adv).item())
```

🎯 面试要点

- **clip-higher 为什么有效**: 上界放松后，低概率 token 的比率有更大的上升空间，熵不会那么快坍缩；抬高的是熵的下界，不是直接把熵加进损失
- **动态采样为什么有效**: 全对/全错的组优势恒为 0，算力全白花；丢掉它们等于按「有没有梯度信号」筛数据
- **token 级归一化的影响**: 按序列长度归一化时，长回答的每个 token 梯度被稀释；除以总 token 数让每个 token 等权，长回答因此学得更好
- **超长惩罚是软约束**: 硬截断会让奖励突变、模型学不到收尾；线性/分段惩罚给的是平滑信号
- **四个改动的取向不同**: clip-higher 管探索（熵），动态采样管效率，token 级损失管长度偏置，超长塑形管长度控制
- **和 GSPO 的对照**: DAPO 保留 token 级比率（但改了归一化和裁剪界），GSPO 直接把比率提到序列级
- **面试常问**: DAPO 相对 GRPO 改了哪几处？各自的动机是什么？

DPO 损失

Direct Preference Optimization

无需奖励模型，直接优化偏好数据

📌 原理与思想

核心概念

将奖励函数参数化为策略与参考策略的对数比率，直接在偏好数据上优化。增加 chosen 概率，降低 rejected 概率。相比 PPO 不需要 reward model 和 critic，训练更简单稳定，效果相当甚至更好。

核心思想

利用 RL 的对偶性，将奖励函数隐式表示为 r(x,y) = β · log(πθ(y|x)/πref(y|x))。这样策略优化目标可以直接用策略的对数概率差来表示，无需显式奖励模型。

算法步骤

1. 收集偏好数据：(prompt, chosen_response, rejected_response)
2. 计算策略与参考策略的对数概率差
3. 构造损失函数：增加 chosen 概率，降低 rejected 概率
4. 直接用梯度下降优化策略模型

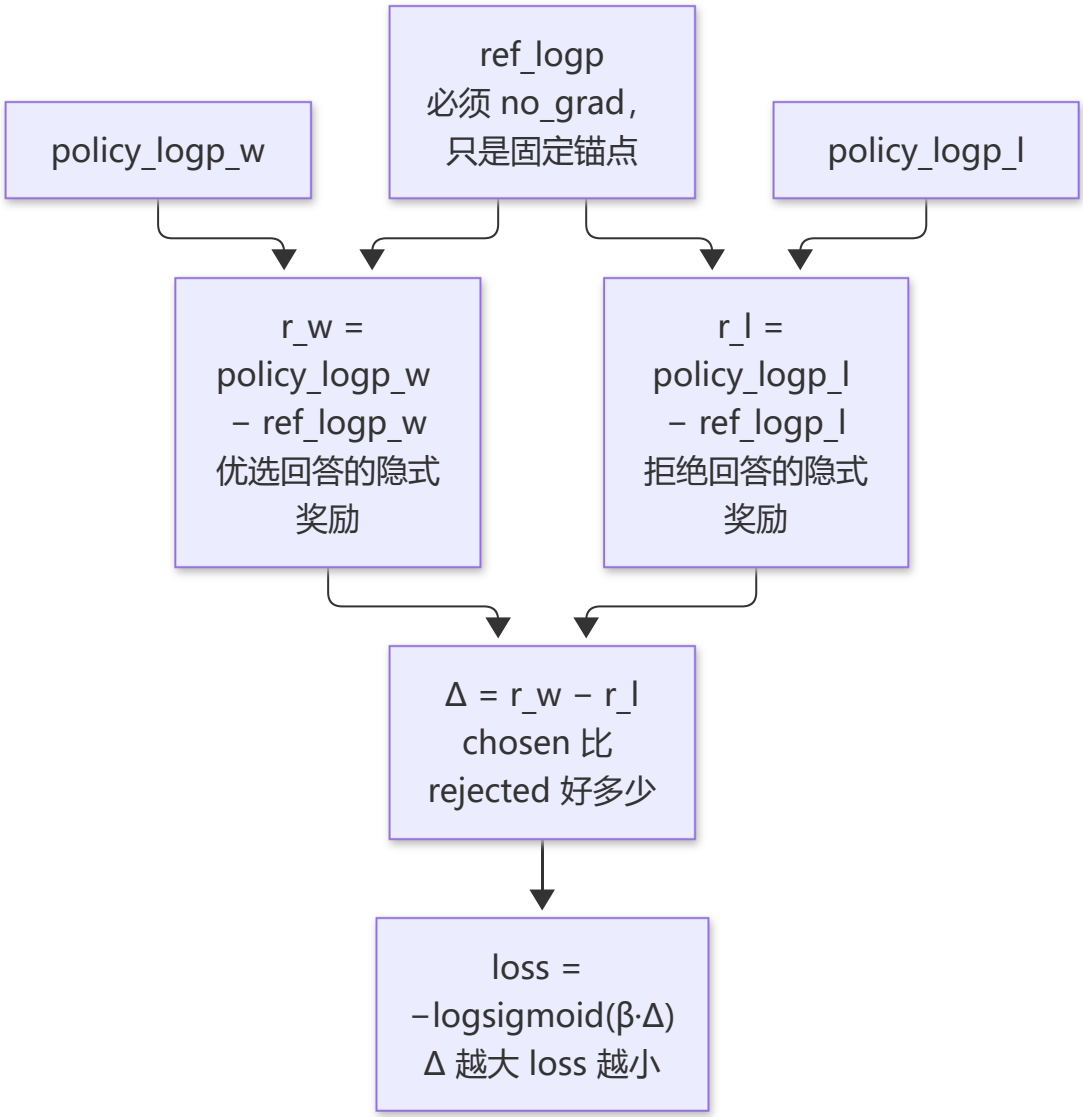
📐 核心公式

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\left(\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)} - \log\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)}\right)\right)\right]$$

$$\text{logits} = \left(\log p_\theta^w - \log p_{\text{ref}}^w\right) - \left(\log p_\theta^l - \log p_{\text{ref}}^l\right), \qquad \mathcal{L} = -\log\sigma(\beta\cdot\text{logits})$$

$$y_w:\ \text{chosen}, \qquad y_l:\ \text{rejected}, \qquad \beta\in[0.1,\ 0.5]$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn.functional as F

def dpo_loss(policy_chosen_logps, policy_rejected_logps,
             ref_chosen_logps, ref_rejected_logps, beta=0.1):
    """
```

```
    DPO 损失

    *_logps: [batch_size] 每个样本的对数概率之和
    """
    # 隐式奖励 = log(π_θ / π_ref)
    chosen_ratio = policy_chosen_logps - ref_chosen_logps
    rejected_ratio = policy_rejected_logps - ref_rejected_logps

    # 奖励差
    logits = chosen_ratio - rejected_ratio

    # DPO 损失
    return -F.logsigmoid(beta * logits).mean()

# 完整训练流程
def compute_logps(model, input_ids, labels):
    """计算模型对 labels 的对数概率之和"""
    logits = model(input_ids).logits
    log_probs = F.log_softmax(logits[:, :-1], dim=-1)
    labels = labels[:, 1:]
    per_token_logps = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)
    return per_token_logps.sum(dim=-1)

# 测试
if __name__ == "__main__":
    B = 4
    policy_chosen = torch.randn(B)
    policy_rejected = torch.randn(B)
    ref_chosen = torch.randn(B)
    ref_rejected = torch.randn(B)

    loss = dpo_loss(policy_chosen, policy_rejected,
                    ref_chosen, ref_rejected, beta=0.1)
    print(f"DPO Loss: {loss.item():.4f}")
```

🎯 面试要点

- **核心**: 奖励 = log(π_θ / π_ref)，无需显式奖励模型
- **chosen 概率↑ + rejected 概率↓** → 对齐人类偏好
- **β 控制偏离参考模型程度**，通常 0.1-0.5
- **比 PPO 简单很多**: 不需要 reward model 和 critic
- **变体**: IPO, KTO, ORPO, SimPO 等
- **面试常问**: DPO 和 PPO 的区别？为什么 DPO 更简单？
- **参考模型的 logp 必须 no_grad**: 它只是固定锚点，只有策略一侧回传梯度

GRPO 损失

Group Relative Policy Optimization

去掉 Critic，组内归一化优势，DeepSeek-R1

📌 原理与思想

核心概念

PPO 的简化版：对同一问题生成 G 个回答，用组内归一化的奖励作为优势，不需要 Critic 网络。相比 PPO 节省约 40% 训练显存，是 DeepSeek-R1 使用的强化学习算法。

核心思想

对同一问题生成 G 个回答，用组内归一化的奖励代替 Critic 网络的价值估计。优势函数 A_i = (r_i - mean(r)) / (std(r) + ε)，结合 PPO clip 机制和显式 KL 惩罚。

组内的优势必然有正有负，梯度才有明确方向；clip 限制单步更新幅度，显式 KL 惩罚则拉住参考策略、防止跑偏。对比 PPO：PPO 要先训一个 Critic 估计 V(s)，再用 A = r + γV(s) − V(s)，GRPO 省掉的正是这一步。

算法步骤

1. 对同一问题 q 生成 G 个回答：o_1, o_2, ..., o_G
2. 计算每个回答的奖励：r_1, r_2, ..., r_G
3. 组内归一化优势：A_i = (r_i - mean) / (std + ε)
4. 计算 PPO clip 损失
5. 添加显式 KL 惩罚：loss += β · KL(π||π_ref)

📐 核心公式

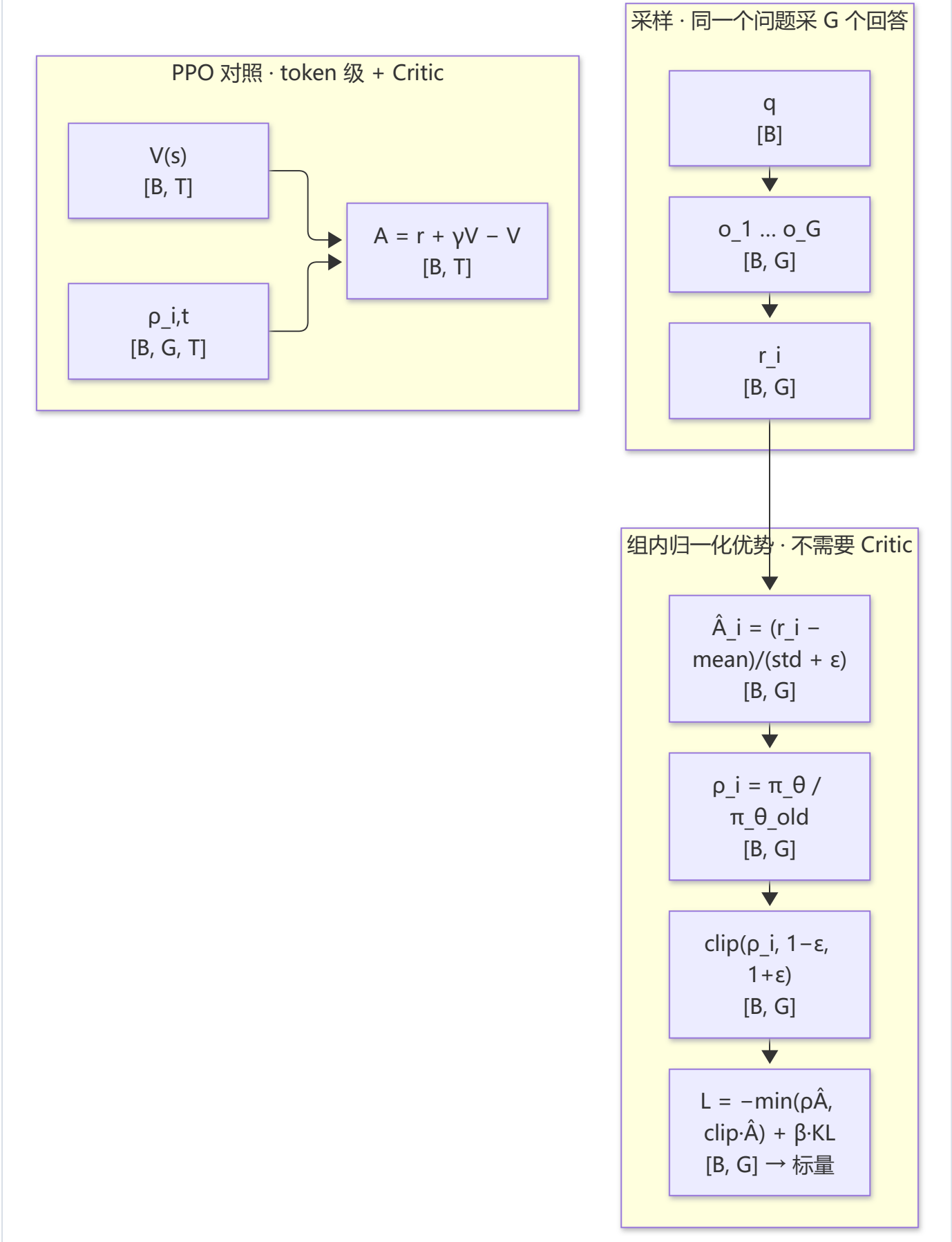
$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r}) + \epsilon}, \qquad \{o_1, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$$

$$\mathcal{L}(\theta) = -\mathbb{E}\Big[\min\Big(\rho_i \hat{A}_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\,\hat{A}_i\Big)\Big] + \beta\,\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

$$\text{GRPO (sequence-level):} \quad \rho_i = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}$$

$$\text{PPO (token-level):} \quad \rho_{i,t} = \frac{\pi_\theta(o_{i,t} \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid x, y_{<t})}$$

张量流程图


采样 · 同一个问题采 G 个回答
q
[B]
o_1 … o_G
[B, G]
r_i
[B, G]
PPO 对照 · token 级 + Critic
V(s)
[B, T]
ρ_i,t
[B, G, T]
A = r + γV − V
[B, T]
组内归一化优势 · 不需要 Critic
Â_i = (r_i − mean)/(std + ε)
[B, G]
ρ_i = π_θ / π_θ_old
[B, G]
clip(ρ_i, 1−ε, 1+ε)
[B, G]
L = −min(ρÂ, clip·Â) + β·KL
[B, G] → 标量

💻 代码实现

```python
import torch

def grpo_advantages(rewards):
    """组内归一化优势 (不需要 Critic)"""
    # rewards: [batch, group_size]
    mean = rewards.mean(dim=-1, keepdim=True)
    std = rewards.std(dim=-1, keepdim=True)
    return (rewards - mean) / (std + 1e-8)

def grpo_loss(old_logp, new_logp, advantages, eps=0.2, beta=0.01, ref_kl=None):
    """
    GRPO 损失 = PPO clip loss + β * KL penalty
    """
    ratio = torch.exp(new_logp - old_logp)
    clipped = torch.clamp(ratio, 1-eps, 1+eps)
    loss = -torch.min(ratio * advantages, clipped * advantages)
    if ref_kl is not None:
        loss = loss + beta * ref_kl       # 显式 KL 惩罚
    return loss.mean()

def kl_penalty(logp, ref_logp):
    """Schulman KL 估计器 (低方差)"""
    ratio = torch.exp(ref_logp - logp)
    return (ratio - (ref_logp - logp) - 1).mean()

# 完整 GRPO 训练流程
class GRPOTrainer:
    def __init__(self, policy_model, ref_model, reward_model, group_size=8):
        self.policy = policy_model
        self.ref = ref_model
        self.reward = reward_model
        self.group_size = group_size

    def train_step(self, prompts):
        # 1. 对每个 prompt 生成 group_size 个回答
        responses = []
        for _ in range(self.group_size):
            responses.append(self.policy.generate(prompts))

        # 2. 计算每个回答的 reward
        rewards = torch.stack([
            self.reward(prompts, resp) for resp in responses
        ], dim=1)  # [batch, group_size]

        # 3. 组内归一化优势
        advantages = grpo_advantages(rewards)

        # 4. GRPO 更新
        for i in range(self.group_size):
            old_logp = self.policy.get_logprobs(prompts, responses[i])
            new_logp = self.policy.get_logprobs(prompts, responses[i])
            ref_logp = self.ref.get_logprobs(prompts, responses[i])

            kl = kl_penalty(new_logp, ref_logp)
            loss = grpo_loss(old_logp, new_logp, advantages[:, i],
```

```
                            ref_kl=kl)
            loss.backward()

# 测试
if __name__ == "__main__":
    # 组内归一化
    rewards = torch.tensor([[1.0, 2.0, 3.0, 0.5],
                            [2.0, 2.5, 1.5, 3.0]])
    advantages = grpo_advantages(rewards)
    print("Advantages:", advantages)
    # 每行均值为 0，标准差为 1
```

🎯 面试要点

- **核心**: 组内归一化代替 Critic 网络
- **对同一问题生成 G 个回答**，计算组内相对优势
- **不需要 Value Head**，节省约 40% 训练显存
- **显式 KL 惩罚**防止偏离参考策略太远
- **DeepSeek-R1 使用 GRPO** 进行强化学习
- **面试常问**: GRPO 和 PPO 的区别？为什么不需要 Critic?

组序列策略优化

Group Sequence Policy Optimization

GRPO 的比率提到序列级，MoE 训练的稳定解

📌 原理与思想

核心概念

GRPO 的重要性比率是 token 级的，每个 token 各自 clip；而奖励本身是序列级的（整条回答对错），粒度对不上。这种不匹配带来高方差：同一条序列里一部分 token 被裁掉、另一部分照常更新，序列内部的更新方向互相拉扯，在 MoE 模型上还会放大路由抖动导致训练发散。GSPO 把重要性比率定义在序列级 —— 对逐 token 对数比做长度归一化，整条序列共享一个标量比率、只 clip 一次；代价是粒度变粗，序列内部个别 token 的差异会被平均掉。

核心思想

既然奖励是序列级的一个标量，比率也应该是序列级的一个标量：整条序列共享同一个比率，只 clip 一次。把逐 token 的对数比先按 mask 求和、再除以序列长度，得到长度归一化的平均对数比；取指数就得到序列级比率 `s_i`，它衡量的是「整条回答在当前策略下比旧策略平均好多少」。

算法步骤

1. 组内优势 `Â_i` 与 GRPO 完全一致：`(r_i - mean(r)) / (std(r) + ε)`。
2. 逐 token 对数比 `[B, G, T]`，按 mask 求和压掉 padding → `[B, G]`，再除以 `|y_i|` 做长度归一化。
3. `s_i = exp(长度归一化的对数比)`，形状 `[B, G]` —— 每条序列一个标量，这正是「序列级」的含义。
4. 目标函数与 PPO 同形，只是把 `ρ` 换成 `s_i`：`min(s_i·Â_i, clip(s_i, 1-ε, 1+ε)·Â_i)`。

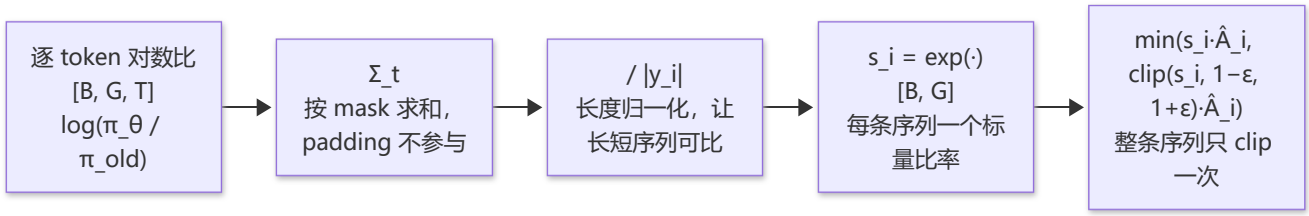
📐 核心公式

$$\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \epsilon}$$

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i \mid x)}{\pi_{\text{old}}(y_i \mid x)} \right)^{1/|y_i|} = \exp\left(\frac{1}{|y_i|} \sum_t \log \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t} \mid x, y_{i,<t})} \right)$$

$$\mathcal{L}_{\text{GSPO}} = -\mathbb{E}\left[\min\left(s_i \hat{A}_i,\ \text{clip}(s_i,\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_i \right) \right], \qquad \epsilon = 3 \times 10^{-4}$$

📊 张量流程图



💻 代码实现

```
import torch

def sequence_ratio(per_token_logp_new, per_token_logp_old, completion_mask):
    """
    序列级重要性比率（长度归一化）
    per_token_logp_*: [B, G, T]  每个 token 的 log π
    completion_mask:  [B, G, T]  1 = 该 token 属于回答
    返回: [B, G]
    """
    # 1. 逐 token 对数比
    log_ratio = per_token_logp_new - per_token_logp_old        # [B, G, T]

    # 2. 长度归一化后求和，再取 exp
    mask = completion_mask.float()
    length = mask.sum(dim=-1).clamp_min(1.0)                   # [B, G]
    avg_log_ratio = (log_ratio * mask).sum(dim=-1) / length    # [B, G]
    return avg_log_ratio.exp()


def gspo_loss(per_token_logp_new, per_token_logp_old, completion_mask,
              advantages, eps=3e-4):
```

```
    """
    GSPO: 整条序列共享一个比率，只 clip 一次
    advantages: [B, G]  组内归一化优势（同 GRPO）
    """
    s = sequence_ratio(per_token_logp_new, per_token_logp_old, completion_mask)
    clipped = torch.clamp(s, 1 - eps, 1 + eps)
    obj = torch.min(s * advantages, clipped * advantages)
    return -obj.mean()


if __name__ == "__main__":
    torch.manual_seed(0)
    B, G, T = 2, 4, 6

    old_logp = torch.randn(B, G, T) - 2.0
    new_logp = old_logp + 0.01 * torch.randn(B, G, T)    # 一次轻微更新

    mask = torch.ones(B, G, T)
    mask[:, :, -2:] = 0                                  # 后 2 个是 padding

    advantages = torch.randn(B, G)
    advantages = (advantages - advantages.mean(-1, keepdim=True)) / \
                 (advantages.std(-1, keepdim=True) + 1e-8)

    s = sequence_ratio(new_logp, old_logp, mask)
    print("序列级比率（应接近 1）:", round(s.mean().item(), 6))
    print("比率范围:", round(s.min().item(), 6), "~", round(s.max().item(), 6))
    print("gspo loss:", gspo_loss(new_logp, old_logp, mask, advantages).item())
```

🎯 面试要点

- **一句话**: GSPO = GRPO + 把重要性比率从 token 级换成序列级（长度归一化）
- **为什么 MoE 更需要它**: token 级比率的高方差会放大专家路由抖动，序列级比率把整条序列的更新绑成一个方向，显著更稳
- **长度归一化不可省**: 不归一化的话，长序列的比率会指数级偏离 1，clip 之后梯度全为 0
- **clip 粒度变了，ε 也要跟着变**: 序列级比率偏离 1 的幅度很小，所以 `ε` 取 `3e-4` 量级，比 PPO 的 `0.2` 小几个数量级
- **和 GRPO 的关系**: 目标函数形式完全一样，`min(·, clip(·))` 的骨架没动，只换了比率的定义
- **面试常问**: 序列级和 token 级重要性比率的区别？为什么 MoE 用 token 级比率容易崩？

在线策略蒸馏

On-Policy Distillation

学生自己生成轨迹，教师逐 token 给稠密监督

📌 原理与思想

核心概念

让学生模型自己采样生成轨迹，再让教师模型在学生实际走过的每个 token 上给出完整分布作为监督信号。相比用教师生成的静态数据做离线蒸馏，on-policy 训练消除了训练与推理的分布不匹配（exposure bias）；相比只有稀疏结果奖励的 RL，教师的逐 token 分布本身就是一个稠密奖励，不需要额外的 reward model。代价是每个 batch 都要现场采样，且必须有一个更强的教师模型可用。

核心思想

让学生自己采样，教师在被采样出来的轨迹上逐 token 给出完整分布 —— 监督落在学生真正会走的路径上，训练分布与推理分布从此一致，exposure bias 被从根上消掉。教师的分布在每个位置都是一个几万维的概率向量，本身就是一个稠密奖励，不需要 reward model。方向用反向 KL：mode-seeking 让学生只去对齐教师的高概率模式，与 Hinton 蒸馏正好相反。

算法步骤

1. 学生前向并采样得到 `y_S`，它带着学生自己的错误 —— 这正是要让它学会纠正的地方。
2. 教师在同一批 `prompt + y_S` 上做一次前向，得到 `teacher_logits: [B, T, V]`，必须 `detach` / `no_grad`。
3. 逐 token 算散度 `D(p_T ‖ p_S)`，得到 `per_token_loss: [B, T]`，再按有效 token 数求平均。
4. 用广义 JSD 插值统一方向：`β = 0` 是前向 KL（mode-covering）、`β = 1` 是反向 KL（mode-seeking）、`β = 0.5` 就是标准 JSD。

📐 核心公式

$$\mathcal{L}_{\text{OPD}} = \mathbb{E}_{y \sim \pi_S(\cdot|x)} \left[\frac{1}{|y|} \sum_t D(p_T(\cdot \mid x, y_{<t}) \,\|\, p_S(\cdot \mid x, y_{<t})) \right]$$

$$M = \beta\, p_T + (1-\beta)\, p_S$$

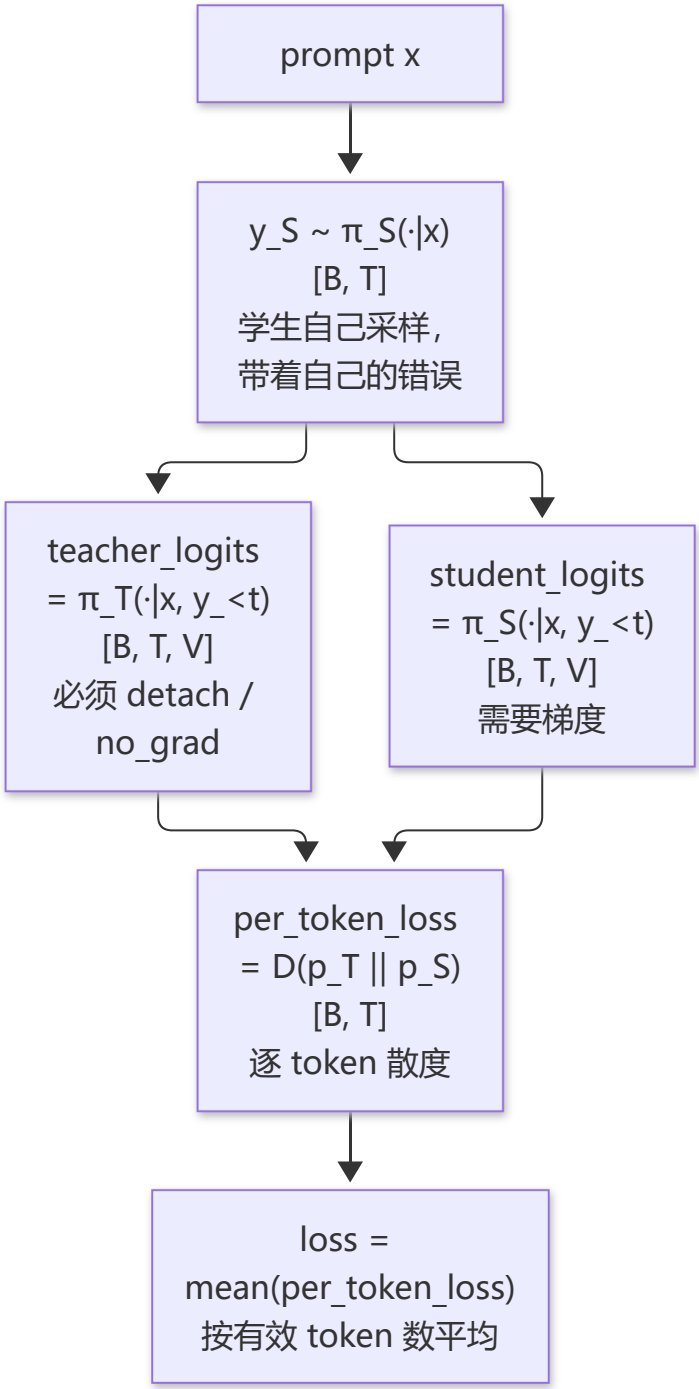
$$D_{\text{GJS}}(\beta) = (1-\beta)\text{KL}(p_T \| M) + \beta \text{KL}(p_S \| M), \qquad \beta \in [0,1]$$

$$\beta = 0 \Rightarrow \text{KL}(p_T \| p_S) \qquad \text{forward KL (mode-covering)}$$

$$\beta = 1 \Rightarrow \text{KL}(p_S \| p_T) \qquad \text{reverse KL (mode-seeking)}$$

$$\beta = 0.5 \Rightarrow \text{standard JSD}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn.functional as F

def generalized_jsd(student_logits, teacher_logits, beta=1.0, temperature=1.0):
    """
    广义 JSD 插值
    beta = 0   → 前向 KL  KL(p_T ‖ p_S)   (mode-covering)
    beta = 1   → 反向 KL  KL(p_S ‖ p_T)   (mode-seeking)
```

```
    beta = 0.5 → 标准 JSD
    student_logits, teacher_logits: [B, T, V]，未归一化的 logits
    返回逐 token 散度 [B, T]
    """
    # 1. 温度缩放 + log_softmax（数值稳定）
    s_logp = F.log_softmax(student_logits / temperature, dim=-1)
    t_logp = F.log_softmax(teacher_logits / temperature, dim=-1)
    s_p = s_logp.exp()
    t_p = t_logp.exp()

    # 2. 混合分布 M = beta * p_T + (1 - beta) * p_S
    #    clamp_min 只是防止 log(0)，V 个 token 上概率不会真的到 0
    m_logp = (beta * t_p + (1 - beta) * s_p).clamp_min(1e-8).log()

    # 3. KL(p_T ‖ M) 与 KL(p_S ‖ M)，在词表维度求和
    kl_t = (t_p * (t_logp - m_logp)).sum(dim=-1)     # [B, T]
    kl_s = (s_p * (s_logp - m_logp)).sum(dim=-1)     # [B, T]

    return (1 - beta) * kl_t + beta * kl_s           # [B, T]


def opd_loss(student_logits, teacher_logits, loss_mask=None,
             beta=1.0, temperature=1.0, top_k=0):
    """
    在线策略蒸馏损失
    loss_mask: [B, T]，1 = 学生在该位置真实采样出的 token
    top_k:     >0 时只保留教师概率最高的 k 个 token
               （黑盒教师只吐 top-k logits 时用，其余位置截断）
    """
    # 教师永远不参与反传，否则「教师」会被同一步更新带跑
    teacher_logits = teacher_logits.detach()

    if top_k > 0:
        # 教师只给 top-k：其余位置截断到一个很大的负数
        # 用 -1e9 而不是 -inf，避免 0 * inf 产生 nan
        kth = teacher_logits.topk(top_k, dim=-1).values[..., -1:]
        teacher_logits = teacher_logits.masked_fill(teacher_logits < kth, -1e9)

    per_token = generalized_jsd(student_logits, teacher_logits, beta, temperature)

    if loss_mask is None:
        return per_token.mean()
    mask = loss_mask.float()
    return (per_token * mask).sum() / mask.sum().clamp_min(1.0)


if __name__ == "__main__":
    torch.manual_seed(0)
    B, T, V = 2, 4, 8
    student = torch.randn(B, T, V)
    teacher = torch.randn(B, T, V)

    # 端点自检：beta=0 应等于前向 KL，beta=1 应等于反向 KL
    s_logp = F.log_softmax(student, dim=-1)
    t_logp = F.log_softmax(teacher, dim=-1)
    # F.kl_div(input, target, log_target=True) = KL(target ‖ input)
    fwd = F.kl_div(s_logp, t_logp, log_target=True, reduction='none').sum(-1)
    rev = F.kl_div(t_logp, s_logp, log_target=True, reduction='none').sum(-1)
```

```
    print("beta=0  :", generalized_jsd(student, teacher, beta=0.0).mean().item())
    print("  前向KL:", fwd.mean().item())
    print("beta=1  :", generalized_jsd(student, teacher, beta=1.0).mean().item())
    print("  反向KL:", rev.mean().item())
    print("beta=0.5:", generalized_jsd(student, teacher, beta=0.5).mean().item(), "(标准 JSD)")

    # top-k 教师 + mask
    mask = torch.ones(B, T)
    mask[:, -1] = 0
    print("top_k=3:", opd_loss(student, teacher, mask, beta=1.0, top_k=3).item())
```

🎯 面试要点

- **为什么要 on-policy**: 消除 exposure bias。离线蒸馏的误差累积是 $O(T^2)$，on-policy 降到 $O(T)$
- **为什么常选反向 KL**: 反向 KL 是 mode-seeking，学生只去匹配教师的高概率模式；前向 KL 是 mode-covering，会给教师几乎不给概率的区域也分配质量
- **KL 方向决定行为**: 前向 KL 覆盖所有模式但分布更散（易产生幻觉区域），反向 KL 更尖锐但可能丢多样性
- **与 RL 的区别**: 不需要 reward model，教师逐 token 的分布就是稠密奖励；同等效果下比 RL 便宜得多
- **教师的 logits 必须 detach**: 教师是「固定靶」，只有学生一侧回传梯度
- **黑盒教师**: 拿不到全词表 logits 时，用 top-k logits 截断近似，其余位置填一个很大的负数（不要填 `-inf`，会在 `0 * inf` 处产生 nan）
- **广义 JSD 一个公式统一了前后向 KL**: `β` 从 0 滑到 1，就是从 mode-covering 滑到 mode-seeking，代码里用端点数值断言自检
- **面试常问**: on-policy 蒸馏和 RLHF 是什么关系？为什么 OPD 不需要 reward model?

PPO 损失

Proximal Policy Optimization

截断重要性采样比率，RLHF 核心

📌 原理与思想

核心概念

通过截断重要性采样比率 r_t = π_new/π_old 到 [1-ε, 1+ε]，限制策略更新幅度，防止策略崩溃。相比普通策略梯度容易更新过大导致崩溃，PPO 通过 clip 机制保证训练稳定性，是 ChatGPT/InstructGPT 的 RLHF 核心算法。

核心思想

重要性采样比率 r_t 衡量新旧策略的差异。通过 clip 将 r_t 限制在 [1-ε, 1+ε] 范围内，当 A>0 时防止 ratio 过大（过度奖励），当 A<0 时防止 ratio 过小（过度惩罚），实现保守更新。

算法步骤

1. 计算重要性采样比率：ratio = exp(new_logp - old_logp)
2. 截断比率：clipped = clamp(ratio, 1-ε, 1+ε)
3. 计算未截断代理损失：surr1 = ratio × advantages
4. 计算截断代理损失：surr2 = clipped × advantages
5. 取较小值：loss = -mean(min(surr1, surr2))

📐 核心公式

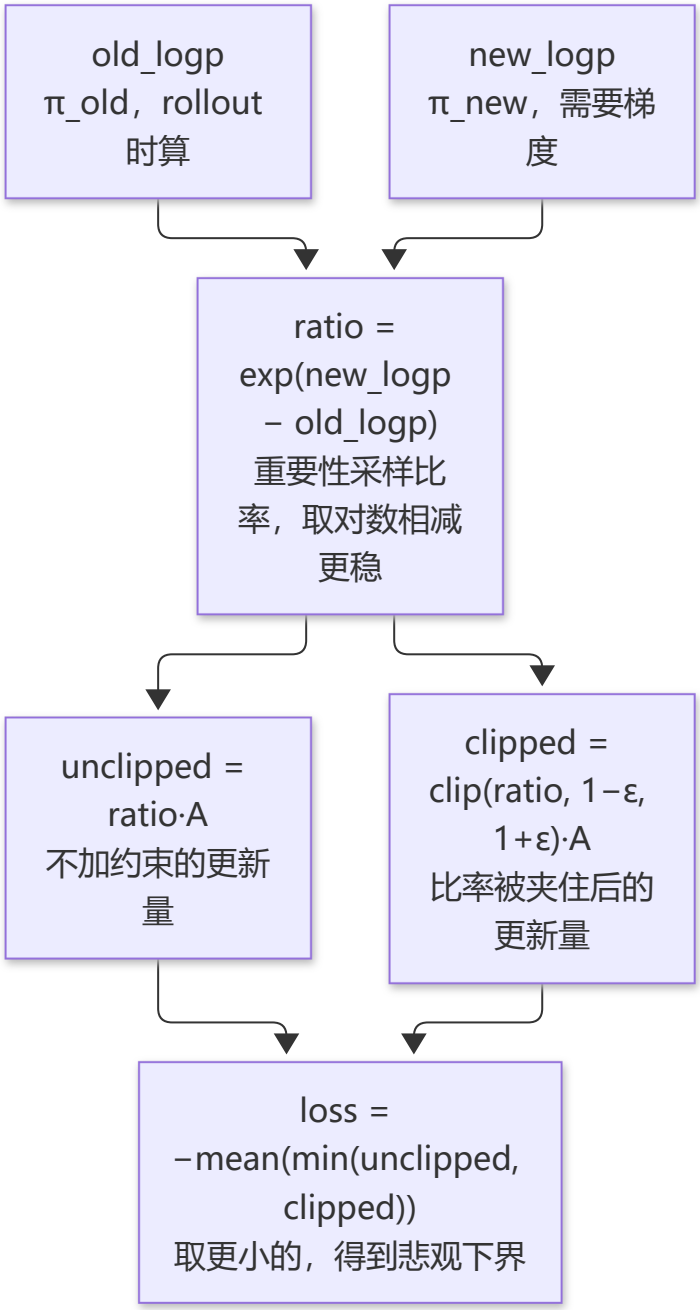
$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t\left[\min\left(r_t A_t,\ \text{clip}\left(r_t,\,1-\epsilon,\,1+\epsilon\right) A_t\right)\right]$$

$$r_t = \exp\left(\log \pi_\theta^{\text{new}} - \log \pi_\theta^{\text{old}}\right), \qquad A_t = \text{GAE advantage}, \qquad \epsilon = 0.2$$

$$A_t > 0: \quad r_t > 1+\epsilon \ \Rightarrow\ r_t A_t \text{ capped at } (1+\epsilon)A_t$$

$$A_t < 0: \quad r_t < 1-\epsilon \ \Rightarrow\ r_t A_t \text{ floored at } (1-\epsilon)A_t$$

📊 张量流程图



💻 代码实现

```python
import torch

def ppo_loss(old_logp, new_logp, advantages, eps=0.2):
    """
    PPO 截断损失

    old_logp, new_logp: [B]  对数概率
    advantages: [B]  优势估计 (GAE)
    """
    ratio = torch.exp(new_logp - old_logp)          # r_t
```

```
    clipped = torch.clamp(ratio, 1-eps, 1+eps)       # clip(r_t)

    surr1 = ratio * advantages                        # 未截断
    surr2 = clipped * advantages                      # 截断

    return -torch.min(surr1, surr2).mean()            # 保守更新

# 完整 PPO 训练流程
class PPOTrainer:
    def __init__(self, policy_model, ref_model, reward_model, critic_model):
        self.policy = policy_model
        self.ref = ref_model
        self.reward = reward_model
        self.critic = critic_model

    def train_step(self, prompts, responses):
        # 1. 计算 reward
        rewards = self.reward(prompts, responses)

        # 2. 计算 value 和 advantage
        values = self.critic(prompts, responses)
        advantages = compute_gae(rewards, values)

        # 3. 计算 log probs
        old_logp = self.policy.get_logprobs(prompts, responses)

        # 4. PPO 更新
        for _ in range(ppo_epochs):
            new_logp = self.policy.get_logprobs(prompts, responses)
            loss = ppo_loss(old_logp, new_logp, advantages)

            # KL penalty (防止偏离参考模型太远)
            ref_logp = self.ref.get_logprobs(prompts, responses)
            kl = (new_logp - ref_logp).mean()
            loss += kl_coef * kl

            loss.backward()
            optimizer.step()

# 测试
if __name__ == "__main__":
    B = 32
    old_logp = torch.randn(B)
    new_logp = old_logp + torch.randn(B) * 0.1
    advantages = torch.randn(B)

    loss = ppo_loss(old_logp, new_logp, advantages, eps=0.2)
    print(f"PPO Loss: {loss.item():.4f}")
```

🎯 面试要点

- **ratio = exp(new_logp - old_logp)** 重要性采样比率
- **clip 到 [1-ε, 1+ε]** 限制更新幅度
- **A>0**: 防止 ratio 过大 → 停止奖励

- **A<0**: 防止 ratio 过小 → 停止惩罚
- **ChatGPT/InstructGPT 的 RLHF 核心算法**
- **需要 reward model + critic model + reference model**
- **面试常问**: PPO 的 clip 机制是如何工作的?
- ε 通常取 0.2；ratio 必须用旧策略的 logp 现算，rollout 与更新要严格配对

在线自蒸馏

On-Policy Self-Distillation

同一个模型既是教师又是学生，用特权上下文造出更强的自己

📌 原理与思想

核心概念

不再依赖外部更强的教师模型：让学生自己采样，同时把同一份权重在特权上下文（参考答案、关键提示、解题方向等）条件下的分布当作教师分布。学生在没有特权信息的条件下学习，把训练时才有的额外信息转成训练信号；教师与学生共享参数，因此既不需要额外的教师显存，也不存在师生能力差距过大导致的负迁移。代价是特权上下文的构造成了新的依赖，它的质量直接决定收益上限。

核心思想

把「更强的模型」换成「信息更全的自己」：同一份权重 `θ`，一边看 prompt，一边额外拼接特权上下文 `c`。两条路径共享参数，教师那条只是多了一段输入，因此既不需要额外显存，也不存在能力差距。学生学的是「在没有特权信息的条件下逼近有特权信息时的分布」，等于把训练期才有的信息蒸馏进参数。

算法步骤

1. 学生路径 `π_θ(·|x)` 必须真采样得到 `y ~ π_θ`，这条路径需要梯度。
2. 教师路径 `π_θ(·|x ⊕ c)` 用同一份权重，只需一次前向，`no_grad` 即可。
3. 逐 token 反向 KL：`KL(π_θ(·|x, y_<t) ‖ π_θ(·|x ⊕ c, y_<t))`，只在 `y` 自己的 token 上回传。
4. 用反向 KL（mode-seeking）而不是前向：学生只对齐「信息更全的自己」的高概率模式，不强行覆盖全部尾巴。

📐 核心公式

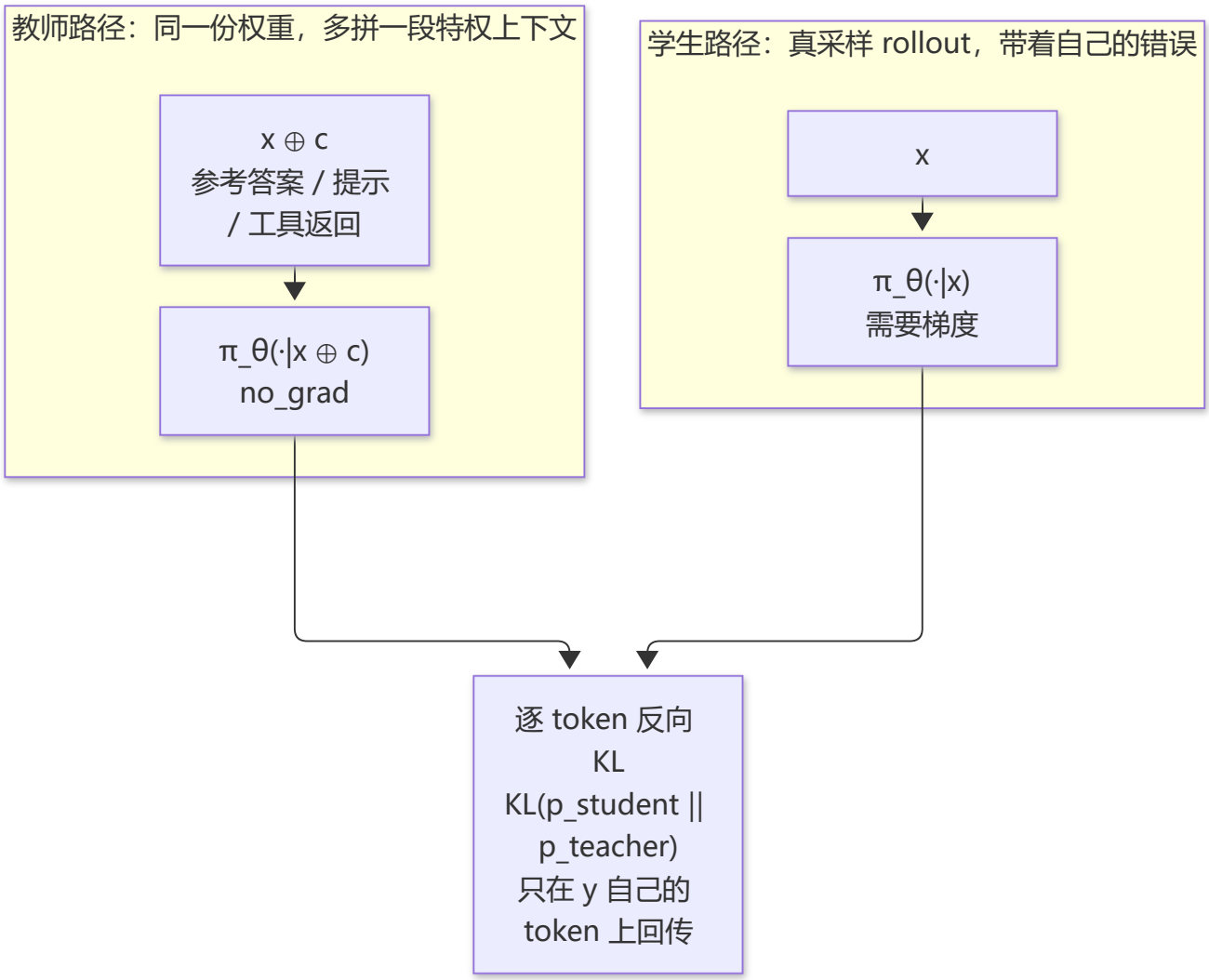
$$\text{student}: \pi_\theta(\cdot \mid x)\ \ (\text{rollout, gradient}), \qquad \text{teacher}: \pi_\theta(\cdot \mid x \oplus c)\ \ (\text{one forward, no_grad})$$

$$\mathcal{L} = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\frac{1}{|y|} \sum_t \mathrm{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \,\|\, \pi_\theta(\cdot \mid x \oplus c, y_{<t})) \right]$$

$$\text{forward KL}: \mathrm{KL}(\pi_T \| \pi_S) \qquad \text{vs} \qquad \text{reverse KL}: \mathrm{KL}(\pi_S \| \pi_T)$$

$$c \in \{\text{reference answer},\ \text{hint},\ \text{strategy name},\ \text{tool output},\ \text{user correction}\}$$

张量流程图



代码实现

```python
import torch
import torch.nn.functional as F
from collections import namedtuple

LMOutput = namedtuple("LMOutput", ["logits"])


@torch.no_grad()
def teacher_forward(model, input_ids, privileged_ids):
    """教师 = 同一模型 + 特权上下文，只做一次前向，不需要梯度"""
    full = torch.cat([privileged_ids, input_ids], dim=1)
    logits = model(full).logits
    # 只取答案段（去掉特权上下文那一段）
    return logits[:, privileged_ids.size(1):]
```

```python
def self_opd_loss(model, input_ids, privileged_ids, answer_mask):
    """
    在线自蒸馏损失
    input_ids:      [B, T] 学生看到的 prompt + 学生自己采样的回答
    privileged_ids: [B, P] 特权上下文（如参考答案），只拼给教师
    answer_mask:    [B, T] 1 = 学生在该位置真实生成的 token
    """
    # 1. 学生前向（需要梯度）—— 与 rollout 时是同一批 token
    student_logits = model(input_ids).logits                      # [B, T, V]

    # 2. 教师前向：同一权重 + 特权上下文，no_grad
    t_logits = teacher_forward(model, input_ids, privileged_ids)   # [B, T, V]

    # 3. 逐 token 反向 KL：KL(p_S ‖ p_T)
    s_logp = F.log_softmax(student_logits, dim=-1)
    t_logp = F.log_softmax(t_logits, dim=-1)
    kl = (s_logp.exp() * (s_logp - t_logp)).sum(dim=-1)            # [B, T]

    # 4. 只在学生自己生成的 token 上回传
    mask = answer_mask.float()
    return (kl * mask).sum() / mask.sum().clamp_min(1.0)


if __name__ == "__main__":
    class ToyLM(torch.nn.Module):
        """玩具模型：真实场景是 HuggingFace 的 CausalLM，这里只验证流程"""
        def __init__(self, vocab=16, dim=8):
            super().__init__()
            self.emb = torch.nn.Embedding(vocab, dim)
            self.head = torch.nn.Linear(dim, vocab)

        def forward(self, ids):
            return LMOutput(self.head(self.emb(ids)))   # [B, T, V]

    torch.manual_seed(0)
    model = ToyLM()
    input_ids = torch.randint(0, 16, (2, 6))
    privileged = torch.randint(0, 16, (2, 3))

    answer_mask = torch.zeros(2, 6)
    answer_mask[:, 3:] = 1.0          # 后 3 个 token 是学生自己生成的

    loss = self_opd_loss(model, input_ids, privileged, answer_mask)
    print("self-opd loss:", loss.item())

    loss.backward()
    print("学生一侧收到梯度:", model.head.weight.grad is not None)
    print("输出形状:", (2, 6), "-> 标量")
```

🎯 面试要点

- **与 OPD 的核心区别**: OPD 的教师是另一个更强的模型；self-OPD 的教师是同一份权重 + 特权上下文
- **为什么需要它**: OPD 需要外部更强的教师，但到了 SOTA 之上往往没有更强的模型可用；而特权信息训练时拿得到、推理时拿不到，正好当监督信号

- **特权上下文的粒度很关键**: 给完整参考答案不一定最优 —— 中间抽象（解题策略名、方法方向、问题类别）往往在更少 hint token 下效果更好，因为完整答案会让教师分布过分偏离学生当前能力
- **必须是同一个模型**: 教师和学生共享参数，所以不存在「师生能力差距过大」导致的负迁移
- **rollout 必须 on-policy**: 训练 token 要来自学生自己的采样，否则退化成普通的上下文蒸馏
- **已知副作用**: 反向 KL 是 mode-seeking，长期训练会压缩输出多样性、让模型变「刚性」，需要靠 hint 设计、散度方向和训练步数来调节
- **用武之地**: 持续学习（把推理时的信息固化成权重）、利用隐式用户反馈做定向纠错
- **面试常问**: 没有外部教师时怎么蒸馏？特权信息在推理时拿不到，为什么还能当监督信号？

⚙ **优化器** Optimizer　　2 题

AdamW 优化器

解耦权重衰减，LLM 训练标配

📌 原理与思想

核心概念

Adam 的改进版：将权重衰减从梯度中解耦，直接作用于参数。相比 Adam 的权重衰减作用在梯度上正则化效果差，AdamW 解耦后正则化效果更好，是 LLM 训练的标准优化器。

核心思想

维护一阶矩（动量）和二阶矩（未中心化的方差），通过偏差修正确保初始阶段的稳定性。权重衰减直接作用于参数而非梯度，实现解耦正则化。

算法步骤

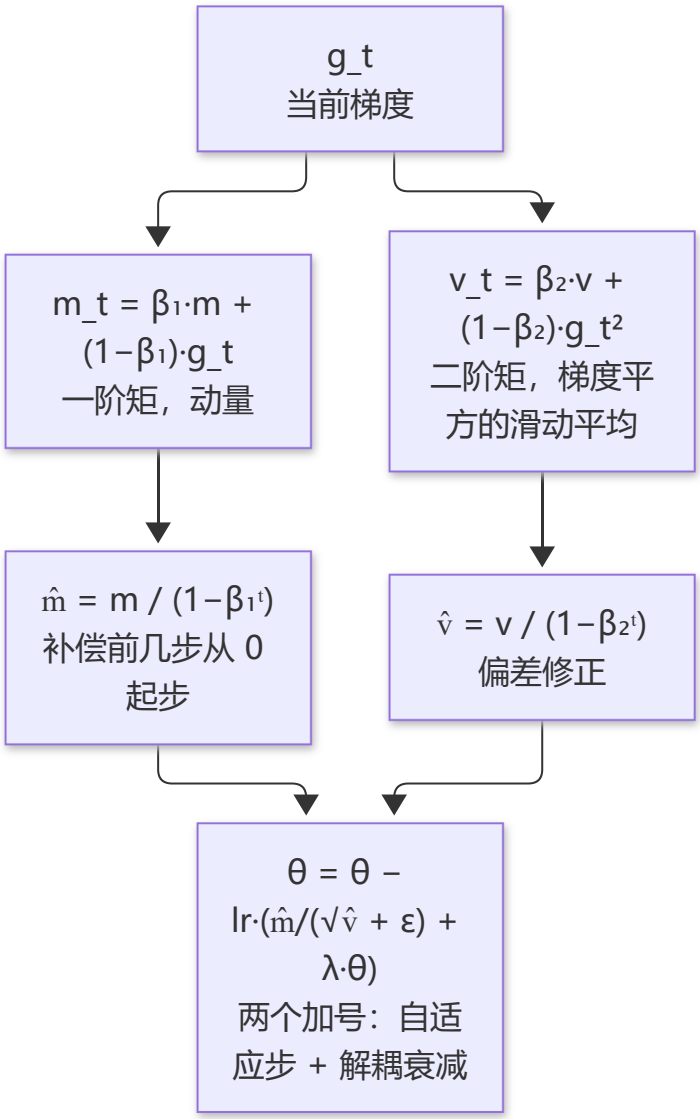
1. 更新一阶矩：$m_t = \beta_1 \cdot m + (1-\beta_1) \cdot g$
2. 更新二阶矩：$v_t = \beta_2 \cdot v + (1-\beta_2) \cdot g^2$
3. 偏差修正：$\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$
4. 解耦权重衰减：$\theta = \theta - lr \cdot \lambda \cdot \theta$
5. 参数更新：$\theta = \theta - lr \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$

📐 核心公式

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1)\, g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \\
\theta_t &= \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \theta_{t-1} \right)
\end{aligned}
$$

$$
\eta = 3 \times 10^{-4}, \qquad (\beta_1, \beta_2) = (0.9,\ 0.95), \qquad \lambda = 0.1, \qquad \epsilon = 10^{-8}
$$

📊 张量流程图



💻 代码实现

```
import torch

class AdamW:
```

```
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999),
                 eps=1e-8, weight_decay=0.01):
        self.params = list(params)
        self.lr, self.eps = lr, eps
        self.b1, self.b2 = betas
        self.wd = weight_decay
        self.m = [torch.zeros_like(p) for p in self.params]
        self.v = [torch.zeros_like(p) for p in self.params]
        self.t = 0

    def step(self):
        self.t += 1
        for i, p in enumerate(self.params):
            if p.grad is None:
                continue
            g = p.grad.data

            # 解耦权重衰减 (AdamW vs Adam 的关键区别)
            if self.wd != 0:
                p.data.mul_(1 - self.lr * self.wd)

            # 更新矩
            self.m[i] = self.b1 * self.m[i] + (1-self.b1) * g
            self.v[i] = self.b2 * self.v[i] + (1-self.b2) * g**2

            # 偏差修正 + 更新
            m_hat = self.m[i] / (1 - self.b1**self.t)
            v_hat = self.v[i] / (1 - self.b2**self.t)
            p.data.add_(-self.lr * m_hat / (v_hat.sqrt() + self.eps))

# 学习率调度 (LLM 标配)
def cosine_schedule_with_warmup(step, warmup_steps, total_steps, max_lr=3e-4):
    if step < warmup_steps:
        return max_lr * (step / warmup_steps)
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return max_lr * 0.5 * (1 + torch.cos(torch.tensor(progress * 3.14159)))

# 测试
if __name__ == "__main__":
    model = torch.nn.Linear(10, 10)
    optimizer = AdamW(model.parameters(), lr=3e-4, weight_decay=0.1)

    x = torch.randn(4, 10)
    y = model(x)
    loss = y.sum()
    loss.backward()
    optimizer.step()
    print("AdamW step completed")
```

🎯 面试要点

- **AdamW vs Adam**: 权重衰减直接作用于参数，非梯度
- **偏差修正**解决初始阶段 m,v 偏小的问题
- **LLM 标配**: lr=3e-4, betas=(0.9,0.95), wd=0.1

- **配合 cosine schedule with warmup** 使用
- **面试常问**: AdamW 和 Adam 的区别？为什么要解耦权重衰减?
- **解耦为什么重要**: Adam 把 λθ 加进梯度再被 1/√v̂ 归一化，衰减强度会随梯度尺度失控；LLM 标配 lr=3e-4、betas=(0.9, 0.95)、weight_decay=0.1

Muon 优化器

Momentum Orthogonalized by Newton-Schulz

动量矩阵先正交化再更新，隐藏层权重的谱范数几何

📌 原理与思想

核心概念

Muon 只用于二维隐藏层权重：累积动量后，用 Newton-Schulz 迭代把动量矩阵近似成正交矩阵（极分解 `UVᵀ`）作为更新方向，让所有奇异方向等步长；嵌入层、输出头和所有一维参数仍交给 AdamW。规模化时 Muon 会把注意力 logit 推到爆炸，Kimi K2 用 QK-Clip 把 Q/K 权重乘 `√γ`（`γ = τ/S_max`）从源头压住。代价是每步多 5 次矩阵乘法，换来约 2× token 效率，正则与 RMS 对齐后 Adam 的超参可以直接迁移。

核心思想

在谱范数几何下做最速下降，最优更新方向是动量矩阵的极分解 `UVᵀ` —— 一个所有奇异值都等于 1 的半正交矩阵。直观说就是只留方向、抹掉幅度：每个奇异方向走同样大的一步，幅度交给学习率统一控制。求极分解不必真做 SVD（慢且数值敏感），Newton-Schulz 迭代只用矩阵乘法就能逼近。

算法步骤

1. 参数分组：只有 ≥2 维的隐藏层权重交给 Muon，embedding、lm_head 和所有 1D 参数（bias、norm）仍用 AdamW。
2. 累积动量 → 除以 F 范数归一化（保证谱范数 ≤ 1，否则迭代发散）→ 5 次 NS 迭代 → `θ ← θ - η·O`。
3. 5 步后奇异值落在约 `[0.5, 1.5]` 就够用：系数 `(3.4445, -4.7750, 2.0315)` 正是为「5 步内尽量压平」调出来的。

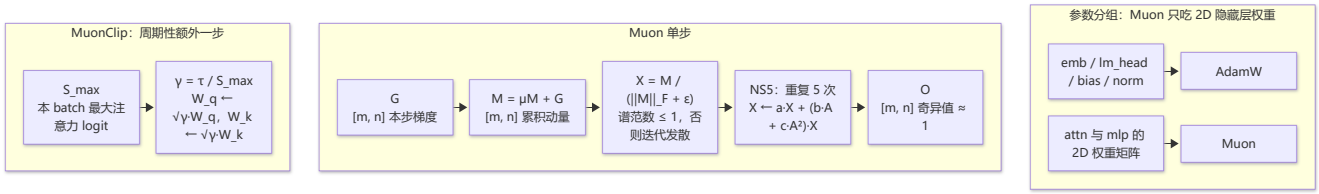
📐 核心公式

$$
\begin{aligned}
M_t &= \mu M_{t-1} + G_t, \qquad \mu \approx 0.95 \\
O_t &= \mathrm{NS}_5(M_t) \\
\theta_t &= \theta_{t-1} - \eta\, O_t
\end{aligned}
$$

$$
\begin{aligned}
X_0 &= \frac{M_t}{\|M_t\|_F + \epsilon} \\
A &= X_k X_k^\top \\
X_{k+1} &= aX_k + (bA + cA^2)X_k, \qquad k = 0, \ldots, 4 \\
(a, b, c) &= (3.4445,\ -4.7750,\ 2.0315)
\end{aligned}
$$

$$
\text{MuonClip:} \quad S_{\max} > \tau \Rightarrow \gamma = \frac{\tau}{S_{\max}}, \qquad W_q \leftarrow \sqrt{\gamma} W_q, \qquad W_k \leftarrow \sqrt{\gamma} W_k, \qquad \tau \approx 100
$$

📊 张量流程图



💻 代码实现

```python
import torch

def zeropower_via_newtonschulz5(G, steps=5, eps=1e-7):
    """
    用 Newton-Schulz 迭代近似 G 的极分解 UVᵀ（即「零次幂」）
    G: [m, n] 动量矩阵；返回同形状矩阵，奇异值趋近 1
    """
    assert G.ndim == 2, "Muon 只处理二维矩阵参数"
    a, b, c = 3.4445, -4.7750, 2.0315        # 调好的五次多项式系数

    X = G.to(torch.bfloat16)
    # 归一化：保证谱范数 ≤ 1，否则迭代会发散
    X = X / (X.norm() + eps)

    for _ in range(steps):
        A = X @ X.mT                         # [m, m]
        B = b * A + c * (A @ A)              # 五次多项式
        X = a * X + B @ X                    # [m, n]

    return X.to(G.dtype)


class Muon(torch.optim.Optimizer):
    """只用于 2D 隐藏层权重的 Muon 优化器"""

    def __init__(self, params, lr=0.02, momentum=0.95, nesterov=True, ns_steps=5):
        defaults = dict(lr=lr, momentum=momentum, nesterov=nesterov, ns_steps=ns_steps)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group["params"]:
                if p.grad is None:
```

```
                continue
            g = p.grad
            state = self.state[p]
            if "momentum_buffer" not in state:
                state["momentum_buffer"] = torch.zeros_like(g)

            buf = state["momentum_buffer"]
            buf.mul_(group["momentum"]).add_(g)
            # Nesterov: 用 g + μ·buf 作为动量方向
            update = g.add(buf, alpha=group["momentum"]) if group["nesterov"] else buf

            # 先正交化，再更新
            p.add_(zeropower_via_newtonschulz5(update, group["ns_steps"]),
                   alpha=-group["lr"])


@torch.no_grad()
def qk_clip_(model, tau=100.0):
    """
    MuonClip 的 QK-Clip: 从源头压住注意力 logit 的增长
    约定: 模块把本 batch 每 head 的最大 logit 存在 qk_max_logit 属性上
    """
    for module in model.modules():
        s_max = getattr(module, "qk_max_logit", None)
        if s_max is None or s_max <= tau:
            continue
        gamma = tau / s_max                      # 缩放因子
        scale = gamma ** 0.5                     # logit ∝ W_q·W_kᵀ，所以取 √γ
        module.q_proj.weight.mul_(scale)
        module.k_proj.weight.mul_(scale)


if __name__ == "__main__":
    torch.manual_seed(0)

    # 1. 正交化效果: 奇异值应该都趋近 1
    G = torch.randn(64, 32)
    O = zeropower_via_newtonschulz5(G)
    sv = torch.linalg.svdvals(O.float())
    print("原始奇异值范围:", round(torch.linalg.svdvals(G).min().item(), 3),
          "~", round(torch.linalg.svdvals(G).max().item(), 3))
    print("正交化后范围:", round(sv.min().item(), 3), "~", round(sv.max().item(), 3))

    # 2. Muon 走一步
    w = torch.nn.Parameter(torch.randn(64, 32))
    opt = Muon([w], lr=0.02)
    w.grad = torch.randn(64, 32)
    opt.step()
    print("Muon 一步更新完成，权重形状:", tuple(w.shape))
```

🎯 面试要点

- **为什么正交化**: 把动量矩阵的所有奇异值归一化为 1，让更新在每个奇异方向上等步长 —— 对应谱范数下的最速下降

- **为什么 5 步够**: Newton-Schulz 收敛很快，5 步后奇异值已经落在 `[0.5, 1.5]`；继续迭代收益很小而算力翻倍
- **bf16 能跑**: 迭代全是矩阵乘法，没有 SVD 那样的数值敏感操作，bfloat16 下稳定
- **Muon 只管 2D 权重**: 嵌入、输出头、所有 1D 参数（bias、norm）留给 AdamW —— 混合优化器是标准做法
- **Adam 与 Muon 的几何差别**: Adam 是逐坐标自适应步长，完全忽略权重矩阵的行列结构；Muon 在谱范数几何下让每个奇异方向等步长
- **规模化会炸 logit**: Muon 训到万亿参数时注意力 logit 会涨到远超正常量级（上千），QK-Clip 用 `√γ` 缩放 Q/K 权重从源头压住，且它是**无损**的（只维持数值稳定，不改变表达力）
- **为什么是 √γ**: logit 正比于 `W_q·W_kᵀ`，放缩一次权重，logit 就被放缩 γ 倍，所以权重取 √γ
- **QK-Clip 会自己退场**: 训练早期生效，模型稳定后 `S_max` 不再超阈值，机制自动不再触发
- **实测收益**: 相比 AdamW 约有 2× token 效率；正则与 RMS 对齐后 Adam 的超参可以直接迁移
- **面试常问**: Muon 和 Adam 的几何差别是什么？为什么 QK-Clip 用 √γ 而不是 γ?
- **MuonClip 的阈值**: 典型 τ = 100；logit 正比于 W_q·W_kᵀ，缩放一次权重等于缩放 γ 倍 logit，所以取 √γ

🎮 **强化学习** RL　　1 题

广义优势估计

GAE

偏差-方差折衷的 λ-return 优势估计

📌 原理与思想

核心概念

通过 λ 参数在蒙特卡洛（低偏差高方差）和 TD(0)（高偏差低方差）之间折衷。λ=1 等价于 MC（无偏差但高方差），λ=0 等价于 TD(0)（高偏差但低方差），GAE 通过 λ 在两者之间取得平衡。

核心思想

利用 TD error 的指数加权求和来估计优势函数。通过 λ 参数控制不同时间步 TD error 的权重，实现偏差-方差的折衷。

算法步骤

1. 计算 TD error：δ_t = r_t + γ·V(s_{t+1}) - V(s_t)
2. 从后向前递推：A_t = δ_t + γλ·A_{t+1}
3. 计算 returns：returns = advantages + values[:-1]

4. 归一化优势（可选）： advantages = (advantages - mean) / std

📐 核心公式

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$A_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l} = \delta_t + \gamma\lambda \, \delta_{t+1} + (\gamma\lambda)^2 \delta_{t+2} + \cdots$$

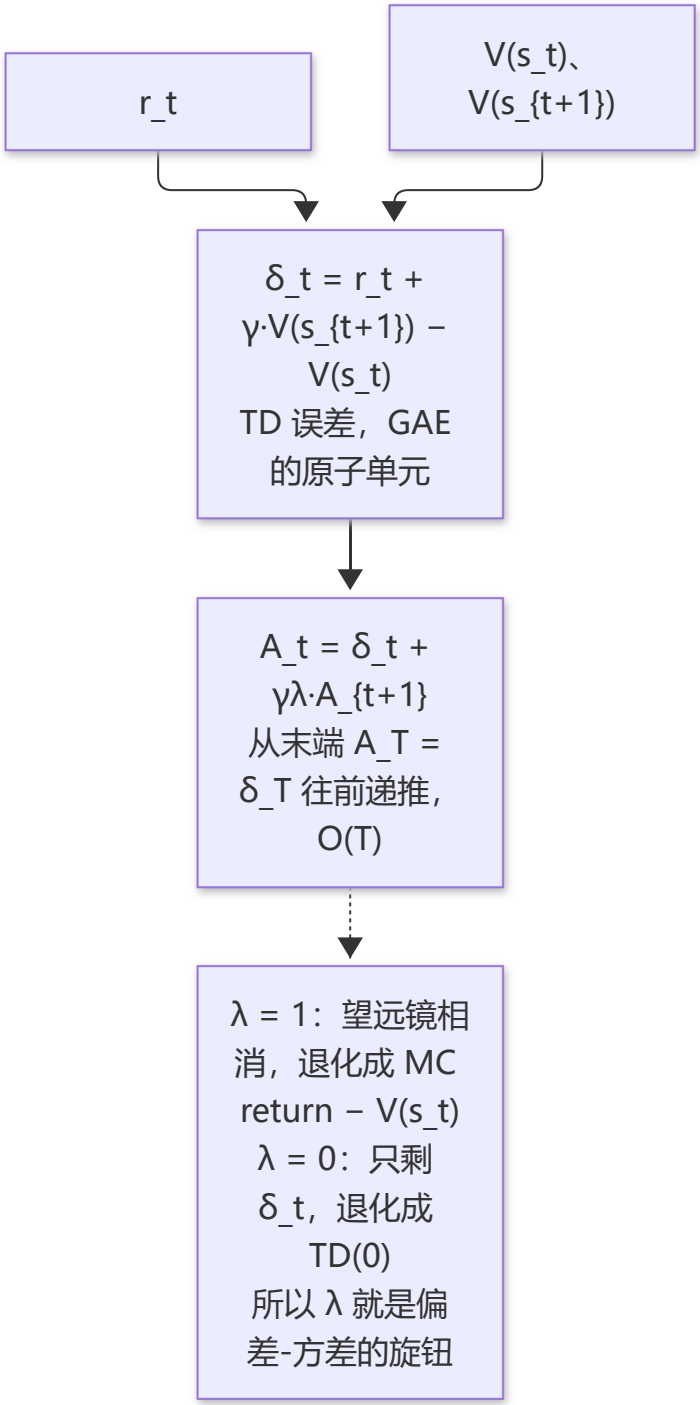
$$A_t = \delta_t + \gamma\lambda \, A_{t+1}, \qquad A_T = \delta_T$$

$$\lambda = 1: \quad A_t = \text{MC return} - V(s_t) \quad \text{(unbiased, high variance)}$$

$$\lambda = 0: \quad A_t = \delta_t \quad \text{(low variance, high bias)}$$

$$\gamma = 0.99, \qquad \lambda = 0.95$$

📊 张量流程图



💻 代码实现

```python
import torch

def compute_gae(rewards, values, gamma=0.99, lam=0.95):
    """
    计算 GAE 优势

    rewards: [T]   每步奖励
    values:  [T+1] 价值估计 (含最后一步)
```

```
    返回: advantages [T], returns [T]
    """
    T = len(rewards)
    advantages = torch.zeros(T)
    gae = 0.0

    for t in reversed(range(T)):
        delta = rewards[t] + gamma * values[t+1] - values[t]
        gae = delta + gamma * lam * gae       # 递推!
        advantages[t] = gae

    returns = advantages + values[:-1]         # returns = A + V
    return advantages, returns

# 测试
if __name__ == "__main__":
    T = 10
    rewards = torch.randn(T)
    values = torch.randn(T + 1)

    advantages, returns = compute_gae(rewards, values, gamma=0.99, lam=0.95)
    print(f"Advantages shape: {advantages.shape}")  # [10]
    print(f"Returns shape: {returns.shape}")        # [10]

    # 验证: returns = advantages + values[:-1]
    assert torch.allclose(returns, advantages + values[:-1])

# PPO 中使用
def ppo_step(model, data, gamma=0.99, lam=0.95):
    rewards = data['rewards']
    values = model.compute_values(data['states'])
    advantages, returns = compute_gae(rewards, values, gamma, lam)

    # 归一化优势
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

    # PPO 更新
    old_logp = model.get_logprobs(data['states'], data['actions'])
    for _ in range(4):  # PPO epochs
        new_logp = model.get_logprobs(data['states'], data['actions'])
        loss = ppo_loss(old_logp, new_logp, advantages)
        loss.backward()
```

🎯 面试要点

- **λ 控制偏差-方差折衷**: λ=1 无偏差高方差, λ=0 高偏差低方差
- **递推实现 O(T)**: A_t = δ_t + γλ·A_{t+1}
- **通常 γ=0.99, λ=0.95**
- **PPO 训练的标准优势估计方法**
- **GRPO 不需要 GAE** (用组内归一化代替)
- **面试常问**: GAE 的 λ 参数有什么作用?

LoRA

Low-Rank Adaptation

ΔW = BA，低秩分解高效微调

📌 原理与思想

核心概念

冻结预训练权重 W，用低秩矩阵 B·A 近似权重更新 ΔW。相比全量微调需要巨大显存，LoRA 只训练少量参数（约 0.1%），效果接近全量微调。A 用高斯初始化，B 用零初始化，保证初始输出不变。

核心思想

利用低秩分解近似权重更新：ΔW ≈ B·A，其中 A: [r, k]，B: [d, r]，r ≪ min(d, k)。B 初始化为零保证训练初期 LoRA 贡献为 0，通过 scaling = α/r 控制更新幅度。

算法步骤

1. 冻结原始权重：linear.requires_grad_(False)
2. 初始化 LoRA 矩阵：A 用 kaiming 初始化，B 用零初始化
3. 前向传播：output = linear(x) + (x @ A.T @ B.T) × scaling
4. 反向传播：只更新 A 和 B
5. 推理时合并：W_new = W + B·A·scaling（无额外开销）

📐 核心公式

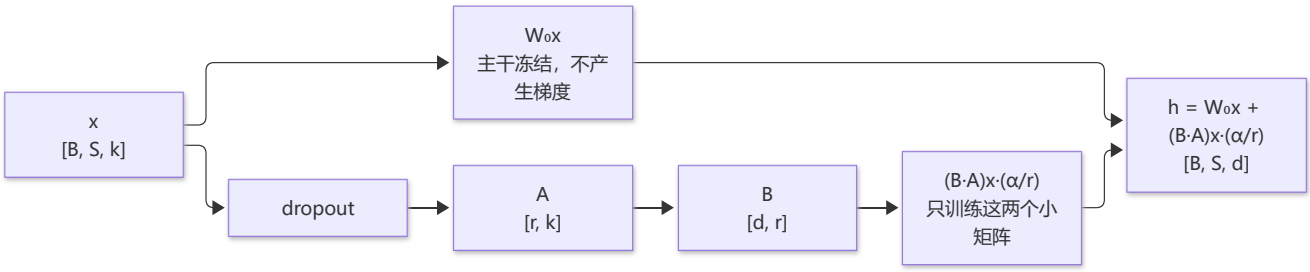
$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x$$

$$W_0 \in \mathbb{R}^{d \times k} \text{ (frozen)}, \qquad A \in \mathbb{R}^{r \times k}, \qquad B \in \mathbb{R}^{d \times r}, \qquad r \ll \min(d, k)$$

$$A \sim \text{kaiming}, \qquad B = \mathbf{0} \;\Rightarrow\; BA = 0 \text{ at init}$$

$$\frac{\text{trainable}}{\text{total}} = \frac{r(d+k)}{dk} \approx 0.1\%, \qquad W_{\text{new}} = W_0 + \frac{\alpha}{r} BA \;\text{ (mergeable at inference)}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn
import math

class LoRALinear(nn.Module):
    def __init__(self, in_f, out_f, rank=8, alpha=16.0, dropout=0.0):
        super().__init__()
        self.scaling = alpha / rank

        # 冻结原始权重
        self.linear = nn.Linear(in_f, out_f, bias=False)
        self.linear.requires_grad_(False)

        # LoRA 低秩矩阵
        self.lora_A = nn.Parameter(torch.zeros(rank, in_f))
        self.lora_B = nn.Parameter(torch.zeros(out_f, rank))
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B)          # B=0 → 初始 ΔW=0

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        """x: [B, S, in_f]"""
        base = self.linear(x)                          # 冻结路径
        lora = self.dropout(x) @ self.lora_A.T @ self.lora_B.T * self.scaling
        return base + lora

    def merge(self):
        """推理时合并权重（无额外开销）"""
        self.linear.weight.data += (self.lora_B @ self.lora_A) * self.scaling

# 在 Transformer 中应用 LoRA
def apply_lora(model, rank=8, alpha=16.0):
    """对模型中的 Q/V 投影应用 LoRA"""
    for name, module in model.named_modules():
        if isinstance(module, nn.Linear):
            if 'q_proj' in name or 'v_proj' in name:
                # 替换为 LoRA 版本
                lora_layer = LoRALinear(
                    module.in_features,
                    module.out_features,
```

```
                    rank=rank,
                    alpha=alpha
                )
                lora_layer.linear.weight.data.copy_(module.weight.data)
                # 替换原模块
                parts = name.rsplit('.', 1)
                parent = model.get_submodule(parts[0]) if len(parts) > 1 else model
                setattr(parent, parts[-1], lora_layer)

# 测试
if __name__ == "__main__":
    B, S, D = 2, 10, 512
    x = torch.randn(B, S, D)

    lora = LoRALinear(D, D, rank=8, alpha=16.0)
    out = lora(x)
    print(f"Input shape: {x.shape}")   # [2, 10, 512]
    print(f"Output shape: {out.shape}") # [2, 10, 512]

    # 验证初始输出等于原始权重输出
    base_out = lora.linear(x)
    diff = (out - base_out).abs().sum()
    print(f"Diff at init (should be 0): {diff.item():.6f}")
```

🎯 面试要点

- **B 初始化为零** → 初始时 LoRA 贡献为 0
- **scaling = α/r** 控制 LoRA 更新幅度
- **推理时可 merge**: W = W + BA·scaling → 无额外开销
- **通常只应用于 Q 和 V 的投影矩阵**
- **QLoRA = LoRA + 4bit 量化**，进一步降低显存
- **面试常问**: LoRA 的初始化策略？为什么 B 初始化为零?
- **参数量**: 可训练参数从 d·k 降到 r·(d+k)，而 r ≪ min(d, k)

💾 高效训练 Efficient

3 题

FP8 分块量化训练

Blockwise FP8 Training

按 128 一块算缩放因子，FP8 训练的关键全在块级缩放

📌 原理与思想

核心概念

训练用 FP8（前向的激活和权重用 e4m3，反向的梯度用 e5m2）能省显存带宽并提高吞吐，但整张量共用一个缩放因子会被少数离群值拖垮 —— 为了覆盖极值，绝大多数正常值被迫压到很低的精度。解决办法是分块：每 128 个元素（或 128×128 的块）单独算 `amax` 和缩放因子，块内共享一个 scale，矩阵乘法在 FP8 上做、累加仍在高精度。代价是需要 Hopper 及以后专门支持 FP8 的硬件，且块越小精度越好、缩放因子的元数据开销也越大。

核心思想

缩放因子不该整张量共用一个，而应该按块各算各的：每 128 个元素单独求 `amax` 定 scale，这样离群值的影响被限制在它自己那一块，其余块按自己的分布正常量化。矩阵乘法在 FP8 上做，但累加提到高精度 —— 精度损失只发生在「存」，不发生在「算」。

算法步骤

1. `scale = amax(block) / 448`，`448` 是 e4m3 的可表示上界，除完正好用满整个范围。
2. 量化 `x_q = clamp(x/scale, -448, 448).to(fp8)`，只保留 3 位尾数；反量化 `x̂ = x_q·scale`。
3. 缩放因子可以提到块外：`y = Σ_block s_x·s_w·(x_q·w_q)`，FP8 相乘、高精度累加，缩放因子不进乘法开销。
4. DeepSeek-V3 的配置：激活用 `1×128` 的块（per-token per-128-channel），权重用 `128×128` 的块。

📐 核心公式

$$s = \frac{\text{amax}(\text{block})}{448}, \qquad 448 = \text{e4m3 max}$$

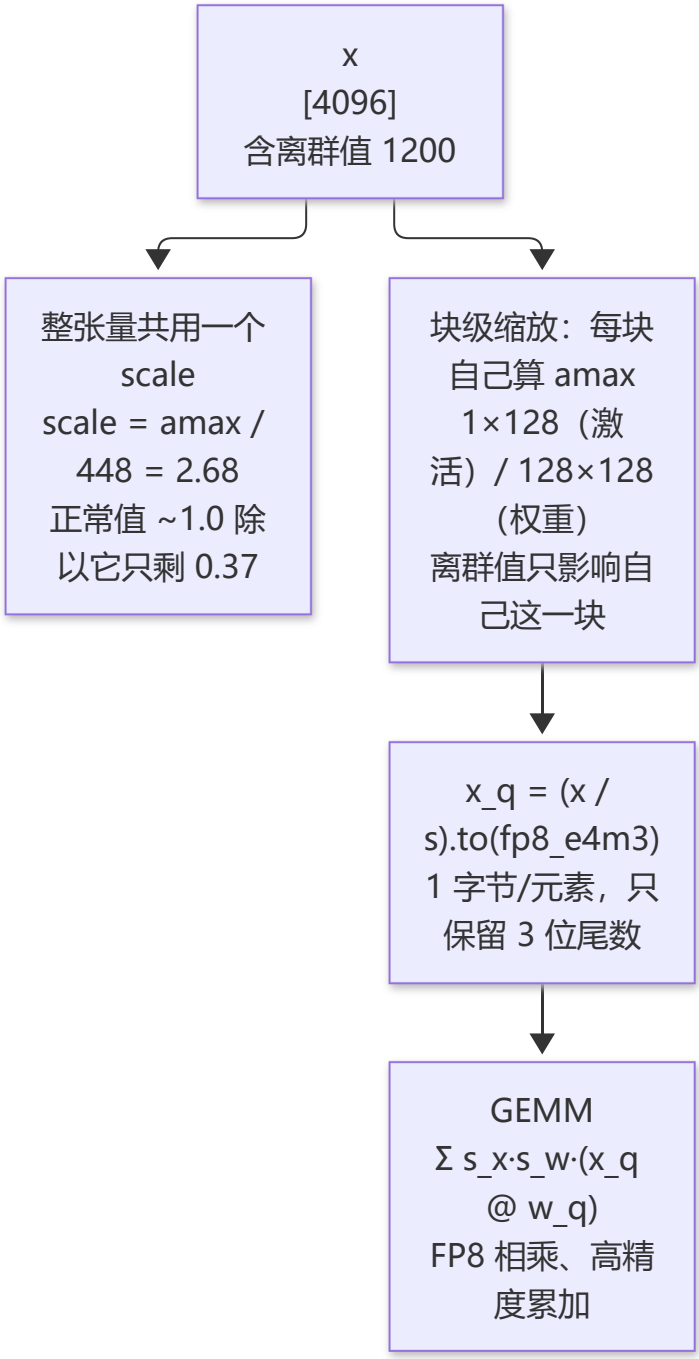
$$x_q = \text{clamp}\left(\frac{x}{s},\ -448,\ 448\right) \to \text{FP8}, \qquad \hat{x} = x_q\, s$$

$$y = \sum_{\text{block}} (x_q s_x)(w_q s_w) = \sum_{\text{block}} s_x s_w\, (x_q w_q)$$

$$\begin{aligned} &\text{e4m3}: \quad 1+4+3 \text{ bits}, \quad \max = 448 \qquad (\text{activations, weights}) \\ &\text{e5m2}: \quad 1+5+2 \text{ bits}, \quad \max = 57344 \qquad (\text{gradients}) \end{aligned}$$

$$\text{DeepSeek-V3}: \quad \text{activations } 1\times 128, \qquad \text{weights } 128\times 128, \qquad \text{accumulate in FP32}$$

📊 张量流程图



💻 代码实现

```
import torch

FP8_MAX = {"e4m3": 448.0, "e5m2": 57344.0}
FP8_DTYPE = {"e4m3": torch.float8_e4m3fn, "e5m2": torch.float8_e5m2}


def per_block_quant(x, block_size=128, fmt="e4m3"):
    """
    按最后一维分块做 FP8 量化
```

```
    x: [..., N]：block_size: 每块元素数
    返回: (量化再反量化后的值, 每块的缩放因子)
    注意: 需要 torch >= 2.1 才有 float8_e4m3fn
    """
    assert x.shape[-1] % block_size == 0, "最后一维要能被 block_size 整除"
    orig_shape = x.shape
    n_blocks = orig_shape[-1] // block_size
    xb = x.reshape(-1, n_blocks, block_size)                    # [M, n_blocks, B]

    # 1. 每块一个 amax → 一个缩放因子（clamp_min 防止除零）
    amax = xb.abs().amax(dim=-1, keepdim=True).clamp_min(1e-12)   # [M, n_blocks, 1]
    scale = amax / FP8_MAX[fmt]

    # 2. 量化到 FP8 再反量化（模拟「FP8 存、高精度用」）
    x_q = (xb / scale).clamp(-FP8_MAX[fmt], FP8_MAX[fmt]).to(FP8_DTYPE[fmt])
    x_hat = x_q.to(x.dtype) * scale

    return x_hat.reshape(orig_shape), scale.reshape(-1, n_blocks)


def fp8_gemm_sim(x, w, block_size=128):
    """
    模拟 FP8 块级矩阵乘法: FP8 乘 + 高精度累加
    x: [M, K]   w: [K, N]
    """
    # 激活按 1×128 的块量化（沿 K 维切块）
    x_hat, _ = per_block_quant(x, block_size, fmt="e4m3")
    # 权重按 128×128 的块量化
    w_hat, _ = per_block_quant(w.t().contiguous(), block_size, fmt="e4m3")

    # 真实实现是 FP8 张量核 + 高精度累加；这里用反量化后的值算等价的数学结果
    return x_hat @ w_hat.t().contiguous()


if __name__ == "__main__":
    torch.manual_seed(0)
    x = torch.randn(256, 512)
    x[0, 0] = 1200.0                        # 人为插一个离群值

    # 对比: 整张量一个 scale  vs  分块 scale
    naive_scale = x.abs().amax() / FP8_MAX["e4m3"]
    x_naive = (x / naive_scale).clamp(-448, 448).to(torch.float8_e4m3fn)
    x_naive = x_naive.to(x.dtype) * naive_scale
    x_block, _ = per_block_quant(x, 128, "e4m3")

    rel = lambda a, b: ((a - b).norm() / b.norm()).item()
    print("朴素量化相对误差:", round(rel(x_naive, x), 5))
    print("分块量化相对误差:", round(rel(x_block, x), 5))

    w = torch.randn(512, 128)
    print("FP8 GEMM 输出形状:", tuple(fp8_gemm_sim(x, w).shape))
```

🎯 面试要点

- **核心一句话**: FP8 训练的成败在缩放粒度，块级缩放把离群值的影响关在块内

- **为什么必须分块**: FP8 的动态范围很窄（e4m3 只有 3 位尾数、上界 448），而激活分布是重尾的；整张量共享 scale 时，为了装下离群值，正常值全跌到只剩几档可表示
- **e4m3 与 e5m2 的分工**: 前向用 e4m3（要精度），反向梯度用 e5m2（要动态范围）
- **累加精度不能省**: FP8 只用来做乘，累加必须回到高精度，否则误差快速累积
- **块大小的权衡**: 块越小，离群值影响越小但缩放因子的存储和计算开销越大；`1×128`（激活）和 `128×128`（权重）是常见折中
- **必须让缩放因子以可融合的方式参与**: 真实实现把 scale 融进张量核的 epilogue，避免额外的反量化访存
- **和 BF16 混合精度的区别**: BF16 混合精度靠「主权重 FP32 + 计算 BF16」保精度；FP8 靠「块级缩放 + 高精度累加」保精度，省的是带宽
- **面试常问**: 为什么 FP8 一定要分块？为什么累加不能用 FP8？e4m3 和 e5m2 各自用在哪一侧？
- **省的是什么**: 块级缩放保精度，相比 FP16 显存与带宽再减半，而累加精度不变

梯度检查点

Gradient Checkpointing

时间换空间，重新计算代替存储激活

📌 原理与思想

核心概念

不保存所有中间激活值，只保存检查点。反向传播时重新计算需要的激活值。相比标准训练保存所有层激活（显存 O(L)），梯度检查点只保存 √L 个检查点（显存 O(√L)），用约 20% 额外计算换取大量显存。

核心思想

时间换空间：不保存所有中间激活值，只保存关键检查点。反向传播时从最近的检查点重新计算需要的激活值，大幅减少显存占用。

算法步骤

1. 前向传播时只保存检查点（如每 √L 层保存一次）
2. 反向传播时从检查点重新计算需要的激活值
3. 使用 torch.utils.checkpoint 包裹需要检查点的层
4. HuggingFace 模型可直接设置 gradient_checkpointing=True

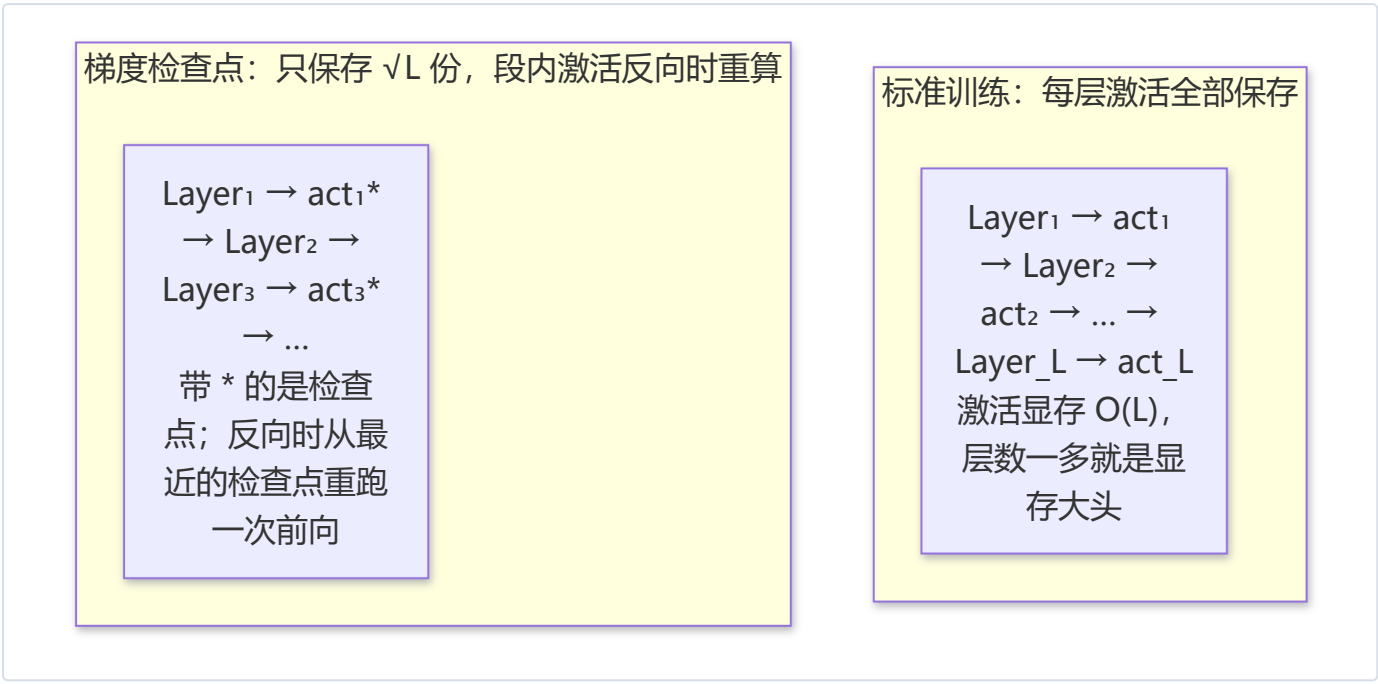
📐 核心公式

$$\text{activations} : O(L) \longrightarrow O\left(\sqrt{L}\right)$$

$$\text{memory} : L\,B\,S\,d \cdot 4\,\text{B} \longrightarrow \sqrt{L}\,B\,S\,d \cdot 4\,\text{B}$$

$$\text{checkpoints} = \sqrt{L}, \qquad \text{time overhead} \approx 20\%$$

张量流程图



代码实现

```python
import torch
from torch.utils.checkpoint import checkpoint

# 方法1: 使用 torch.utils.checkpoint
def custom_forward(x, weight1, weight2):
    return torch.relu(x @ weight1) @ weight2

# 不保存中间激活，反向时重新计算
output = checkpoint(custom_forward, x, w1, w2, use_reentrant=False)

# 方法2: HuggingFace 模型直接开启
from transformers import AutoModel

model = AutoModel.from_pretrained(
    "model_name",
    gradient_checkpointing=True    # 一行开启
)

# 方法3: 手动实现
class CheckpointBlock(nn.Module):
    def __init__(self, layer):
        super().__init__()
        self.layer = layer

    def forward(self, x):
        return checkpoint(self.layer, x, use_reentrant=False)

# 在 Transformer 中使用
class TransformerBlock(nn.Module):
```

```python
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ffn = FFN(d_model)
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)

    def forward(self, x, mask=None):
        # 使用 checkpoint 包裹
        x = x + checkpoint(self.attn, self.norm1(x), mask, use_reentrant=False)
        x = x + checkpoint(self.ffn, self.norm2(x), use_reentrant=False)
        return x

# 测试显存节省
def measure_memory(use_checkpoint=False):
    """测量显存使用"""
    import torch.cuda

    model = build_model(use_checkpoint=use_checkpoint)
    x = torch.randn(4, 2048, 4096, device='cuda')

    torch.cuda.reset_peak_memory_stats()
    output = model(x)
    loss = output.sum()
    loss.backward()

    peak_memory = torch.cuda.max_memory_allocated() / 1e9
    print(f"Peak memory: {peak_memory:.2f} GB")
    return peak_memory

# 对比
print("Without checkpoint:")
mem1 = measure_memory(use_checkpoint=False)

print("\nWith checkpoint:")
mem2 = measure_memory(use_checkpoint=True)

print(f"\nMemory saved: {(mem1 - mem2):.2f} GB ({(1 - mem2/mem1)*100:.1f}%)")
```

🎯 面试要点

- **用 ~20% 额外计算时间换取大量显存**
- **显存从 O(L) 降到 O(√L)**
- **HuggingFace 模型一行开启**: `gradient_checkpointing=True`
- **常与 LoRA 配合**: LoRA 减少参数 + 检查点减少激活
- **use_reentrant=False** 是新版推荐设置
- **面试常问**: 梯度检查点的原理？代价是什么？
- 省下来的显存可以直接换成更大的 batch 或更长的序列

混合精度训练

Mixed Precision Training

FP16/BF16 计算 + FP32 主权重

📌 原理与思想

核心概念

使用前向/反向用 FP16/BF16 减少显存和加速计算，但保持 FP32 主权重防止精度损失。FP16/BF16 只需 2 bytes（显存减半、计算加速），FP32 需 4 bytes（精度高），混合使用实现既快又准。配合 loss scaling 防止梯度下溢。

核心思想

前向和反向传播使用低精度（FP16/BF16）加速计算并减少显存，但主权重始终保持 FP32 防止精度损失。BF16 指数位更多（8 vs 5），不需要 loss scaling。

算法步骤

1. FP32 主权重 → copy → FP16 权重
2. FP16 前向传播 → FP16 loss
3. Loss scaling：loss × loss_scale
4. FP16 反向传播 → FP16 梯度
5. 梯度转 FP32 → 更新 FP32 主权重

📐 核心公式

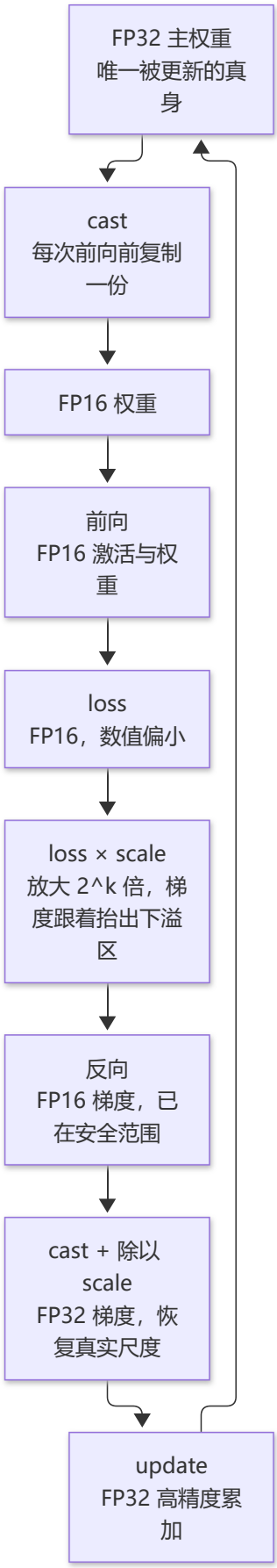
$$\text{value} = (-1)^s \cdot 2^{e-\text{bias}} \cdot (1.m)$$

$$\begin{aligned} \text{FP32}: &\quad 1+8+23 \text{ bits} \\ \text{FP16}: &\quad 1+5+10 \text{ bits} \\ \text{BF16}: &\quad 1+8+7 \text{ bits} \end{aligned}$$

$$\begin{aligned} \text{forward / backward}: &\quad \text{FP16 (2 B)} \\ \text{master weights}: &\quad \text{FP32 (4 B)} \\ \text{gradient}: &\quad g_{\text{FP16}} \longrightarrow \text{FP32} \end{aligned}$$

$$\mathcal{L}' \leftarrow S \cdot \mathcal{L}, \qquad g \leftarrow \frac{\text{cast}(g_{\text{FP16}})}{S}, \qquad \text{BF16}: \; e_{\text{bits}} = 8 \;\Rightarrow\; S = 1$$

📊 张量流程图


FP32 主权重
唯一被更新的真身
cast
每次前向前复制一份
FP16 权重
前向
FP16 激活与权重
loss
FP16，数值偏小
loss × scale
放大 2^k 倍，梯度跟着抬出下溢区
反向
FP16 梯度，已在安全范围
cast + 除以 scale
FP32 梯度，恢复真实尺度
update
FP32 高精度累加

💻 代码实现

```python
import torch

# 方法1: PyTorch 原生 AMP (Automatic Mixed Precision)
scaler = torch.amp.GradScaler('cuda')

for batch in dataloader:
    optimizer.zero_grad()

    # 自动将操作转为 FP16
    with torch.amp.autocast('cuda'):
        loss = model(batch)

    # Loss scaling 防止梯度下溢
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

# 方法2: BF16 更简单 (不需要 loss scaling)
with torch.amp.autocast('cuda', dtype=torch.bfloat16):
    loss = model(batch)
loss.backward()   # 直接反向，不需要 scaler
optimizer.step()

# 方法3: HuggingFace Trainer 自动处理
from transformers import Trainer, TrainingArguments

args = TrainingArguments(
    output_dir='./output',
    fp16=True,           # 使用 FP16
    # bf16=True,         # 或使用 BF16 (A100/H100 推荐)
    fp16_full_eval=True, # 评估也用 FP16
)
trainer = Trainer(model=model, args=args)
trainer.train()

# 验证混合精度
def check_mixed_precision(model):
    """检查模型是否使用混合精度"""
    for name, param in model.named_parameters():
        print(f"{name}: {param.dtype}")
        # 应该看到大部分参数是 fp16/bf16
        # 但某些关键参数（如 LayerNorm）保持 fp32
```

🎯 面试要点

- **FP16**: 1+5+10 bits, 需要 loss scaling 防梯度下溢
- **BF16**: 1+8+7 bits, 指数位多 → 不需要 loss scaling
- **显存减半 + 计算加速** (Tensor Core)
- **现代 GPU (A100/H100) 推荐 BF16**
- **主权重始终 FP32** 保证精度

- **面试常问**: FP16 和 BF16 的区别？为什么 BF16 不需要 loss scaling?

推理加速 Inference

2 题

分页注意力

PagedAttention

vLLM 核心，分页管理 KV Cache

📌 原理与思想

核心概念

借鉴操作系统虚拟内存的分页思想，将 KV Cache 分成固定大小的块（页），通过块表映射到非连续物理内存。相比传统 KV Cache 预分配 max_len 导致大量浪费和内存碎片，PagedAttention 按需分配，内存利用率可达 ~96%。

核心思想

逻辑块是连续的 token 块（如 16 tokens/block），物理块是 GPU 显存中的实际存储位置。通过块表（block table）将逻辑块映射到非连续的物理块，消除预分配浪费和内存碎片。

算法步骤

1. 预分配物理块池：physical_blocks = zeros(num_blocks, ...)
2. 为序列分配物理块：allocate(seq_id, num_tokens)
3. 写入 KV 到对应物理块：write(seq_id, position, key, value)
4. 注意力计算时从物理块 gather K/V
5. 序列结束时释放物理块：free(seq_id)

📐 核心公式

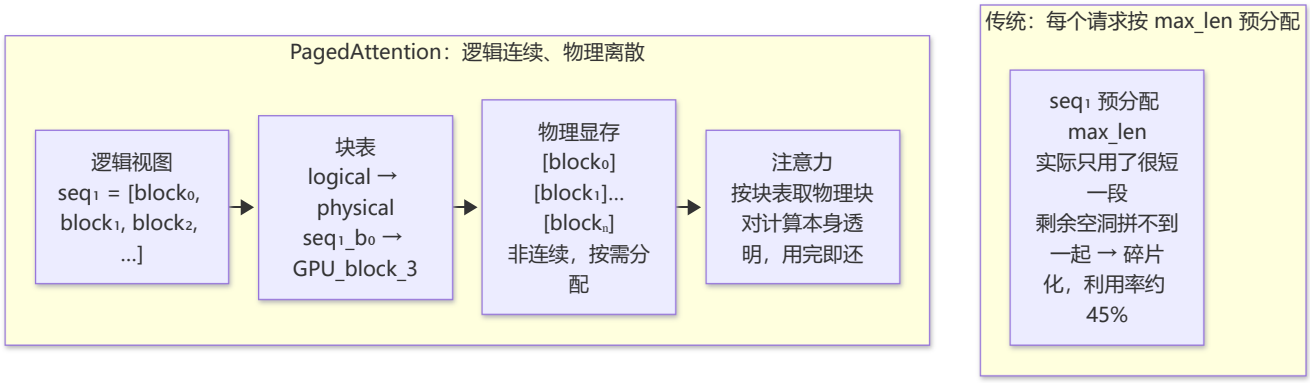
$$\text{phys}(s, j) = \text{block_table}[s][j]$$

$$\text{utilization} = \frac{\sum_i \ell_i}{N_{\text{blocks}} \cdot B}, \qquad B = 16 \text{ tokens/block}$$

$$\text{naive} : \text{batch} \times S_{\max} \text{ preallocated}, \qquad \text{paged} : \sum_i \ell_i \text{ allocated on demand}$$

$$\text{utilization} : 45\% \longrightarrow 96\%$$

📊 张量流程图



💻 代码实现

```
import torch

# PagedAttention 核心思想（简化伪代码）
class PagedKVCache:
    def __init__(self, block_size=16, num_blocks=1000, n_layers=32,
                 n_heads=32, head_dim=128):
        self.block_size = block_size
        # 物理块池（预分配所有显存）
        self.physical_blocks = torch.zeros(
            num_blocks, n_layers, 2, block_size, n_heads, head_dim,
            dtype=torch.float16, device='cuda'
        )
        # 块表：每个序列的逻辑块 → 物理块映射
        self.block_tables = {}   # seq_id → [physical_block_ids]
        self.free_blocks = list(range(num_blocks))

    def allocate(self, seq_id, num_tokens):
        """为序列分配物理块"""
        num_blocks = (num_tokens + self.block_size - 1) // self.block_size
        if num_blocks > len(self.free_blocks):
            raise RuntimeError("Out of memory")

        allocated = self.free_blocks[:num_blocks]
        self.free_blocks = self.free_blocks[num_blocks:]
        self.block_tables[seq_id] = allocated

    def write(self, seq_id, position, key, value):
        """写入 KV 到对应的物理块"""
        block_idx = position // self.block_size
        block_offset = position % self.block_size
        phys_id = self.block_tables[seq_id][block_idx]
        self.physical_blocks[phys_id, :, :, block_offset] = (key, value)

    def attention(self, seq_id, q):
        """使用块表进行注意力计算"""
        phys_ids = self.block_tables[seq_id]
        # 从非连续物理块中 gather K/V → 计算注意力
        k = self.physical_blocks[phys_ids, :, 0]   # gather
```

```
        v = self.physical_blocks[phys_ids, :, 1]
        return paged_attention_kernel(q, k, v)

    def free(self, seq_id):
        """释放序列的块"""
        if seq_id in self.block_tables:
            self.free_blocks.extend(self.block_tables[seq_id])
            del self.block_tables[seq_id]

# PagedAttention kernel (简化版)
def paged_attention_kernel(q, k_blocks, v_blocks, block_size=16):
    """
    q: [B, H, 1, D]
    k_blocks: [num_blocks, n_layers, block_size, H, D]
    v_blocks: [num_blocks, n_layers, block_size, H, D]
    """
    # 实际实现需要 CUDA kernel
    # 这里展示核心思想: 从非连续块中 gather K/V

    B, H, _, D = q.shape
    num_blocks = k_blocks.shape[0]

    # Flatten blocks
    k = k_blocks.view(-1, H, D)  # [num_blocks * block_size, H, D]
    v = v_blocks.view(-1, H, D)

    # 标准注意力
    scores = torch.matmul(q, k.transpose(-2, -1))
    attn = torch.softmax(scores, dim=-1)
    output = torch.matmul(attn, v)

    return output

# 使用示例
if __name__ == "__main__":
    cache = PagedKVCache(block_size=16, num_blocks=100)

    # 序列 1: 32 tokens
    cache.allocate(seq_id=1, num_tokens=32)

    # 序列 2: 20 tokens
    cache.allocate(seq_id=2, num_tokens=20)

    print(f"Allocated blocks for seq 1: {cache.block_tables[1]}")
    print(f"Allocated blocks for seq 2: {cache.block_tables[2]}")
    print(f"Free blocks: {len(cache.free_blocks)}")
```

🎯 面试要点

- **借鉴 OS 虚拟内存分页思想**
- **逻辑块连续，物理块可以不连续**
- **消除预分配浪费和内存碎片**
- **内存利用率从 ~45% 提升到 ~96%**
- **vLLM 的核心创新**，大幅提升推理吞吐

- **支持 copy-on-write** 实现 beam search 的 KV 共享
- **面试常问**: PagedAttention 的原理？如何提高内存利用率?

投机解码

Speculative Decoding

小模型草稿，大模型验证，加速 2-3x

📌 原理与思想

核心概念

用一个小模型（draft model）快速生成多个候选 token，然后用大模型一次性验证。相比标准自回归每步只生成 1 token 很慢，投机解码一次验证多个 token，可加速 2-3x，且输出分布与只用大模型完全一致（无损）。

核心思想

小模型快速生成 γ 个候选 token，大模型一次性前向验证所有位置。通过接受/拒绝机制保证输出分布不变：accept if r < p_target(x_i) / p_draft(x_i)。

算法步骤

1. Draft model 生成 γ 个候选 token
2. Target model 一次性前向，得到所有位置的概率
3. 逐 token 接受/拒绝：比较 p_target 和 p_draft
4. 拒绝时从修正分布重新采样
5. 全部接受时额外采样一个 bonus token

📐 核心公式

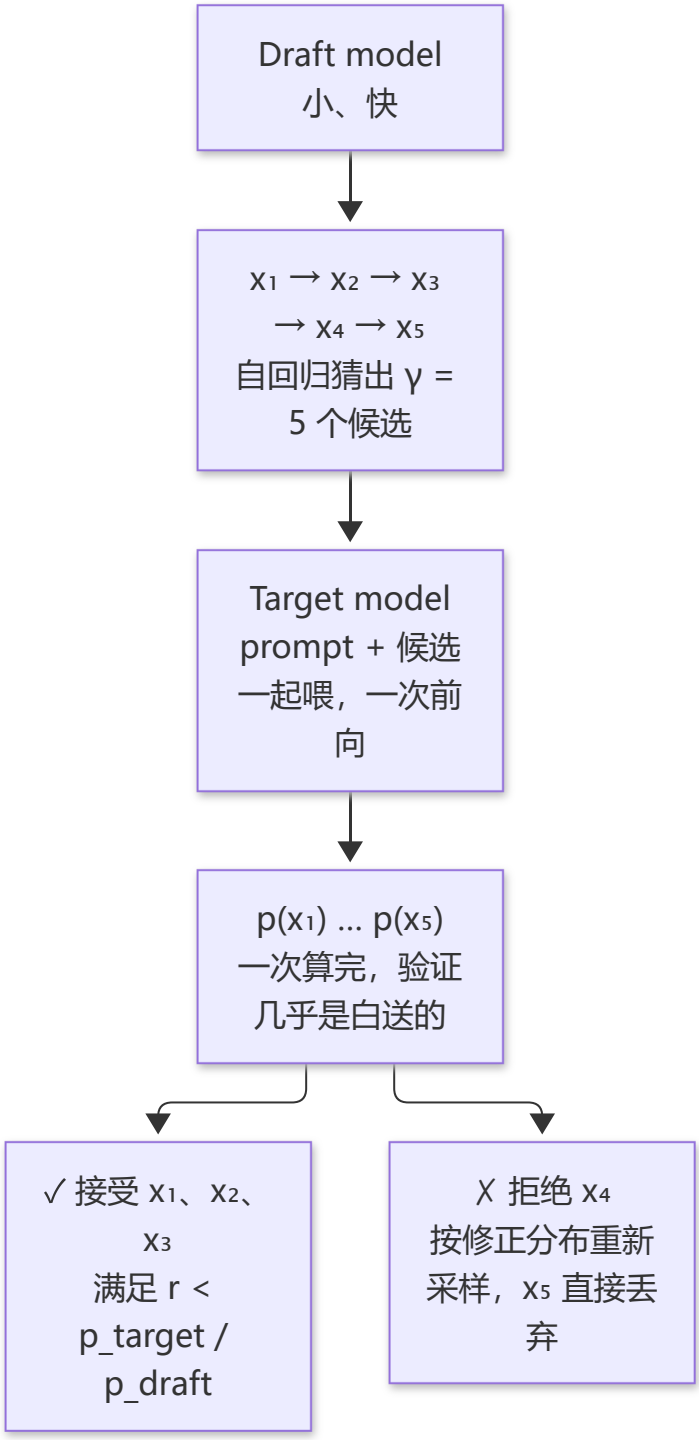
$$\text{accept } x_i \text{ iff } r < \frac{p_{\text{target}}(x_i)}{p_{\text{draft}}(x_i)}, \qquad r \sim U[0,1]$$

$$\mathbb{E}\left[\#\text{accepted}\right] = \sum_{t=1}^{\gamma} \prod_{i=1}^{t} \alpha_i, \qquad \alpha_i = \min\left(1, \frac{p_{\text{target}}(x_i)}{p_{\text{draft}}(x_i)}\right)$$

$$\gamma = 5, \qquad \mathbb{E}\left[\#\text{accepted}\right] \sim 3 \text{ to } 4, \qquad \text{speedup} \sim 2 \text{ to } 3\times$$

$$\text{standard} : 1 \text{ forward} \to 1 \text{ token}, \qquad \text{speculative} : 1 \text{ forward} \to 3 \text{ to } 4 \text{ tokens}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn.functional as F

@torch.no_grad()
def speculative_decode(draft_model, target_model, prompt_ids,
                       gamma=5, temperature=1.0, max_len=100):
    """
```

```
    投机解码

    draft_model: 小模型 (快速生成候选)
    target_model: 大模型 (验证候选)
    gamma: 每轮草稿长度
    """
    generated = prompt_ids.tolist()

    while len(generated) < max_len:
        # Step 1: Draft model 生成 γ 个候选
        draft_tokens = []
        draft_probs = []
        input_ids = torch.tensor([generated])

        for _ in range(gamma):
            logits = draft_model(input_ids).logits[:, -1]
            probs = F.softmax(logits / temperature, dim=-1)
            next_token = torch.multinomial(probs, 1)
            draft_tokens.append(next_token.item())
            draft_probs.append(probs)
            input_ids = torch.cat([input_ids, next_token.unsqueeze(0)], dim=1)

        # Step 2: Target model 并行验证所有候选
        full_ids = torch.tensor([generated + draft_tokens])
        target_logits = target_model(full_ids).logits
        target_probs = F.softmax(
            target_logits[0, len(generated)-1:-1] / temperature, dim=-1)

        # Step 3: 逐 token 接受/拒绝
        accepted = 0
        for i, token in enumerate(draft_tokens):
            p_target = target_probs[i, token].item()
            p_draft = draft_probs[i][0, token].item()
            r = torch.rand(1).item()

            if r < p_target / p_draft:
                # 接受
                accepted += 1
                generated.append(token)
            else:
                # 拒绝: 从修正分布重新采样
                corrected = torch.clamp(target_probs[i] - draft_probs[i][0], min=0)
                corrected /= corrected.sum()
                generated.append(torch.multinomial(corrected, 1).item())
                break
        else:
            # 全部接受: 再采样一个 bonus token
            bonus = torch.multinomial(target_probs[-1], 1).item()
            generated.append(bonus)

    return generated

# 测试
if __name__ == "__main__":
    # 模拟 draft 和 target 模型
    class SimpleModel(torch.nn.Module):
        def __init__(self, vocab_size=1000, d_model=256):
            super().__init__()
            self.emb = torch.nn.Embedding(vocab_size, d_model)
```

```
            self.head = torch.nn.Linear(d_model, vocab_size)

        def forward(self, x):
            h = self.emb(x)
            logits = self.head(h)
            return type('obj', (object,), {'logits': logits})

    draft = SimpleModel()
    target = SimpleModel()

    prompt = [1, 2, 3]
    output = speculative_decode(draft, target, prompt, gamma=5, max_len=20)
    print(f"Generated: {output}")
```

🎯 面试要点

- **核心: 小模型草稿 + 大模型并行验证**
- **输出分布与只用大模型完全一致**（无损!）
- **加速比取决于 draft model 的接受率**
- **一次大模型前向可获得多个 token**
- **典型加速: 2-3x** (取决于任务和 draft model 质量)
- **面试常问**: 投机解码的原理？为什么无损?

🎲 采样策略 Sampling

1 题

采样策略

Sampling Strategies

控制输出多样性：Temperature / Top-k / Top-p

📌 原理与思想

核心概念

Temperature 控制分布锐度；Top-k 只保留概率最高的 K 个 token；Top-p 保留累积概率达到 P 的最小集合。相比 Greedy（确定性但容易重复）和纯 Sampling（多样性但可能低质量），采样策略在质量和多样性之间取得平衡。实践中常组合使用。

核心思想

Temperature 通过除以温度参数 T 调整分布锐度（T<1 更确定，T>1 更随机）。Top-k 固定保留 K 个候选，Top-p 动态选择累积概率达到 P 的最小集合，后者更灵活。

算法步骤

1. 应用 Temperature：logits = logits / T
2. Top-k 过滤：只保留最大的 k 个，其余设为 -inf
3. Top-p 过滤：排序后累积概率超过 p 的设为 -inf
4. Softmax 归一化：probs = softmax(logits)
5. 多项式采样：next_token = multinomial(probs)

📐 核心公式

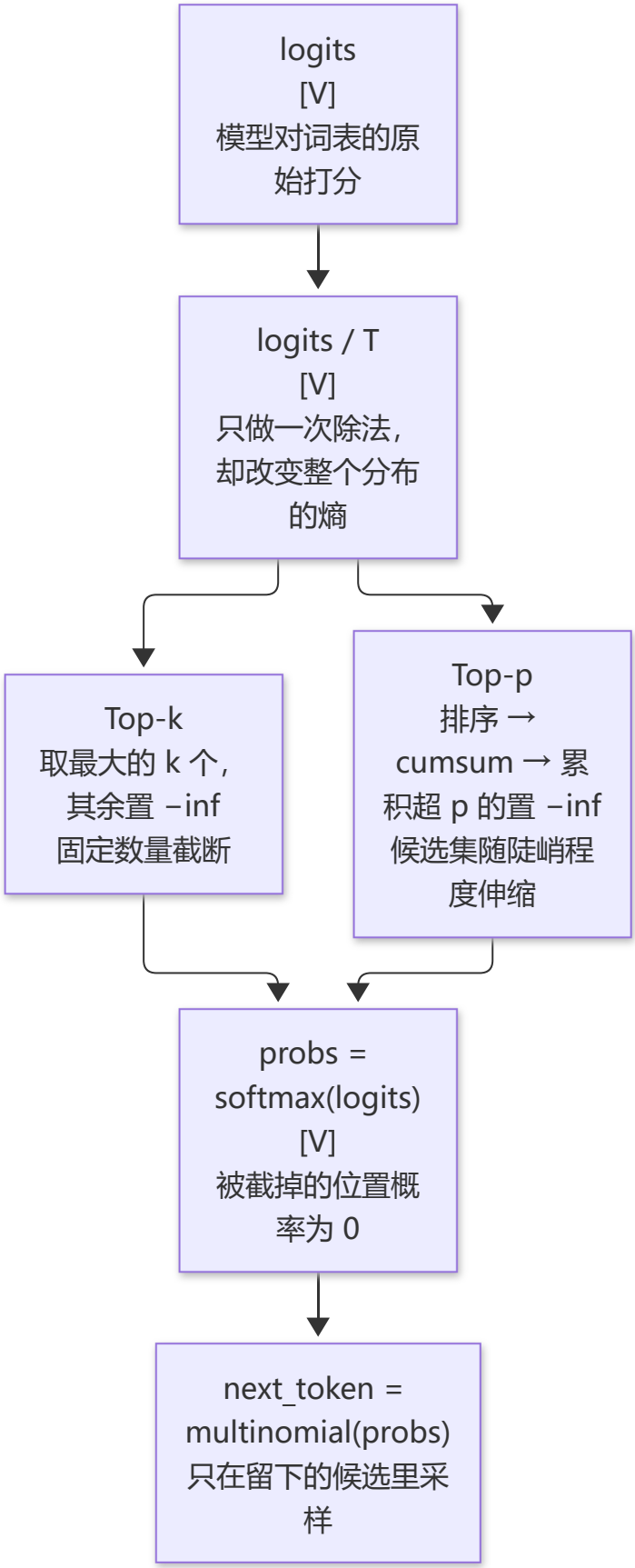
$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

$$T < 1:\ \text{sharper}, \qquad T > 1:\ \text{flatter}, \qquad T \to 0:\ \text{greedy}$$

$$\text{top-}k: \quad \mathcal{S}_k = \{i:\ \text{rank}(z_i) \le k\}, \qquad p_i \leftarrow 0 \text{ for } i \notin \mathcal{S}_k$$

$$\text{top-}p: \quad \mathcal{S}_p = \min\left\{V' \subseteq V:\ \sum_{i \in V'} p_i \ge p\right\}$$

📊 张量流程图


logits
[V]
模型对词表的原始打分
logits / T
[V]
只做一次除法，却改变整个分布的熵
Top-k
取最大的 k 个，其余置 −inf
固定数量截断
Top-p
排序 → cumsum → 累积超 p 的置 −inf
候选集随陡峭程度伸缩
probs = softmax(logits)
[V]
被截掉的位置概率为 0
next_token = multinomial(probs)
只在留下的候选里采样

💻 代码实现

```python
import torch
import torch.nn.functional as F

def top_k_top_p_sampling(logits, temperature=0.7, top_k=50, top_p=0.9):
    """组合采样策略"""
    logits = logits / temperature

    # Top-k: 只保留最大的 k 个
    if top_k > 0:
        top_k_vals, _ = torch.topk(logits, top_k)
        threshold = top_k_vals[..., -1:]
        logits = logits.masked_fill(logits < threshold, float('-inf'))

    # Top-p: 累积概率截断
    if top_p < 1.0:
        sorted_logits, sorted_idx = torch.sort(logits, descending=True)
        cum_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
        mask = cum_probs > top_p
        mask[..., 1:] = mask[..., :-1].clone()   # 至少保留一个
        mask[..., 0] = False
        sorted_logits[mask] = float('-inf')
        logits = sorted_logits.scatter(1, sorted_idx, sorted_logits)

    probs = F.softmax(logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)

# 单独的采样函数
def temperature_sampling(logits, temperature=1.0):
    """温度采样"""
    return logits / temperature

def top_k_sampling(logits, top_k=50, temperature=1.0):
    """Top-K 采样"""
    logits = logits / temperature
    top_k_vals, _ = torch.topk(logits, top_k)
    threshold = top_k_vals[..., -1:]
    return logits.masked_fill(logits < threshold, float('-inf'))

def top_p_sampling(logits, top_p=0.9, temperature=1.0):
    """Top-P (Nucleus) 采样"""
    logits = logits / temperature
    sorted_logits, sorted_idx = torch.sort(logits, descending=True)
    cum_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)

    mask = cum_probs > top_p
    mask[..., 1:] = mask[..., :-1].clone()
    mask[..., 0] = False
    sorted_logits[mask] = float('-inf')

    return sorted_logits.scatter(1, sorted_idx, sorted_logits)

# 自回归生成
@torch.no_grad()
def generate(model, prompt_ids, max_new_tokens=100, temperature=0.7,
             top_k=50, top_p=0.9):
```

```
    """使用采样策略生成文本"""
    input_ids = prompt_ids.clone()

    for _ in range(max_new_tokens):
        logits = model(input_ids).logits[:, -1, :]

        # 应用采样策略
        logits = top_k_top_p_sampling(logits, temperature, top_k, top_p)

        # 采样下一个 token
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)

        input_ids = torch.cat([input_ids, next_token], dim=1)

        # 遇到 EOS 停止
        if next_token.item() == model.config.eos_token_id:
            break

    return input_ids

# 测试
if __name__ == "__main__":
    V = 32000
    logits = torch.randn(1, V)

    # 不同采样策略
    print("Temperature=0.1 (更确定):")
    sampled = top_k_top_p_sampling(logits, temperature=0.1, top_k=50, top_p=0.9)
    print(f"  Non-inf tokens: {(sampled > float('-inf')).sum().item()}")

    print("\nTemperature=1.0 (标准):")
    sampled = top_k_top_p_sampling(logits, temperature=1.0, top_k=50, top_p=0.9)
    print(f"  Non-inf tokens: {(sampled > float('-inf')).sum().item()}")

    print("\nTemperature=2.0 (更随机):")
    sampled = top_k_top_p_sampling(logits, temperature=2.0, top_k=50, top_p=0.9)
    print(f"  Non-inf tokens: {(sampled > float('-inf')).sum().item()}")
```

🎯 面试要点

- **T<1**: 更确定 (适合翻译/代码); **T>1**: 更随机 (适合创作)
- **Top-k 固定数量**，Top-p 动态集合 → Top-p 更灵活
- **实践中通常组合**: 先 Top-k 再 Top-p
- **Greedy (T→0)** 可复现，sampling 不可复现
- **面试常问**: Temperature 的作用？Top-k 和 Top-p 的区别？

Decoder-Only Transformer

GPT 架构

因果注意力 + 自回归，现代 LLM 标配

📌 原理与思想

核心概念

只有 decoder 的 Transformer 架构。使用因果注意力（只能看到之前的 token），通过自回归方式逐 token 生成。相比 Encoder-Decoder 架构，Decoder-Only 统一训练和推理（训练时预测下一个 token，推理时生成），因果注意力保证自回归特性，适合生成任务。GPT/LLaMA/Mistral 等主流 LLM 都采用此架构。

核心思想

每层包含自注意力（带 causal mask）和 FFN，使用 Pre-Norm 结构（先归一化再进子层）。现代 LLM 标配：RMSNorm + SwiGLU + RoPE。

算法步骤

1. Token Embedding + Position Embedding (RoPE)
2. 创建因果掩码：mask = tril(ones(S, S))
3. 逐层处理：x = x + Attn(LN(x), mask); x = x + FFN(LN(x))
4. 最终归一化：x = RMSNorm(x)
5. LM Head：logits = x @ W_vocab
6. 训练：Shift + CrossEntropy；推理：Sample next token

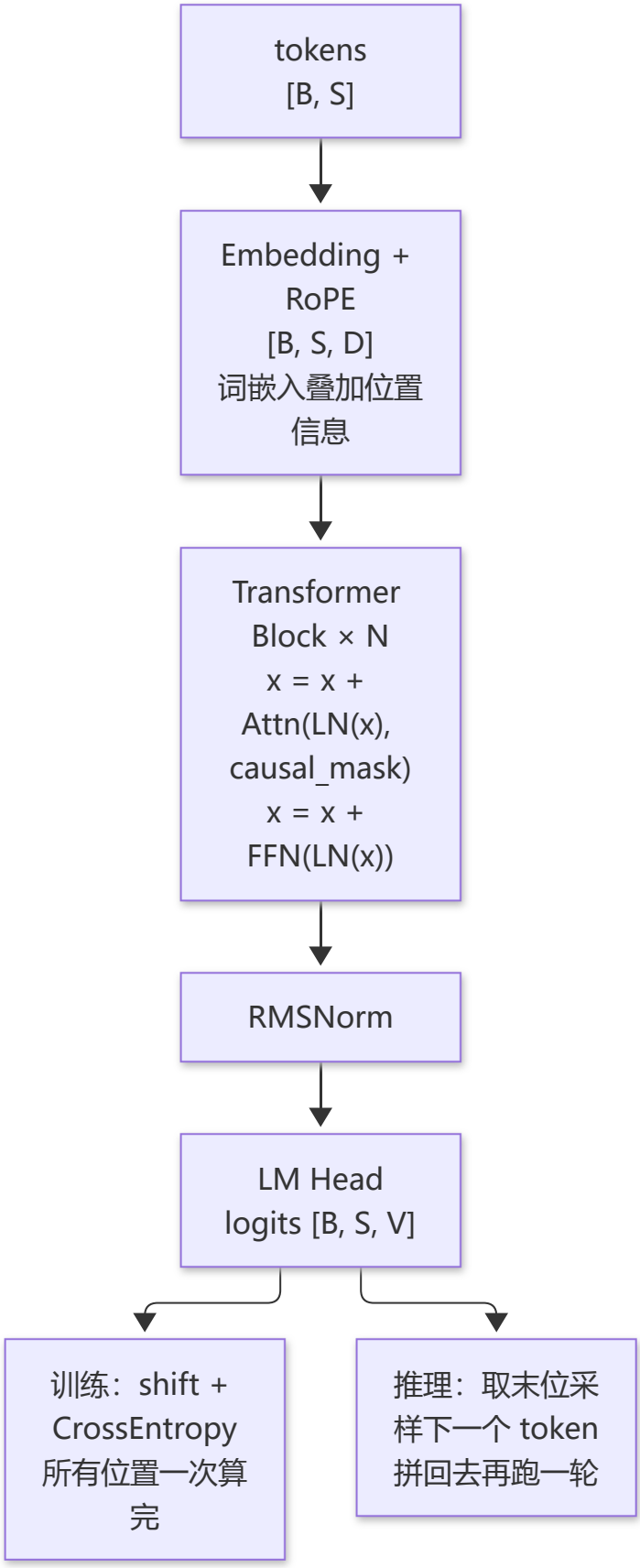
📐 核心公式

$$x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x),\ \text{causal mask}), \qquad x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x))$$

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t})$$

$$\{\mathrm{RMSNorm},\ \mathrm{SwiGLU},\ \mathrm{RoPE},\ \text{Pre-Norm}\}$$

📊 张量流程图


tokens
[B, S]
Embedding + RoPE
[B, S, D]
词嵌入叠加位置信息
Transformer Block × N
x = x + Attn(LN(x), causal_mask)
x = x + FFN(LN(x))
RMSNorm
LM Head
logits [B, S, V]
训练：shift + CrossEntropy 所有位置一次算完
推理：取末位采样下一个 token 拼回去再跑一轮

💻 代码实现

```
import torch
import torch.nn as nn

class DecoderBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff):
        super().__init__()
        self.attn_norm = RMSNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ffn_norm = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, d_ff)

    def forward(self, x, mask):
        # Pre-Norm: 先归一化再进子层
        x = x + self.attn(self.attn_norm(x), mask=mask)
        x = x + self.ffn(self.ffn_norm(x))
        return x

class GPTModel(nn.Module):
    def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, max_len=2048):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.blocks = nn.ModuleList([
            DecoderBlock(d_model, n_heads, d_ff) for _ in range(n_layers)
        ])
        self.norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, tokens):
        B, S = tokens.shape
        x = self.tok_emb(tokens)

        # 因果掩码
        mask = torch.tril(torch.ones(S, S, device=tokens.device))

        for block in self.blocks:
            x = block(x, mask)

        return self.lm_head(self.norm(x))

# 自回归生成
@torch.no_grad()
def generate(model, prompt_ids, max_new_tokens=100, temperature=0.7):
    input_ids = prompt_ids.clone()

    for _ in range(max_new_tokens):
        logits = model(input_ids)[:, -1, :]
        logits = logits / temperature
        probs = torch.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)
        input_ids = torch.cat([input_ids, next_token], dim=1)

        if next_token.item() == model.config.eos_token_id:
            break

    return input_ids
```

```python
# 测试
if __name__ == "__main__":
    vocab_size = 32000
    d_model = 512
    n_heads = 8
    n_layers = 6
    d_ff = 2048

    model = GPTModel(vocab_size, d_model, n_heads, n_layers, d_ff)

    # 统计参数量
    total_params = sum(p.numel() for p in model.parameters())
    print(f"Total parameters: {total_params:,}")  # 约 50M

    # 前向传播
    tokens = torch.randint(0, vocab_size, (2, 10))
    logits = model(tokens)
    print(f"Input shape: {tokens.shape}")   # [2, 10]
    print(f"Output shape: {logits.shape}")  # [2, 10, 32000]
```

🎯 面试要点

- **Causal mask**: 只能看到之前的 token
- **Pre-Norm**: 先归一化再进子层 (更稳定)
- **RMSNorm + SwiGLU** 是现代 LLM 标配
- **RoPE 替代传统位置编码**
- **GPT/LLaMA/Mistral/Qwen 都是 decoder-only**
- **面试常问**: Decoder-only 和 Encoder-Decoder 的区别？为什么现代 LLM 都用 Decoder-only?
- **训练与推理共用一套权重**: 形式统一是 scaling 的前提 —— 训练是 shift + CE 一次算完，推理是逐 token 拼回去再跑一轮

多 Token 预测

Multi-Token Prediction

一次预测未来 n 个 token，同一份数据给出 n 倍训练信号

📌 原理与思想

核心概念

在主模型预测下一个 token 之外，串行接上若干 MTP 模块，每个模块用「上一层 MTP 的隐状态 + 第 `i+k` 个 token 的 embedding」去预测第 `i+1+k` 个 token。训练时提供更密集的监督信号，迫使隐状态包含更长程的

前瞻性；推理时这些模块可以直接当投机解码的 draft，几乎白送一个加速器。`Emb` 与 `lm_head` 和主模型共享，参数开销很小；代价是串行结构让模块 `k` 必须等模块 `k-1`，训练时还要多算 `k` 个模块的前向。

核心思想

在主干之外串行接上若干 MTP 模块，第 `k` 个模块预测第 `i+1+k` 个 token，让同一份数据提供 `k` 倍的监督信号。模块的输入是「上一层 MTP 的隐状态 + 第 `i+k` 个 token 的 embedding」，逼着隐状态携带更长程的信息。推理时这些模块天然就是一个 draft 模型，可以直接接投机解码 —— 一份结构两处收益。

算法步骤

1. 主干输出 `h⁰: [B, T, D]`，接 `lm_head` 预测 `x₂`，这是主损失。
2. 第 `k` 个模块：`h^k = M_k[RMSNorm(h^{k-1}) ; RMSNorm(Emb(x_{i+k}))]`，把上一层隐状态与目标 token 的 embedding 拼起来。
3. 每个模块各接 `lm_head` 预测 `x_{i+1+k}`，得到辅助损失。
4. 总损失 `L = L_main + (λ/D)·Σ_k Σ_i CE(...)`，`λ` 典型取 0.3（前 10T tokens），之后衰减到 0.1。

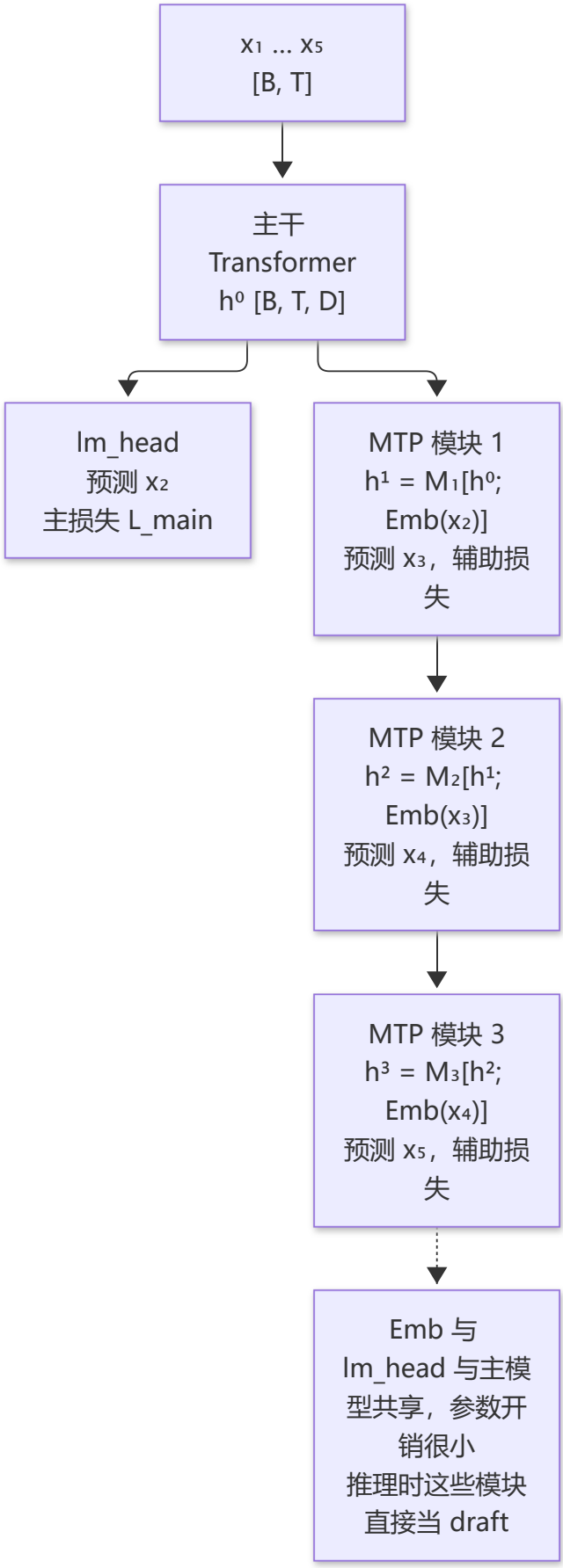
📐 核心公式

$$h_i^k = M_k\left[\mathrm{RMSNorm}\left(h_i^{k-1}\right);\ \mathrm{RMSNorm}(\mathrm{Emb}(x_{i+k}))\right], \qquad p_{i+k+1} = \mathrm{lm_head}\left(h_i^k\right)$$

$$\mathcal{L}_{\mathrm{MTP}} = \frac{\lambda}{D}\sum_k\sum_i \mathrm{CE}(p_{i+k+1},\ x_{i+1+k}), \qquad \mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{main}} + \mathcal{L}_{\mathrm{MTP}}$$

$$\lambda = 0.3\ (\text{first } 10\mathrm{T}\ \text{tokens}) \longrightarrow 0.1, \qquad \mathrm{Emb},\ \mathrm{lm_head}\ \text{shared with the main model}$$

📊 张量流程图


$x_1 \dots x_5$
[B, T]
主干
Transformer
h^0 [B, T, D]
lm_head
预测 x_2
主损失 L_main
MTP 模块 1
$h^1 = M_1[h^0; Emb(x_2)]$
预测 x_3，辅助损失
MTP 模块 2
$h^2 = M_2[h^1; Emb(x_3)]$
预测 x_4，辅助损失
MTP 模块 3
$h^3 = M_3[h^2; Emb(x_4)]$
预测 x_5，辅助损失
Emb 与 lm_head 与主模型共享，参数开销很小
推理时这些模块直接当 draft

💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class RMSNorm(nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(d))
        self.eps = eps

    def forward(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) * self.weight


class MTPModule(nn.Module):
    """一个 MTP 模块: 拼接上一层隐状态与下一个 token 的 embedding, 再过一个 Transformer 块"""

    def __init__(self, d_model, n_heads=8):
        super().__init__()
        self.norm_h = RMSNorm(d_model)
        self.norm_e = RMSNorm(d_model)
        self.proj = nn.Linear(2 * d_model, d_model, bias=False)   # 2d → d
        self.block = nn.TransformerEncoderLayer(
            d_model, n_heads, dim_feedforward=4 * d_model,
            batch_first=True, norm_first=True)

    def forward(self, h_prev, tok_emb):
        # h_prev:  [B, T, D] 上一层 MTP 的隐状态
        # tok_emb: [B, T, D] 第 i+k 个 token 的 embedding
        x = self.proj(torch.cat([self.norm_h(h_prev), self.norm_e(tok_emb)], dim=-1))
        return self.block(x)                                      # [B, T, D]


def mtp_loss(mtp_modules, h0, input_ids, embed, lm_head, num_heads=1, lam=0.3):
    """
    多 token 预测损失
    h0:        [B, T, D] 主模型最后一层隐状态
    input_ids: [B, T]    输入 token
    返回: (MTP 部分损失, 各层预测的 logits 列表)
    """
    total, h_prev, logits_list = 0.0, h0, []

    for k in range(1, num_heads + 1):
        if input_ids.size(1) - k <= 1:
            break
        target = input_ids[:, 1 + k:]             # 目标: x_{i+1+k}
        tok_emb = embed(input_ids[:, k:-1])       # 输入: x_{i+k}
        h_prev = mtp_modules[k - 1](h_prev[:, :-1], tok_emb)

        logits = lm_head(h_prev)                  # [B, T-k-1, V]
        logits_list.append(logits)
        # 用 sum 再统一除以 D, 把 k 个预测的损失归一到同一量级
        total = total + F.cross_entropy(
            logits.reshape(-1, logits.size(-1)), target.reshape(-1), reduction="sum")
```

```
    return lam * total / input_ids.size(1), logits_list


if __name__ == "__main__":
    torch.manual_seed(0)
    B, T, D, V = 2, 8, 32, 64

    embed = nn.Embedding(V, D)
    lm_head = nn.Linear(D, V, bias=False)
    mtp_modules = nn.ModuleList([MTPModule(D, 4) for _ in range(2)])

    input_ids = torch.randint(1, V, (B, T))
    h0 = torch.randn(B, T, D)

    loss, logits_list = mtp_loss(mtp_modules, h0, input_ids, embed, lm_head,
                                 num_heads=2, lam=0.3)
    print("MTP 损失:", round(loss.item(), 4))
    for k, lg in enumerate(logits_list, start=1):
        print(f"  第 {k} 层预测形状:", tuple(lg.shape), "← 每深一层少一个位置")

    loss.backward()
    print("梯度已回传:", mtp_modules[0].proj.weight.grad is not None)
```

🎯 面试要点

- **和「并行预测头」的区别**: Meta 的早期方案给每个未来位置一个独立头，彼此不依赖；DeepSeek-V3 的 MTP 是**串行**的，第 k 层吃第 k-1 层的隐状态，保留了因果链
- **为什么能提升效果**: 强迫隐状态编码更长程的信息，相当于给模型的「规划能力」加正则
- **参数开销很小**: embedding 和 lm_head 与主模型共享，每个 MTP 模块只有一个投影 + 一个 Transformer 块
- **注意偏移**: 模块的**输入**是 `x_{i+k}`，**目标**是 `x_{i+1+k}` —— 整体比主模型前移 k 步，别把两者搞反
- **为什么要除以 D**: 把 k 个预测的损失归一化回与主损失同一量级，否则 k 越大辅助损失越压过主损失
- **λ 为什么要衰减**: 训练后期主损失收敛，辅助信号的边际价值下降，过大反而干扰
- **推理时的双重身份**: 训练时是辅助损失，推理时是投机解码的 draft —— 一次前向能给多个候选 token
- **面试常问**: MTP 为什么用串行而不是并行头？推理时怎么把它变成加速手段？

📖 **基础** Basics　　2 题

激活函数

Activation Functions

ReLU / GELU / SiLU 及其梯度

📌 原理与思想

核心概念

激活函数引入非线性。ReLU 简单高效但有 dead neuron 问题；GELU 平滑更优（Transformer 常用）；SiLU/Swish 用于门控机制。`ReLU` 实现最便宜，`GELU`、`SiLU` 平滑、处处可导、训练更稳，代价是要算 exp 或 erf，比 `max` 贵一些。

核心思想

用一个逐元素的非线性函数把线性层的结果掰弯，让「多层」真正带来表达力 —— 没有激活函数时，多层线性变换叠起来仍然等价于单层线性变换。现代选择集中在平滑版 ReLU 上：负区间给一点非零梯度，避免神经元被永久关死。门控形式的 `x·σ(x)` 顺带把「通过多少」也变成可学的，这正是 SwiGLU 的基础。

算法步骤

1. `ReLU(x) = max(0, x)`：正区间导数恒为 1，负区间恒为 0。
2. `GELU(x) = x·Φ(x)`（`Φ` 是标准正态 CDF）：负区间仍有小梯度，是 ReLU 的平滑版。
3. `SiLU(x) = x·σ(x)`：`σ` 充当 0~1 的开关，负区间先降后回升，自带门控。
4. 三者都是逐元素作用，形状完全不变，可以随意替换。
5. 梯度的差别只有一档：`ReLU'` 只有 0 / 1 两档，在 0 处不连续；`GELU'`、`SiLU'` 处处可导 —— 这正是平滑版训练更稳的原因。

📐 核心公式

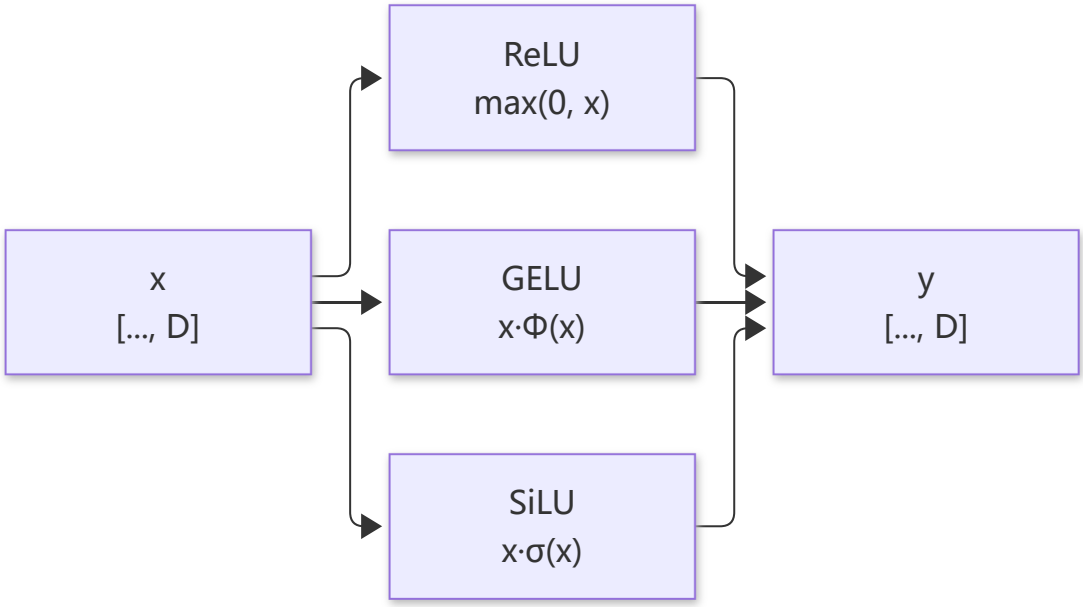
$$\mathrm{ReLU}(x) = \max(0, x), \qquad \mathrm{ReLU}'(x) = (x > 0)$$

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \Phi = \text{standard normal CDF}$$

$$\mathrm{SiLU}(x) = x\,\sigma(x), \qquad \mathrm{SiLU}'(x) = \mathrm{SiLU}(x) + \sigma(x)\,(1 - \mathrm{SiLU}(x))$$

$$\mathrm{Swish} \equiv \mathrm{SiLU}$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn.functional as F

# ReLU
def relu(x):
    return torch.clamp(x, min=0)    # 或 F.relu(x)

# GELU (近似版本更快)
def gelu(x):
    return 0.5 * x * (1 + torch.tanh(
        (2/3.14159)**0.5 * (x + 0.044715 * x**3)
    ))
    # 或直接用 F.gelu(x)

# SiLU (Swish)
def silu(x):
    return x * torch.sigmoid(x)     # 或 F.silu(x)

# 梯度对比
def check_gradients():
    x = torch.linspace(-3, 3, 100, requires_grad=True)

    # ReLU
    y_relu = relu(x).sum()
    y_relu.backward()
    print("ReLU grad:", x.grad[:5])     # [0, 0, 0, ..., 1, 1]
    x.grad.zero_()

    # GELU
    y_gelu = gelu(x).sum()
    y_gelu.backward()
```

```
    print("GELU grad:", x.grad[:5])     # 平滑过渡
    x.grad.zero_()

    # SiLU
    y_silu = silu(x).sum()
    y_silu.backward()
    print("SiLU grad:", x.grad[:5])     # 平滑 + 门控

# 在 Transformer 中使用
class FFN_ReLU(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.w2(F.relu(self.w1(x)))

class FFN_GELU(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.w2(F.gelu(self.w1(x)))

class FFN_SwiGLU(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w_gate = nn.Linear(d_model, d_ff, bias=False)
        self.w_up = nn.Linear(d_model, d_ff, bias=False)
        self.w_down = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        gate = F.silu(self.w_gate(x))
        up = self.w_up(x)
        return self.w_down(gate * up)

# 测试
if __name__ == "__main__":
    check_gradients()

    # 可视化
    import matplotlib.pyplot as plt

    x = torch.linspace(-3, 3, 100)

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.plot(x.numpy(), relu(x).detach().numpy())
    plt.title('ReLU')
    plt.grid(True)

    plt.subplot(1, 3, 2)
    plt.plot(x.numpy(), gelu(x).detach().numpy())
    plt.title('GELU')
    plt.grid(True)
```

```
    plt.subplot(1, 3, 3)
    plt.plot(x.numpy(), silu(x).detach().numpy())
    plt.title('SiLU/Swish')
    plt.grid(True)

    plt.tight_layout()
    plt.savefig('activation_functions.png')
    print("Plot saved to activation_functions.png")
```

🎯 面试要点

- **ReLU**: 简单高效，但有 dead neuron (负区间梯度=0)
- **GELU**: 平滑版 ReLU，BERT/GPT 使用
- **SiLU = Swish**: 用于 SwiGLU 门控机制
- **面试常问**: 各激活函数的梯度和使用场景?
- **使用场景**: ReLU 是原始 Transformer FFN 的标配；GELU 用于 BERT、GPT-2/3；SiLU 用于 SwiGLU（LLaMA、PaLM）
- **对比**: ReLU vs GELU vs SiLU 的优缺点?

梯度与反向传播

Gradient & Backpropagation

链式法则，深度学习的基石

📌 原理与思想

核心概念

反向传播利用链式法则从输出向输入逐层计算梯度。计算图的前向传播保存中间变量，反向传播利用这些变量计算梯度。相比手动推导梯度，反向传播自动计算且高效（一次前向 + 一次反向），是深度学习训练的基础。

核心思想

链式法则：$\partial L/\partial x = \partial L/\partial y \cdot \partial y/\partial x$。前向传播时保存中间变量（计算图），反向传播时利用这些变量和链式法则逐层计算梯度。

算法步骤

1. 前向传播：逐层计算并保存中间变量
2. 计算损失：L = loss(y_pred, y_true)
3. 反向传播：从输出向输入逐层计算梯度
4. 常见梯度：$y=Wx \rightarrow \partial L/\partial W = \partial L/\partial y \cdot x^T$；ReLU $\rightarrow \partial L/\partial x = \partial L/\partial y \cdot (x>0)$

5. 参数更新：θ = θ - lr · ∂L/∂θ

📐 核心公式

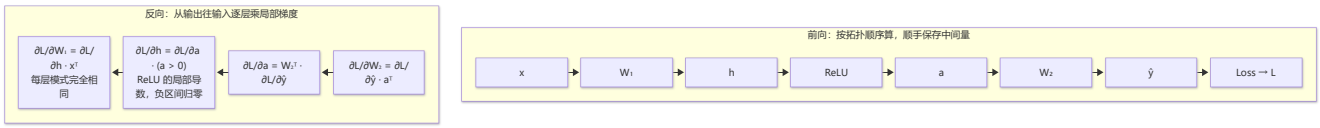
$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial x}$$

$$y = Wx: \quad \frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial y} x^\top, \quad \frac{\partial \mathcal{L}}{\partial x} = W^\top \frac{\partial \mathcal{L}}{\partial y}$$

$$y = \mathrm{ReLU}(x): \quad \frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \odot (x > 0)$$

$$y = \mathrm{softmax}(z): \quad \frac{\partial \mathcal{L}}{\partial z} = y - \mathrm{one_hot}(\mathrm{target}) \quad (\text{with CE})$$

📊 张量流程图



💻 代码实现

```python
import torch

# 手动实现简单两层网络的反向传播
class TwoLayerNet:
    def __init__(self, d_in, d_hidden, d_out):
        self.W1 = torch.randn(d_in, d_hidden) * 0.01
        self.W2 = torch.randn(d_hidden, d_out) * 0.01

    def forward(self, x):
        self.x = x
        self.h = x @ self.W1              # 线性
        self.a = torch.relu(self.h)       # 激活
        self.y = self.a @ self.W2         # 输出
        return self.y

    def backward(self, y_true, lr=0.01):
        # Loss = MSE = mean((y - y_true)²)
        dy = 2 * (self.y - y_true) / y_true.size(0)  # ∂L/∂y

        # ∂L/∂W2 = aᵀ · dy
        dW2 = self.a.T @ dy
        da = dy @ self.W2.T

        # ReLU 梯度
        dh = da * (self.h > 0).float()

        # ∂L/∂W1 = xᵀ · dh
        dW1 = self.x.T @ dh
```

```
        # 更新
        self.W1 -= lr * dW1
        self.W2 -= lr * dW2

# 使用 PyTorch 自动求导
class TwoLayerNetAuto:
    def __init__(self, d_in, d_hidden, d_out):
        self.W1 = torch.randn(d_in, d_hidden, requires_grad=True) * 0.01
        self.W2 = torch.randn(d_hidden, d_out, requires_grad=True) * 0.01

    def forward(self, x):
        h = x @ self.W1
        a = torch.relu(h)
        y = a @ self.W2
        return y

# 测试
if __name__ == "__main__":
    # 手动反向传播
    net = TwoLayerNet(10, 20, 5)
    x = torch.randn(4, 10)
    y_true = torch.randn(4, 5)

    y_pred = net.forward(x)
    loss = ((y_pred - y_true) ** 2).mean()
    print(f"Loss before: {loss.item():.4f}")

    net.backward(y_true, lr=0.01)

    y_pred = net.forward(x)
    loss = ((y_pred - y_true) ** 2).mean()
    print(f"Loss after: {loss.item():.4f}")

    # PyTorch 自动求导
    net_auto = TwoLayerNetAuto(10, 20, 5)
    y_pred = net_auto.forward(x)
    loss = ((y_pred - y_true) ** 2).mean()
    loss.backward()

    print(f"\nW1 grad shape: {net_auto.W1.grad.shape}")
    print(f"W2 grad shape: {net_auto.W2.grad.shape}")
```

🎯 面试要点

- **链式法则是反向传播的数学基础**
- **前向保存中间变量，反向利用它们计算梯度**
- **矩阵求导**: $\partial(xW)/\partial W = x^T \cdot \partial L/\partial y$
- **ReLU 梯度**: 正区间为 1，负区间为 0
- **Softmax + CE 的梯度特别简洁**: y - one_hot
- **面试常问**: 反向传播的原理？链式法则如何应用？
- 一次前向 + 一次反向就得到全部参数的梯度，复杂度与前向同阶；代价是中间激活要留到反向，显存随层数线性增长